

School of Electrical and Computer Engineering
Intelligent Systems Laboratory

Latency Optimized Node and Traffic Scheduling in Kubernetes

Master's Thesis

of

Apostolos Lazidis

Examination Committee:

Professor Euripides G.M. Petrakis (Supervisor)

Professor Michail G. Lagoudakis

Associate Professor Vasileios Samoladas

Chania, February 2026

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εργαστήριο Προγραμματισμού και Τεχνολογίας Ευφυών Υπολογιστικών
Συστημάτων

Βέλτιστη ανάθεση υπολογιστικών κόμβων και δρομολόγηση επικοινωνίας με επίγνωση καθυστερήσης στο Kubernetes

Μεταπτυχιακή Εργασία

του

Απόστολου Λαζίδη

Εξεταστική Επιτροπή:

Καθηγητής Ευριπίδης Πετράκης (Επιστημονικά Υπεύθυνος)

Καθηγητής Μιχαήλ Λαγουδάκης

Αναπληρωτής Καθηγητής Βασίλειος Σαμολαδάς

Χανιά, Φεβρουάριος 2026

Abstract

Containerized microservices have become the de facto standard for building modern applications providing improved fault isolation, scalability and resilience. To ensure fault-tolerance and maintain good performance under fluctuating workloads, microservices should be scaled horizontally by deploying multiple replicas. High availability is achieved by placing these replicas across multiple servers. Kubernetes is a powerful orchestrator tool that manages the lifecycle of containerized applications, providing features for automated scaling, self-healing, service discovery and seamless deployment. The Horizontal Pod Autoscaler (HPA) is the Kubernetes default tool for scaling the containerized microservices, known as Pods. Pods often communicate with each other in applications. During scaling, Pods should ideally be scheduled based on their communication frequency to minimize cross-Node communication latency. Reducing cross-Node communication can significantly improve response time and minimize cloud costs, when the Kubernetes is deployed on a cloud provider. However, HPA relies on the default Kubernetes Scheduler to assign replica Pods to Nodes, which can sometimes lead to sub-optimal placement of replicas across Nodes, because it does not rely on Pods' communication patterns (i.e., inter/intra-node or inter/intra-cluster communication). Aside from scaling, Kubernetes by default relies on iptables to route traffic to appropriate Pods. However, iptables uses a randomized algorithm and does not consider Node-to-Node latency, Pods' resource usage or the number of available replica Pods, resulting in overloaded Pods and long delays, when requests are sent to remote Nodes. To address the problem of Kubernetes replica Pod placement and load-balancing, we introduce in this Master's thesis StornX. StornX is a Kubernetes Pod autoscaler and traffic scheduler that optimized spreads the Pod replicas within a cluster across Availability Zones (AZ), optimizes efficiency by co-locating additional replicas, and dynamically manages traffic distribution using Istio Service Mesh locality weights by directing requests to co-located replicas, while also rerouting traffic away from Pods that have reached their threshold limits toward replicas with available resources. StornX is tested on AWS cloud environment with two microservice-based applications. The experimental results reveal that StornX reduces the response time, resource usage, number of replica Pods and reduces the cost in stress and load test scenarios. Furthermore, Availability Zone failure experiments show that StornX recovers faster and with lower performance degradation by dynamically redistributing traffic among the remaining replicas.

Περίληψη

Οι μικροϋπηρεσίες σε containers αποτελούν το πρότυπο για την ανάπτυξη σύγχρονων εφαρμογών, προσφέροντας βελτιωμένη απομόνωση σφαλμάτων, δυνατότητα κλιμάκωσης και υψηλή ανθεκτικότητα. Για να διασφαλιστεί η ανοχή σε σφάλματα, οι μικροϋπηρεσίες πρέπει να κλιμακώνονται οριζόντια με την ανάπτυξη πολλαπλών αντιγράφων, ενώ η υψηλή διαθεσιμότητα επιτυγχάνεται με την τοποθέτησή τους σε πολλαπλούς εξυπηρετητές. Το Kubernetes αποτελεί ένα ισχυρό εργαλείο ενορχήστρωσης που διαχειρίζεται τον κύκλο ζωής των εφαρμογών σε containers, παρέχοντας λειτουργίες αυτόματης κλιμάκωσης, αυτόματης αποκατάστασης και ανακάλυψης υπηρεσιών. Ο Horizontal Pod Autoscaler (HPA) είναι το εργαλείο του Kubernetes για την αναπαραγωγή αντιγράφων των μικροϋπηρεσιών σε containers (Pods). Τα Pods σε μια εφαρμογή επικοινωνούν μεταξύ τους. Κατά την κλιμάκωση, ιδανικά πρέπει να τοποθετούνται βάσει της συχνότητας της μεταξύ τους επικοινωνίας, ώστε να ελαχιστοποιείται η καθυστέρηση μεταξύ των κόμβων (Nodes). Η μείωση της επικοινωνίας μεταξύ απομακρυσμένων κόμβων μπορεί να βελτιώσει σημαντικά τον χρόνο απόκρισης και να μειώσει το κόστος υποδομής, όταν το Kubernetes τρέχει σε έναν πάροχο cloud. Ωστόσο, ο HPA βασίζεται στον προεπιλεγμένο Kubernetes Scheduler για την τοποθέτηση των νέων Pods στους κόμβους, κάτι που μπορεί να οδηγήσει σε μη βέλτιστη κατανομή, καθώς ο Scheduler δεν λαμβάνει υπόψη τα πρότυπα επικοινωνίας των Pods (inter/intra-node ή inter/intra-cluster communication). Πέραν της κλιμάκωσης, το Kubernetes βασίζεται εξ ορισμού στα iptables για τη δρομολόγηση των αιτημάτων προς τα κατάλληλα Pods. Ωστόσο, τα iptables χρησιμοποιούν έναν αλγόριθμο τυχαιότητας και δεν λαμβάνουν υπόψη την καθυστέρηση μεταξύ κόμβων, τη χρήση πόρων των Pods ή τον αριθμό διαθέσιμων αντιγράφων, οδηγώντας σε υπερφορτωμένα Pods και μεγάλες καθυστερήσεις, όταν τα αιτήματα εξυπηρετούνται από απομακρυσμένους κόμβους. Για την αντιμετώπιση του προβλήματος της τοποθέτησης των Pods και της αποτελεσματικής κατανομής φορτίου στο Kubernetes, παρουσιάζουμε το StornX. Το StornX είναι ένας μηχανισμός αυτόματης κλιμάκωσης αντιγράφων και δρομολόγησης αιτημάτων που κατανέμει βέλτιστα τα αντίγραφα Pod εντός του Kubernetes cluster και μεταξύ των ζωνών διαθεσιμότητας στην περίπτωση του υπολογιστικού νέφους, βελτιώνει την απόδοση μέσω συντοποθέτησης επιπλέον αντιγράφων και διαχειρίζεται δυναμικά την κατανομή αιτημάτων χρησιμοποιώντας τα βάρη τοπολογίας του Istio Service Mesh. Δρομολογεί τα αιτήματα σε συντοποθετημένα αντίγραφα, απομακρύνοντάς τα από Pods που φτάνουν τα ορισμένα όρια φόρτου, στέλνοντάς τα σε Pods με διαθέσιμους πόρους. Τα πειράματα στο StornX έγιναν στο AWS cloud χρησιμοποιώντας δύο εφαρμογές. Τα πειραματικά αποτελέσματα δείχνουν ότι το StornX μειώνει τον χρόνο απόκρισης, τη χρήση πόρων, τον αριθμό των αντιγράφων και το συνολικό κόστος σε σενάρια stress και load testing. Επιπλέον, έγιναν πειράματα όπου «πέφτει» μία ζώνη διαθεσιμότητας δείχνοντας ότι το StornX ανακάμπτει πιο γρήγορα και διατηρεί καλύτερη απόδοση, κατανέμοντας την κίνηση στα διαθέσιμα αντίγραφα.

Acknowledgments

I am sincerely grateful for the invaluable guidance and unwavering support provided by my supervisor, Professor Euripides Petrakis, throughout the entire journey of conducting my Master's Thesis. His extensive expertise, profound knowledge, and unwavering dedication have played a crucial role in shaping the trajectory of my research and steering me toward achieving significant outcomes.

Contents

1	Introduction	8
1.1	Problem Definition	8
1.2	Scope of Thesis	8
1.3	Thesis Outline	9
2	Related work and Contribution	10
2.1	Related work	10
2.2	Contributions of the work	13
3	Background	15
3.1	Kubernetes	15
3.1.1	Related Components	15
3.1.2	Control Plane	16
3.1.3	Worker Nodes	17
3.1.3.1	Kube-proxy	17
3.1.4	Kube-scheduler	18
3.1.5	Kubernetes API Resources	18
3.1.5.1	Namespaces	19
3.1.5.2	Deployments and StatefulSets	20
3.1.6	Horizontal Pod Autoscaler	21
3.2	Service Mesh	22
3.2.1	Istio	23
3.3	Prometheus	24
3.4	Regions and Availability Zones	25
4	System Overview	27
4.1	Kube-netlag	30
4.2	System Prerequisites	30
4.3	StornX Scope	31
4.4	System Architecture	32
4.4.1	OptiScaler	36
4.4.1.1	Efficient placement of replicas across Nodes and Availability Zones	38
4.4.1.2	Efficient communication-aware placement	42
4.4.2	OptiBalancer	44
4.4.2.1	OptiBalancer Operational Cases	46
5	Experimental Results	51
5.1	Cloud Infrastructure	51
5.2	Benchmark Applications, Workloads, and Evaluation Tools	52
5.2.1	Benchmark Applications	53
5.2.2	Workload Generation	56
5.2.3	Infrastructure Cost Analysis Tool	57
5.3	Results	57
5.3.1	Mapping concurrent users to request rate	58
5.3.2	Stress Test Comparison	58
5.3.2.1	Response time	59
5.3.2.2	CPU utilization	59
5.3.2.3	Cost &Egress Traffic	60
5.3.2.4	Replicas	61
5.3.2.5	Summary of Stress Test Comparison	61
5.3.3	Load Test Comparison	62
5.3.3.1	Gaussian workload profile	62
5.3.3.2	Response time	63
5.3.3.3	CPU	64
5.3.3.4	Cost &Egress Traffic	64

5.3.3.5	Replicas	65
5.3.3.6	Summary of Load Test Comparison	66
5.3.4	High Availability Under Availability Zone Failure	67
5.3.4.1	Experimental Setup	67
5.3.4.2	Online Boutique Results	67
5.3.4.3	OpenTelemetry Demo Results	68
5.3.4.4	Summary of High Availability Evaluation	69
5.3.5	Overall Summary of Experimental Results	70
5.4	StornX Performance	71
6	Conclusion	72

List of Figures

1	Kubernetes architecture	16
2	Namespaces	20
3	Service Mesh Architecture	23
4	Istio Architecture	24
5	Regions & Availability Zones on AWS	25
6	Single-zone & single-node placement	27
7	Single-zone & multi-node placement	27
8	Multi-zone & multi-node placement	28
9	Architecture Overview	31
10	Node replica pods overloaded	35
11	StornX Architecture	36
12	Phase A: spreading the first replicas across different Availability Zones for fault-tolerance and high-availability	39
13	Phase B: co-locating replicas on existing Nodes to reduce communication overhead and costs	40
14	Replica Pod placement	44
15	Case 1 - The replicated Pod acts as the Front-end service (UI).	47
16	Case 2 - The replicated Pod has one or more upstream components.	47
17	Case 3 - The replicated Pod has multiple upstream components.	48
18	Overview of the AWS infrastructure used for the experiments.	52
19	Online-boutique architecture	53
20	OpenTelemetry Demo architecture	55
21	Stress test P99 latency results comparing StornX, Istio Random and OptTraffic	59
22	Stress test Replicas count & CPU utilization results comparing StornX, Istio Random and OptTraffic	59
23	Stress test Cost & Egress Traffic results comparing StornX, Istio Random and OptTraffic.	60
24	Stress test Replicas count results comparing StornX, Istio Random and OptTraffic	61
25	Gaussian-shaped workload used in the load tests, showing the evolution of the number of concurrent users over time for both benchmark applications.	62
26	Estimated RPS over time during the Gaussian load test, derived from concurrent users using the Locust workload model.	63
27	Load test P99 latency results comparing StornX, Istio Random and OptTraffic	63
28	Load test CPU results comparing StornX, Istio Random and OptTraffic	64
29	Load test Cost & Egress Traffic results comparing StornX, Istio Random and OptTraffic.	65
30	Load test Replicas count results comparing StornX, Istio Random and OptTraffic	66
31	P99 Latency and Error Rate results comparing StornX, Istio Random and OptTraffic on an AZ failure for Online Boutique	68
32	P99 Latency and Error Rate results comparing StornX, Istio Random and OptTraffic on an AZ failure, for OpenTelemetry	69

1 Introduction

1.1 Problem Definition

Microservices have become a popular domain-driven design pattern for modern applications. They are loosely coupled services, which can be developed, deployed and maintained independently. Each of these services is responsible for executing a part of the application logic or even a simple task within an application. This architecture design pattern offers advantages over traditional monolithic applications, as it improves scalability, resilience and maintenance [1]. Nowadays, the most efficient way to deploy applications is through containers. Containers are lightweight, isolated environments for packaging microservices that can be deployed on any operating system. Containerization provides consistency across environments, fast setup and startup times and efficient resource usage. To ensure fault-tolerance, microservices should be scaled horizontally by deploying multiple replicas. To maintain the lifecycle of containers and their replicas, an orchestration platform is required.

Kubernetes is the most widely adopted container orchestrator, offering automatic deployment, scaling, load-balancing, service-discovery, and self-healing. In Kubernetes, a Pod is a group of one or more containers, a Node is the server where the Pods run, and a cluster is a group of Nodes. In a Kubernetes cluster, multiple components of a microservice application, which communicate with each other, are scheduled on different Nodes, but network latency between the Nodes, can lead to higher response times and consequently a worse user experience, especially in cloud environments, where the Nodes can be located across multiple Availability Zones (AZ). Availability Zones are isolated data center locations within a cloud region, typically separated to reduce the risk of a single event affecting multiple AZs. In cloud environments, the high availability and fault-tolerance of a microservice architecture are achieved by deploying replica components across multiple Availability Zones, since a failure in one AZ does not impact the others. In this work, we consider applications deployed within a single Kubernetes cluster that spans multiple Availability Zones within the same cloud region. This is a common deployment model supported by major cloud providers, where a single cluster can include worker Nodes distributed across AZs to improve availability and resilience.

Application performance can also be degraded under high traffic loads, placing additional stress on Pod resources. Kubernetes through its internal mechanisms can deploy multiple replica Pods across different Nodes to handle these high-traffic scenarios. However, Kubernetes default scheduler places Pods based on Node resources and does not take into account Pod communications links. If all replica Pods are placed on a single Node, the redundancy mechanisms will be violated, meaning that if the Node fails or restarts unexpectedly, all Pods of the service-oriented architecture (SOA) will attempt to schedule on other Nodes, which will increase downtime of the whole application. Conversely, placing replica Pods across different Nodes introduces higher costs, when the Kubernetes cluster is hosted by a cloud provider. Cloud providers, like Amazon Web Services (AWS), Microsoft Azure and Google Cloud Platform (GCP), charge the traffic that occurs between Nodes. This raises the issue of how to optimally "place" replica Pods across availability zones so that response time and costs are reduced, while high availability and fault-tolerance are preserved. Moreover, Kubernetes by default forwards traffic to Pods and their replicas using a random distribution algorithm, without considering Pod placement. As a result, traffic may be sent across Nodes, even when local replicas are available, leading to unnecessary cross-Node communication and higher costs.

Controlling the scheduling process of replica Pods and placing them based on communication links and the underlying cloud infrastructure topology, while also controlling traffic distribution, is essential for balancing application performance, operational costs, and fault-tolerance. Achieving this requires a carefully designed scheduling approach and a load-balancing algorithm that are aware of the network topology and the workload characteristics of the application.

1.2 Scope of Thesis

In this work, we attempt to address the problem of replica Pod placement across Kubernetes Nodes in cloud environments, along with the challenge of load-balancing among replica Pods. To address this, we propose StornX, a Kubernetes autoscaler for horizontal scaling that schedules

new replica Pods onto specific Nodes based on Pod communication links, while ensuring high availability and fault-tolerance.

StornX introduces two core mechanisms. First, Node Scheduling optimized spread the initial replica Pods across multiple Availability Zones (AZs) to ensure resilience against zone-level failures. Having spread Pods to multiple availability zones, later (new) replicas are co-located to improve communication efficiency and reduce cross-zone traffic costs. Second, Traffic Scheduling leverages Istio Service Mesh locality weights to implement a weighted load-balancing algorithm that prioritizes routing traffic to co-located replicas whenever possible, while dynamically adjusting routing based on factors, such as Node-to-Node latency, CPU utilization, and memory usage. This ensures that traffic is redirected away from overloaded Pods toward underutilized replicas.

All experimental evaluations were conducted on AWS using two microservice-based applications. The results demonstrate that StornX load-balancing algorithm, combined with horizontal scaling, improves inter-Node communication efficiency and reduces unnecessary replica creation by dynamically adjusting Pod resources to prevent overload. Finally, by decreasing replica counts, reduces Node’s resource usage, which in turn reduces the total number of Nodes that are required and thereby lower costs.

Beyond performance and cost efficiency, this thesis also evaluates the behavior of StornX under Availability Zone failures. Controlled experiments are conducted in which all Nodes of a single Availability Zone are removed under steady workload conditions, allowing the assessment of recovery behavior, latency degradation, and error rates. This evaluation provides insight into how adaptive placement and traffic management affect system availability during large-scale infrastructure disruptions.

1.3 Thesis Outline

This thesis is organized as follows. Chapter 2 reviews the related work in the areas of Kubernetes autoscaling, communication-aware scheduling, and traffic management, highlighting the limitations of existing approaches and motivating the need for a unified framework. Chapter 3 introduces the necessary background concepts, including Kubernetes architecture, autoscaling mechanisms, scheduling strategies, and service mesh-based traffic management, which form the foundation of the proposed system. Chapter 4 presents the design and architecture of StornX, describing its overall system model and detailing its two core components, OptiScaler for autoscaling and replica placement, and OptiBalancer for metric-aware traffic distribution. Chapter 5 provides the experimental evaluation, outlining the cloud infrastructure, benchmark applications, workload models, and evaluation metrics, and comparing StornX with baseline approaches under stress tests, load tests, and Availability Zone failure scenarios. Finally, Chapter 6 concludes the thesis by summarizing the main contributions, discussing limitations, and outlining directions for future research.

2 Related work and Contribution

2.1 Related work

Wojciechowski et al. (2021) [2] propose NetMARKS, a network metrics-aware scheduler extender for Kubernetes that leverages Istio service mesh monitoring to handle Pod placement. Instead of replacing the default scheduler, NetMARKS extends it with a scoring mechanism that co-locates Pods with their high-traffic communication links. Experimental results showed that NetMARKS significantly reduced application response times and inter-Node bandwidth consumption.

Wu et al. (2021) [3] propose Blender, a traffic-aware container placement approach that models inter-container communication as a Zipf-like distribution to optimize placement decisions. Blender first groups containers into blocks based on their communication intensity. By co-locating containers that frequently communicate within the same block, Blender minimizes inter-block traffic, while also improving load balancing by grouping containers according to their resource demands and distributing them across multiple physical machines. The experimental results showed that it significantly reduces communication traffic compared to existing placement strategies such as SBP and CA-WFD and achieves more balanced resource utilization.

Zhou et al. (2022) [4] propose an adaptive traffic management scheme for microservices that extends Istio with DTMA (Dynamic Traffic Management Assistant), a dynamic plug-in that continuously adjusts traffic distribution policies in response to workload changes. Unlike Istio’s static round-robin or least-connection strategies, DTMA leverages global load information from Nodes and service instances to update flow weights in real time and selects the suitable load-balancing strategy. Experimental results showed that the algorithm significantly reduced response times while maintaining stable load across Nodes compared to Istio’s default configuration.

Li et al. (2023) [5] propose MOTAS, a topology-aware scheduling framework for microservice applications in cloud environments, which leverages both microservice and cluster topologies to optimize network overhead and resource utilization. MOTAS replaces the default Kubernetes scheduler with a topology-aware heuristic, using a graph-mapping algorithm that places services with strong dependencies closer together in the cluster while ensuring balanced resource usage. The framework was evaluated against the default Kubernetes scheduler, showing effective cluster resource utilization, reduced end-to-end latency, and improved throughput. However, the authors note that their experiments were conducted on a limited-scale testbed, which does not align a production cloud environments, and they suggest further investigation in future research.

Petrakis et al. (2024) [6] introduce ModSoft-HP, a custom Kubernetes scheduler that applies fuzzy graph clustering for microservice placement, followed by heuristic packing to optimize Node utilization. Unlike hard clustering methods, ModSoft-HP not only clusters services with high communication affinity so they are co-located on the same Kubernetes Node, but also enables replicas of heavily utilized microservices to be placed across different Nodes. This dual strategy reduces inter-Node communication, improves latency, and balances workloads more effectively, while also enhancing resilience. The framework integrates with Kubernetes through Istio, Prometheus, and Kiali to monitor affinities between services, and it automatically updates deployments based on clustering results. Evaluated on Google Cloud Platform (GCP) with Google’s Online Boutique eShop and the iXen IoT architecture, ModSoft-HP achieved reductions in egress traffic, improved response times, and lower hosting costs compared to the default Kubernetes scheduler. Moreover, experiments showed that ModSoft-HP complemented the Horizontal Pod Autoscaler (HPA), providing better initial placements that allowed HPA to adapt more effectively under load, thereby consistently outperforming both the default scheduler and hard clustering approaches.

Prountzos et al. (2024) [7] propose DeFog, an innovative dynamic microservice placement and load-balancing framework for hybrid Cloud-Fog-Edge infrastructures that minimizes applications response times while balancing workloads across clusters. DeFog periodically monitors Kubernetes metrics to decide when to replicate or redeploy services, applying algorithms such as Least Frequently Used (LFU), Response Latency-based Service Deployment (RLSD), and their RAM-aware variants. In addition, it leverages the Linkerd service mesh TrafficSplit feature to redirect traffic across clusters once utilization thresholds are exceeded, preventing overload and reducing latency. The framework was evaluated on Google Cloud Platform (GCP) with real

multi-cluster K3s deployments running Google’s Online Boutique and the iXen IoT platform. Experimental results showed that RLSD consistently reduced response times compared to LFU, while the RAM-aware variants offered better performance under resource constraints.

Datta et al. (2024) [8] present a replication scheduling strategy for microservices aimed at improving fault-tolerance, reducing network traffic, and lowering latency in cloud environments. Their approach creates replicas of existing services and places them near the services with the highest communication dependencies in order to reduce communication costs, using a modified Particle Swarm Optimization (PSO) algorithm to select candidate machines and a modified Round Robin (RR) method to balance them across physical Nodes. While the authors assume Kubernetes as the target platform for deploying and managing replicas, the evaluation was conducted in a simulation environment using Alibaba and Google cluster datasets, focusing on metrics such as network traffic, latency, and load balancing across physical machines.

At [9] Zhu et al. (2023) propose OptTraffic, a network-aware scheduling system for multi-replica microservices that minimizes cross-machine communication and improves latency. The framework introduces a lightweight monitoring mechanism that estimates traffic between container pairs, applying a local-first allocation policy that prioritizes intra-Node communication, and a dynamic scheduling strategy that redistributes replicas when imbalances occur. OptTraffic was implemented on Kubernetes and evaluated with both benchmarks and real microservice applications. The results show that it significantly reduces cross-machine traffic and tail latency while maintaining balanced resource utilization across machines.

Park et al. (2025) [10] introduce a latency-aware scheduling framework that combines Latency-Aware Replica Placement (LARP) and Latency-Aware Traffic Allocation (LATA). LARP places replicas on Nodes with sufficient resources and low inter-Node latency, while LATA formulates traffic allocation as a minimum-cost flow problem to distribute traffic efficiently across replicas. All experimental results were conducted on Kubernetes and demonstrate that the approach significantly reduces average response times and tail latency compared to default Kubernetes, Localization, and OptTraffic.

The authors in [9] and [10] focuses explicitly on multi-replica microservices, addressing both replica placement and traffic distribution. OptTraffic achieves load balancing by reallocating traffic and migrating replicas across Nodes based only on upstream and downstream replica counts. On the traffic distribution side, its main limitation is that allocation remains static as long as the number of replicas does not change, ignoring real-time factors such as resource utilization and network latency.

Additionally, OptTraffic exchanges or reschedule Pods when an imbalance of replicas occurs. However, under very high workloads, moving replicas between Nodes is risky because Pods may take several minutes to become ready to handle traffic. During this time, resources remain allocated until the exchange is completed and the initial Pod is deleted, which may leave the cluster Node’s without enough resources to schedule new replicas. For this reason, we consider the initial placement to be the best strategy under heavy workloads. On the other hand, the author at [10] propose LARP and LATA which unlike OptTraffic use latency metrics to place replicas on low-latency Nodes and allocate traffic through a minimum-cost flow problem to reduce communication overhead.

The above works improve to highly reduce the response times and tail latency between microservices compared to the Localization and default Kubernetes load balancing algorithm, which uses a randomize policy [11] to distribute the traffic. While the two above studies demonstrate how replica placement and traffic allocation can reduce response times and tail latency, they do not evaluate whether these strategies interact safely with autoscalers, nor do they validate their approaches in real cloud environments.

It is important to note that, except for DeFog, which targets hybrid multi-cluster Kubernetes Cloud-Fog-Edge environments, all the reviewed works focus on single-cluster Kubernetes environments.

The following table provides a comparative overview of the microservice placement and traffic-management approaches discussed in this section. Each entry is categorized according to the core technique or model used (e.g., graph-based or heuristic methods), the placement behavior (static or adaptive), the type of load balancing applied (static or adaptive), and whether the approach incorporates fault-tolerance or high-availability mechanisms. The platform column

indicates where each method was evaluated, such as simulated environments, single or multi-cluster Kubernetes deployments, or specific cloud environment. The final entry represents our proposed method.

Ref.	Technique / Model	Placement Type	Load Balancing Type	Fault tolerance & High Availability	Platform
[2] NetMARKS (Wojciechowski et al.)	Graph (Directed weighted communication graph and heuristic scoring based on real-time traffic affinity)	Adaptive	Static	Not addressed	Kubernetes
[3] Blender (Wu et al.)	Graph (Traffic-aware block placement)	Adaptive	Static	Not addressed	Simulation environment (Container-CloudSim)
[4] DTMA (Zhou et al.)	Adaptive traffic management with traffic splitting and updates routing rules based on load	No placement mechanism	Adaptive with dynamic policy selection	Not addressed	Kubernetes
[5] MOTAS (Li et al.)	Graph (Topology-aware mapping using heuristic placement and degradation-aware rescheduling)	Adaptive	Static	Not addressed	Kubernetes
[6] ModSoft-HP (Petrakis et al.)	Clustering (Fuzzy clustering with heuristic packing)	Static	Static	Partial (multiple replicas may be placed across nodes, improving resilience but no availability-zone awareness)	GCP Google Kubernetes Engine (GKE)
[7] DeFog (Prountzos et al.)	Dynamic multi-cluster placement with adaptive policy switching and resource-aware traffic split	Adaptive	Adaptive (TrafficSplit-based routing & policy switching based on resource metrics)	Yes (multi-cluster failover capability)	Kubernetes (K3s multi-cluster)
[8] PSO and RR-Based Scheduling Strategy (Datta et al.)	Heuristic (Particle Swarm Optimization & Round Robin scheduling)	Adaptive	Static	Partial (Replica placement across nodes but no availability-zone awareness)	Simulation environment (Alibaba)
[9] OptTraffic (Zhu et al.)	Graph (Directed Acyclic Graph; traffic scheduling and local-first allocation)	Adaptive	Adaptive locality-based routing based on replica distribution	Partial (balances replicas across nodes but no availability-zone awareness)	Kubernetes
[10] LARP + LATA (Park et al.)	Graph (DAG model with latency-aware replica grouping & minimum-cost-flow routing)	Adaptive	Adaptive latency-aware traffic allocation (Min-Cost-Flow based)	Partial (Replica placement across nodes but no availability-zone awareness)	Kubernetes
StornX	Heuristic (Topology-aware AZ placement with adaptive scaling & intelligent traffic routing)	Adaptive	Dynamic traffic shifting based on real-time metrics	Yes (Replica spreading across AZs)	AWS Elastic Kubernetes Cluster (EKS)

Table 1: Comparison of related approaches in microservice placement, traffic management, and resiliency

2.2 Contributions of the work

All of the above studies propose valuable optimization strategies for microservice scheduling and load balancing. They show that network traffic and latency have a major impact on cloud performance and that replication is essential in fault tolerance. Several also introduce traffic allocation algorithms designed to reduce response times. However, none of these approaches

explicitly consider the availability zones of the cloud provider during scaling to ensure that new Pods are placed on the most suitable Nodes for resiliency, nor do they evaluate whether their strategies might lead to unnecessary replica creation under high-stress scenarios. Therefore, we propose StornX, a Kubernetes Pod autoscaler for horizontal scaling that integrates replica placement and traffic distribution into a unified framework. StornX spreads the first replicas across Availability Zones (AZs) to ensure redundancy, then co-locates additional replicas to minimize cross-zone traffic. It leverages Istio Service Mesh locality weights to implement a weighted load-balancing mechanism that dynamically adjusts traffic in real time, directing requests to the nearest and least loaded replicas, thereby reducing cross-Node communication costs. By combining autoscaling with topology- and metric-aware traffic management, StornX improves system efficiency, reduces unnecessary replica creation, lowers costs, and enhances resiliency in real cloud environments.

In this work we introduce StornX, a framework that integrates autoscaling, placement and traffic distribution for Kubernetes-based microservice deployments. The main contributions of this work are as follows:

- **Integration of autoscaling with topology-aware replica placement:** StornX combines horizontal Pod autoscaling with topology-aware replica placement. The first replica Pods are scheduled on Nodes with sufficient resources across availability zones to ensure high availability and fault-tolerance. Once these requirements are met, additional replicas are co-located with Pods that have the strongest communication affinities, thereby minimizing cross-Node traffic.
- **Metric-aware traffic distribution:** StornX incorporates a weighted traffic distribution mechanism implemented with Istio Service Mesh locality weights. Traffic allocation is dynamically adjusted using real-time metrics such as CPU, memory, and Node-to-Node latency. This prevents overloading individual replicas, minimizes cross-Node communication, reduces costs in cloud environments, and avoids unnecessary replica creation during temporary workload spikes.
- **Evaluation in a real cloud environment:** StornX was implemented and tested on Amazon Web Services (AWS) using two microservice-based applications. The experimental results show that StornX achieved better response times than OptTraffic and outperformed the default Kubernetes load-balancing mechanism in terms of communication cost, CPU efficiency, and replica count. It also created fewer replicas than HPA, showing that it scales only when necessary and maintains stable performance while using resources more efficiently.
- **High availability evaluation under Availability Zone failures:** The proposed framework is evaluated under controlled Availability Zone failure scenarios, where an entire zone is removed during steady workload conditions. The results show that StornX recovers faster and with lower performance degradation compared to baseline approaches, highlighting the benefits of adaptive, metric-aware traffic redistribution for maintaining service availability.

3 Background

Service-Oriented Architecture (SOA) [12] is a cloud-native approach where a single application is built from many loosely connected and independently deployable components, often called microservices. These services usually communicate with each other through REST APIs. This modular style makes applications easier to maintain and allows development teams to work on different services at the same time. SOA is well suited for modern software development because it supports agility and helps applications adapt to changing requirements. However, it also introduces challenges, especially for managing and deploying services in distributed environments. To address these issues, developers often use Kubernetes (k8s), an open-source platform that automates the deployment, scaling, and management of containerized applications across different infrastructures.

3.1 Kubernetes

Kubernetes [13] is a portable, extensible, open-source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation. The application of service containers is preferred among the other traditional options of physical servers or VMs running on a specific computer and Operating System (OS) due to the fact that containers have relaxed isolation properties to share the OS among the applications. They are considered lightweight and are capable of allocating specific resources and disk space on their own. They are highly portable across cloud environments and OS distributions. Containers achieve high efficiency on image creation and ease of use, continuous development, good observability on the application and metrics, they provide resource isolation, utilization and they decouple applications from the infrastructure.

Kubernetes provides all the essential tools and a framework to run distributed systems with resilience and handle container behavior and maintenance. It handles the scaling of the application and containers, the failure over situations and provides efficient deployment patterns. It must be mentioned that Kubernetes does not limit the types of applications supported, nor it deploys automatically source code or build the application. Furthermore, it does not provide developers with dictating logging, monitoring or alerting solutions. The main features of Kubernetes include service discovery and load balancing, storage orchestration, automated rollouts and rollbacks, automatic bin packing, self-healing, secret and configuration management, and horizontal scaling

3.1.1 Related Components

In Kubernetes, applications run inside a cluster. A cluster is a group of worker machines, called Nodes, that run containerized applications. As shown in Figure 1, a Kubernetes cluster consists of a control plane (master Node/s), which manages the cluster and worker Nodes, which run the application workloads. Each part contains several components that work together to ensure scalability, fault-tolerance and high availability.

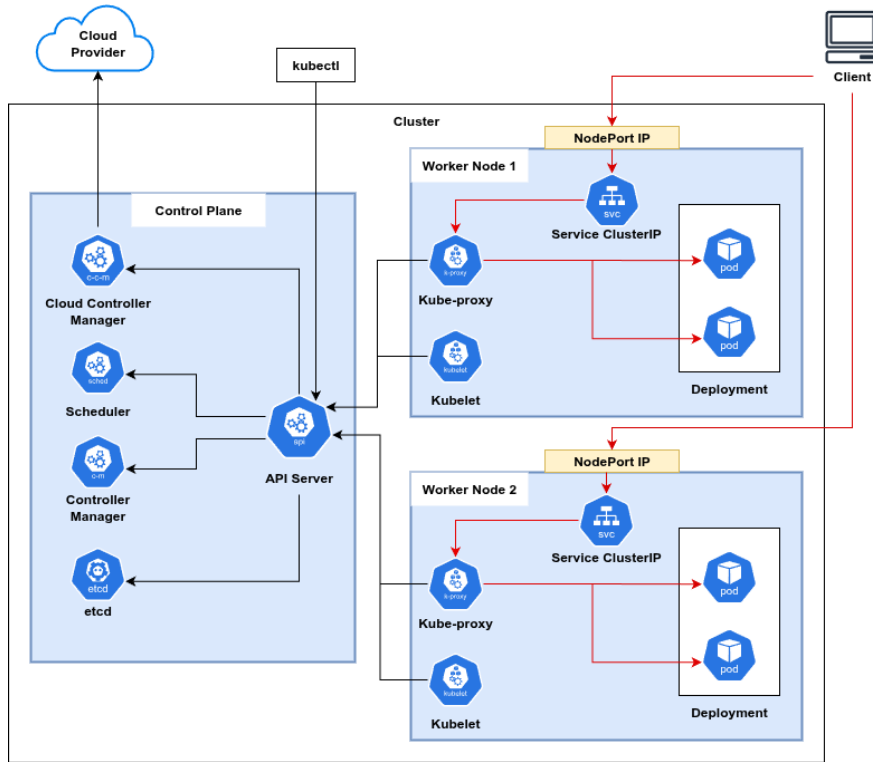


Figure 1: Kubernetes architecture

3.1.2 Control Plane

The Control Plane is the central management layer of a Kubernetes cluster. It monitors and manages the overall state of the cluster, making scheduling and scaling decisions while ensuring that the system operates according to the desired configuration. The Control Plane consists of the following core components:

- **etcd**: ETCD is a distributed key-value store that serves as the primary data storage component of the cluster. It records all cluster-related information and events, except for application data, and ensures data consistency across the cluster.
- **kube-scheduler**: The kube-scheduler is the default scheduler of Kubernetes and is responsible for assigning newly created Pods to suitable Nodes. When a Pod is created, it has specific requirements for resources such as CPU or memory, as well as possible constraints like affinity, anti-affinity rules and taints. The scheduler evaluates the available Nodes and selects the most appropriate one based on these requirements and the current cluster state. If no Node is suitable, the Pod remains unscheduled until resources become available. The scheduler is highly configurable and can be extended or customized to meet the needs of different applications, which makes it a flexible component for improving performance, balancing workloads, and ensuring efficient resource utilization in the cluster.
- **kube-apiserver**: Kube-apiserver is the main entry point for communication with the Kubernetes cluster. It exposes the Kubernetes API and allows administrators and users to interact with the cluster through tools such as kubectl. The API server also coordinates communication with Worker Nodes to manage Pods and other resources.
- **kube-controller-manager**: Kube-controller-manager runs a set of controller processes responsible for maintaining the desired state of the cluster. If differences are detected, it takes corrective actions to reconcile the system, ensuring that the cluster remains in the desired state.

- **cloud-controller-manager:** The cloud-controller-manager is a component that embeds cloud-specific control logic. The cloud controller manager link the cluster into cloud provider's API, and distinguishes between components that only communicate with the cluster and those that communicate with that cloud platform. The cloud-controller-manager only runs if the cluster is deployed on the cloud, if the cluster is on premises the cluster does not have a cloud controller manager.

3.1.3 Worker Nodes

Worker Nodes are responsible for running the application workloads. These workloads are packaged into Pods, the smallest deployable units in Kubernetes. A group of Worker Nodes that share the same configuration, such as CPU, memory, and operating system, is defined as a Node Group and a Kubernetes cluster can consist of multiple Node Groups. Each Worker Node consist of the following core components:

- **kubelet:** The kubelet is an important Kubernetes component that runs on every Node. It makes sure that containers inside Pods are running properly and that each Node can communicate with the Control Plane. The kubelet follows the PodSpec, which is a YAML or JSON file that describes how a Pod should run. It also helps with tasks such as starting Pods, managing resources, and checking the health of applications, which keeps the cluster stable and reliable.
- **kube-proxy:** Kube-proxy is a network proxy that runs on each Node in the cluster, implementing part of the Kubernetes orchestration logic and maintains the network policies of Nodes (i.e network traffic open ports, protocols of communication etc).
- **Container runtime:** The container runtime is the software responsible for running containers inside Pods. Kubernetes uses the Container Runtime Interface (CRI) to communicate with the runtime, allowing the kubelet to start, stop, and manage containers. This design makes Kubernetes flexible as it can work with different container runtimes without changing its core components.

3.1.3.1 Kube-proxy

Kube-proxy is a network component that runs on every Node in a Kubernetes cluster. Its main role is to manage communication between Pods and Services by creating and updating networking rules. It ensures that traffic sent to a Service is forwarded to the correct Pod. Kube-proxy also supports load balancing, which distributes traffic evenly across Pods to improve availability and performance. It operates in different modes depending on the operating system. On Linux, kube-proxy can run in iptables, IPVS, or nftables mode. The iptables mode works entirely in the Linux kernel by installing routing rules and selecting Pods randomly through iptables, and it is the default kube-proxy mode in Kubernetes. The IPVS mode also works in the kernel but relies on the netlink interface and supports multiple load-balancing algorithms such as round-robin, weighted round-robin, least connections, and hashing-based methods, offering higher throughput and lower latency than iptables. The nftables mode, introduced in newer Linux kernels, replaces iptables and provides improved scalability and faster rule updates, especially in large clusters. However, it is still relatively new and may not be compatible with all network plugins. On Windows, kube-proxy operates in kernelspace mode by configuring packet forwarding rules in the Virtual Filtering Platform (VFP), which rewrites packets to reach the correct backend Pods. By maintaining stable network connections and routing rules across these modes, kube-proxy ensures reliable and scalable communication in Kubernetes.

Although kube-proxy offers several load-balancing modes, it does not consider the resource usage of replica Pods or the network latency between the Nodes where they are placed. Consequently, response times increase when traffic is directed to overloaded Pods or routed through high-latency connections. These limitations can significantly impact the performance of delay-sensitive workloads in cloud environments. In Chapter 4, we present *StornX*, which addresses these issues by integrating topology-aware placement with metric-driven adaptive traffic distribution, using Istio instead of kube-proxy.

3.1.4 Kube-scheduler

The kube-scheduler is the default scheduler for Kubernetes and runs as part of the control plane. It is responsible for selecting the most suitable Node on which to run newly created or unscheduled Pods. Kube-scheduler ensures that Pods and Nodes are matched so that Kubelet can run them, is one of the most important components. A scheduler keeps track of newly created Pods that are unassigned to any Nodes. For every Pod that the scheduler detects, the scheduler becomes responsible for finding the best Node for that Pod to run on. The Kubernetes scheduler selects the first unscheduled Pod from his priority queue throughout the scheduling cycle, discovers feasible nodes, and then executes a series of functions to assess each Node. In a procedure known as binding, the scheduler assigns the Pod to the feasible Node with the greatest score and notifies the kube-apiserver of his choice. Several factors influence the scheduler's decision-making process. These factors include resource requirements, hardware and software constraints, policy restrictions, affinity and anti-affinity rules, data locality, and inter-workload interference.

In this work, **taints**¹ were used to control Pod scheduling. A taint is a setting applied to a Node that marks it as unavailable for scheduling under normal conditions. When a Node has a taint, the scheduler avoids placing Pods on that Node. Taints can include a key, a value, and an effect, such as **NoSchedule**, which means that Pods are not allowed to run on that Node.

In this setup, when the scheduling process runs and the appropriate Node is selected, the taints were applied to all Nodes in the cluster except one, which remained untainted. This configuration prevents the scheduler from placing Pods on the tainted Nodes and ensures that the Pods are scheduled only on the Node without a taint. This approach allows precise control over where workloads are deployed and guarantees predictable scheduling behavior. Therefore, the Pod is placed on the Node that does not have a taint.

3.1.5 Kubernetes API Resources

In addition to the core components of the Control Plane and Worker Nodes, Kubernetes provides a set of API resources that allow users to define, configure, and manage applications. These resources are typically described in YAML or JSON manifest files and applied to the cluster through the API server. Some of the most important resources include:

- **Namespaces:** Namespaces provide a mechanism for isolating groups of resources within a single cluster. Namespaces are a common approach to structuring and deploying applications, since each application (or environment such as development, testing, and production) is often deployed in its own namespace. This separation makes it easier to manage resources, apply access controls and set resource quotas while still sharing the same physical cluster.
- **Deployments:** A Deployment defines the desired state for a group of identical Pods, including the number of replicas, the container image, and update strategy. Update strategy is the policy that specifies how Pods are replaced when the Deployment is updated, such as rolling updates or recreating all Pods. Kubernetes ensures that the specified number of Pods is always running and automatically replaces failed Pods. Deployments are one of the most commonly used resources for managing stateless applications.
- **StatefulSets:** Unlike Deployments, StatefulSets are used for managing Pods that require persistent identity and stable storage, such as databases. They ensure that Pods are created in a specific order and retain their identity even if rescheduled. These persistent identifiers are crucial because they allow Kubernetes to reliably match existing storage volumes to replacement Pods after failures or rescheduling. For example, when a Pod in a StatefulSet fails, Kubernetes uses its stable identity to reattach the corresponding persistent volume to the new Pod, thereby ensuring continuity of the application's state.
- **DaemonSets:** A DaemonSet ensures that a copy of a specific Pod runs on all Nodes in the cluster. This is particularly useful for system-level services such as log collectors, monitoring agents, or networking components that must be present across the cluster. When new Nodes are added, DaemonSets automatically schedule the Pod onto them, ensuring full coverage.

¹<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>

- **Services:** Services provide stable network identities for Pods and enable reliable communication between components of an application. Since Pods are ephemeral and their IP addresses may change, Services abstract this complexity by exposing a fixed IP address or DNS name. When a request reaches a Service, it is handled by kube-proxy, which applies the necessary networking rules and forwards the traffic to one of the available Pods. This ensures reliable communication and balances requests across multiple Pods. Common Service types include ClusterIP, NodePort, and LoadBalancer. ClusterIP exposes the Service only within the cluster, making it accessible to internal components. NodePort maps the Service to a static port on each Node’s IP, allowing limited external access. Finally, LoadBalancer Services enable the requests from clients through an Internal or an External IP of the network. Each Service defined as LoadBalancer is associated with a separate External IP from the Node IP. LoadBalancer handles efficiently the load balancing between the application’s containers.
- **PersistentVolume (PV):** A PV is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It’s a cluster-level resource and not bound to a namespace. PVs act like volume plugins, but with a lifecycle independent of any individual Pod that uses them.
- **PersistentVolumeClaim (PVC):** A PVC is a request for storage by a user. Just like Pods consume Node resources, PVCs consume PV resources.

These resources, together with other Kubernetes objects such as ConfigMaps, Secrets, Jobs, and Ingresses, provide a powerful and flexible way to define application behavior and infrastructure in a declarative manner. By combining them, users can deploy scalable, fault-tolerant, and secure applications across distributed environments.

Figure 1 illustrates the communication flow from a client to the Pods inside a Kubernetes cluster. A client sends a request to the cluster through a Service, which may be exposed externally via a NodePort or LoadBalancer. The Service provides a stable network entry point and forwards the request to kube-proxy running on the target Node. Kube-proxy applies the appropriate networking rules and redirects the traffic to one of the available Pods associated with the Service. This process abstracts the underlying complexity so that the client interacts with a single Service endpoint, while Kubernetes automatically handles routing and load balancing across multiple Pods and Worker Nodes.

3.1.5.1 Namespaces

Namespaces in Kubernetes provide a mechanism for organizing and isolating resources within the same cluster. They act as logical partitions that allow teams to separate environments, applications, or projects while still sharing the underlying infrastructure. It is common practice to host each application in its own namespace for easier management, clearer separation of concerns, and the ability to apply resource quotas and access controls at the namespace level. Almost all Kubernetes resources, such as Pods, Services, Deployments, and StatefulSets, are created within a namespace, except for cluster-wide objects like Nodes and PersistentVolumes.

Figure 2 illustrates three namespaces: *eshop*, *bank*, and *booking*, where each is dedicated to a specific application. Within each namespace, multiple workloads can run across different Nodes in the cluster. For example, the *eshop* namespace contains both Deployments and StatefulSets, while the *bank* and *booking* namespaces include only Deployments. Each Deployment or StatefulSet represents a microservice, and together they compose the complete application within that namespace. This separation allows each application to be managed independently while still sharing the same underlying cluster infrastructure.

In this work, StornX is designed to operate at the namespace level and currently supports only Deployments, not StatefulSets. It monitors the resource usage of all Deployment Pods within a namespace and applies scaling, placement, and traffic management strategies at the namespace level, rather than allowing users to select individual Deployments per namespace to be managed. For example, a user might want to apply scaling and traffic distribution rules only to the *cart* Deployment of an e-shop application, but StornX would apply its strategies across the entire namespace. In future work, we plan to extend StornX’s functionality to enable handling individual microservices.

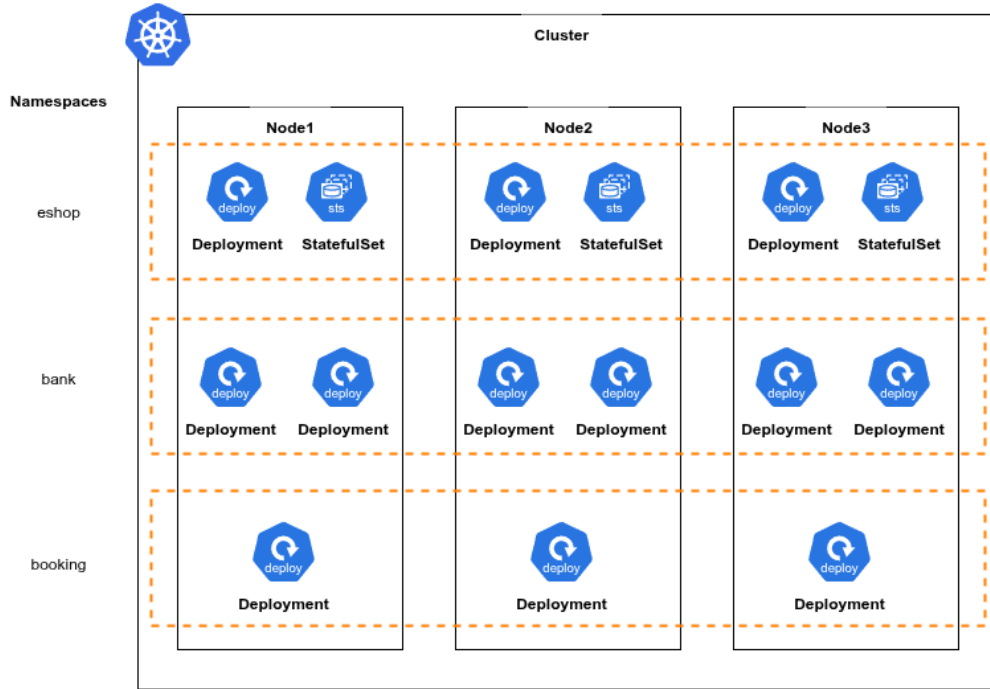


Figure 2: Namespaces

3.1.5.2 Deployments and StatefulSets

Microservices are typically either stateless or stateful. Stateless services do not retain client-specific data between requests, which makes them easy to replicate and scale horizontally. Stateful services, by contrast, preserve data or session information across requests and therefore require persistent storage and coordination among replicas, such as in databases or message queues. Relying on session data makes scaling and load balancing harder, because the state has to be shared across Pods.

In Kubernetes, stateless applications are managed with Deployments, while stateful applications are managed with StatefulSets. A Deployment defines the desired state of a set of Pods and ensures that this state is maintained by automatically creating or replacing Pods through an underlying ReplicaSet. Deployments provide self-healing by recreating failed Pods, support rolling updates and rollbacks for controlled application upgrades, and allow flexible scaling to handle variable workloads. On the other hand, StatefulSets are designed for workloads that need persistent identities and stable storage, ensuring that Pods retain their names and storage volumes even when rescheduled. Replicating stateful workloads is more complex, as it requires clustering and consistent data synchronization across Pods.

In this work, we focus on Deployments because StornX operates at the namespace level and the ability to select individual microservices for management has not yet been designed. Supporting both Deployments and StatefulSets in the same namespace could also introduce complications, since not all StatefulSets are capable of clustering or data synchronization. For this reason, StornX currently supports only stateless Deployments and ignore StatefulSets running in the provided namespaces. Extending its functionality to handle stateful workloads is considered future work. The following YAML (Listing 1) defines a simple Deployment of an nginx web server.

Listing 1: Example Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
```

```

    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  containers:
    - name: nginx
      image: nginx:1.27.0
      ports:
        - containerPort: 80
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"

```

Below there is an explanation of some key fields that need to be understood from the Deployment configuration of Listing 1.

- **replicas:** Specifies the number of replica pods. A pod can have one or more instances to ensure fault-tolerance.
- **image:** Specifies the container image to use for the application.
- **resources:** Defines the minimum and maximum resources that a container can use in terms of CPU and Memory. The resources are used to define how much CPU and Memory the pods within the Deployment can use. These two resources are the requests, which specify the minimum amount of resources a pod needs to start and run and limits, which define the maximum amount of resources a pod can use. The Kubernetes Scheduler uses the request value to find a Node that can satisfy the resources required for the pod to start. If no Node has sufficient resources, the pod remains unscheduled. On the other hand, if a pod exceeds its Memory resource limits, Kubernetes will terminate the pod, whereas if the CPU resource limits are exceeded the pod will be throttled.

3.1.6 Horizontal Pod Autoscaler

A core concern in microservice architectures is scalability. Production systems commonly rely on horizontal scaling to add replicas and sustain availability and low latency under high load. Previous work has proposed scaling strategies to address service unavailability [14] [15] [16] caused by high volume of requests and to reduce end-to-end latency [17] [18]. In Kubernetes, horizontal scaling is typically achieved through the Horizontal Pod Autoscaler (HPA), which adjusts workloads dynamically based on resource or custom metrics.

The Horizontal Pod Autoscaler (HPA) [19] automatically adjusts the number of replica Pods in a workload to match the current demand. It works as a control loop in the Kubernetes control plane, checking metrics at regular intervals and updating the number of replicas when needed. HPA can rely on metrics such as CPU, memory, or custom application metrics to decide when to scale up or down. HPA also allows configuration of how quickly scaling should happen, both when adding and when removing replicas. For example, scaling up could be set to happen every 15 seconds, while scaling down might wait for 60 seconds to avoid removing Pods too early, since the application may still experience bursts of traffic during that time.

In Kubernetes, a Pod's request value is reserved on the Node whether it is used or not. The limit defines the maximum resources a Pod can consume, and if it is exceeded, the Pod is throttled for CPU or terminated for memory. If requests are not set, Kubernetes might schedule a pod on a Node that does not actually have enough CPU or memory to run properly. This can lead to poor performance and even cause the Node to run out of resources. Setting requests and limits is a good practice in Kubernetes. Requests make sure the scheduler places a Pod only on a Node with enough resources to run it, while limits act as a safeguard by defining the

maximum resources a Pod can use. Together, they make workloads more predictable, reliable, and help keep the cluster stable.

HPA’s scaling decisions depend on the resource requests defined in the Deployment template (Figure 1) and not on limits. This can lead to inefficiencies and wasted Node capacity when requests are not estimated accurately, particularly for workloads that receive sudden traffic spikes. In practice, requests are difficult to predict accurately. Over-estimating requests leads to wasted Node capacity because the cluster reserves resources that may never be fully used, while under-estimating requests can lead to high number of replicas. For example, suppose a Deployment where each Pod actually needs about 250 millicores of CPU to run properly, but the request is set to 500 millicores (over-estimated) to prepare for high traffic. The cluster reserves the 500 millicores even though the Pod rarely consumes more than 300. Additionally, if the initial 500 millicores are already consumed and HPA adds a new replica, the cluster reserves another 500 millicores even though the new Pod may only need 250 to 350 to handle the traffic. If the cluster does not have enough free capacity to satisfy the 500 millicore request, the Pod will remain unschedulable even if sufficient resources exist for its actual 250 millicore demand. This shows that scaling decisions based on requests can waste resources and limit scalability.

StornX addresses this limitation by depend on resource limits instead of requests when making scaling decisions. This approach allows the cluster to run more Pods without being blocked by requests that reserve more resources than the Pods actually use. If a Deployment does not define requests or limits, StornX estimates them by setting the requests to the actual usage observed over the last two minutes and the limits to twice that value. In addition, when the specified limits exceed the available capacity of a Node, StornX automatically considers the Node’s maximum free resources as the limit. This scaling design keeps scaling decisions both safe and efficient, reducing wasted capacity while ensuring that workloads can still obtain the resources they need.

3.2 Service Mesh

A service mesh is an infrastructure layer designed to manage communication between microservices. When moving from a monolithic application to a microservice-based architecture, several new challenges appear that do not exist in monolithic systems. For example, consider an online shopping application that is built with multiple microservices: a web server that handles user interface requests, a payment service for processing transactions, a shopping cart service and an inventory service. Each of these microservices contains its own business logic, but to deliver the full functionality of the application they must communicate with each other. This creates the need for reliable communication mechanisms.

It is crucial to know how services can discover each other, determine the correct endpoints for sending requests and automatically update them when new service instances are deployed. As a result, additional configuration and management of communication between microservices is required. Beyond service discovery, microservice communication also introduces important concerns around security, resilience, and monitoring. In a microservices architecture, services communicate with each other over the network, making them vulnerable to failures and potential attacks. Developers often must implement retry policies to handle temporary failures, add encryption to protect sensitive data, and include metrics or tracing to understand traffic flow through the services. Without a unified approach, these concerns are implemented differently in each service, which increases complexity and slows down development.

A service mesh (Figure 3) solves these challenges by introducing an infrastructure layer that handles communication between services [20]. Instead of embedding networking, security, and monitoring logic into each service code, the service mesh extracts all the non-business outside of the microservices adding a sidecar application that handles all such functionality acting as a proxy. These proxies manage traffic transparently, while a control plane coordinates their behavior. As a result, developers can focus on business logic of the application.

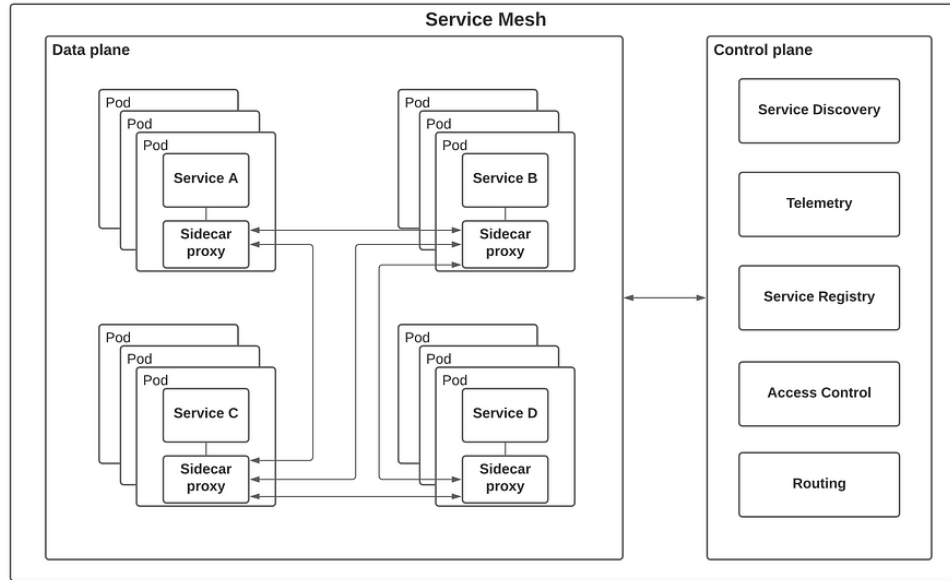


Figure 3: Service Mesh Architecture

Kubernetes is the most common platform for running microservices today, but while it manages the application lifecycle effectively, it does not fully address the complexity of service-to-service communication. Kubernetes Services provide only basic service discovery and load balancing. Traffic between Pods inside the cluster is not encrypted by default, and although Kubernetes Network Policies² can restrict which Pods are allowed to communicate, they do not provide authentication or encryption. As a result, any Pod can attempt to communicate with another Pod in an unsecure way. In addition, Kubernetes itself does not provide mechanisms for advanced traffic routing, retries, or distributed tracing, all of which are essential to run microservices reliably at scale.

Service mesh addresses these limitations by adding a communication layer on top of Kubernetes. It secures traffic between Pods with mutual TLS (mTLS) encryption, enforces authentication and authorization policies, and provides fine-grained control over how requests are routed and balanced across services. It also collects detailed telemetry data such as latency, error rates, and request traces, which are critical for monitoring and troubleshooting microservice-based applications. Furthermore, a service mesh introduces resilience patterns such as retries, timeouts, and circuit breakers to ensure that failures in one service do not affect and disrupt the entire system.

3.2.1 Istio

One of the most widely adopted service meshes is Istio, originally created by Google, IBM, and Lyft in 2016. Today, Istio is a graduated open-source project under the Cloud Native Computing Foundation (CNCF), alongside other cloud-native technologies such as Kubernetes and Prometheus. It has become a trusted solution for managing communication in modern microservices environments, enabling enterprises to run distributed workloads reliably across clusters, clouds, and platforms.

Istio provides advanced security and governance controls, including mutual TLS (mTLS), fine-grained policy management, and access control. It also supports powerful networking capabilities such as canary deployments, A/B testing, intelligent load balancing, failure recovery, and advanced traffic management. In addition, Istio enhances observability by capturing metrics, logs, and tracing information.

The Istio consists of two components: the data plane and the control plane (Figure 4). The data plane is composed of Envoy sidecar proxies that are automatically injected into each Pod

²<https://kubernetes.io/docs/concepts/services-networking/network-policies/>

to intercept and manage all incoming and outgoing traffic. These proxies enforce security, apply routing and load balancing rules, and collect telemetry data without requiring changes to the application code. The control plane, called Istiod, provides service discovery, configuration, and certificate management, while converting high-level traffic and security policies into Envoy-specific configurations that are distributed to sidecar proxies at runtime. It acts as a certificate authority to enable secure mTLS communication, supports authentication and authorization based on service identity, and works across multiple environments such as Kubernetes and virtual machines.

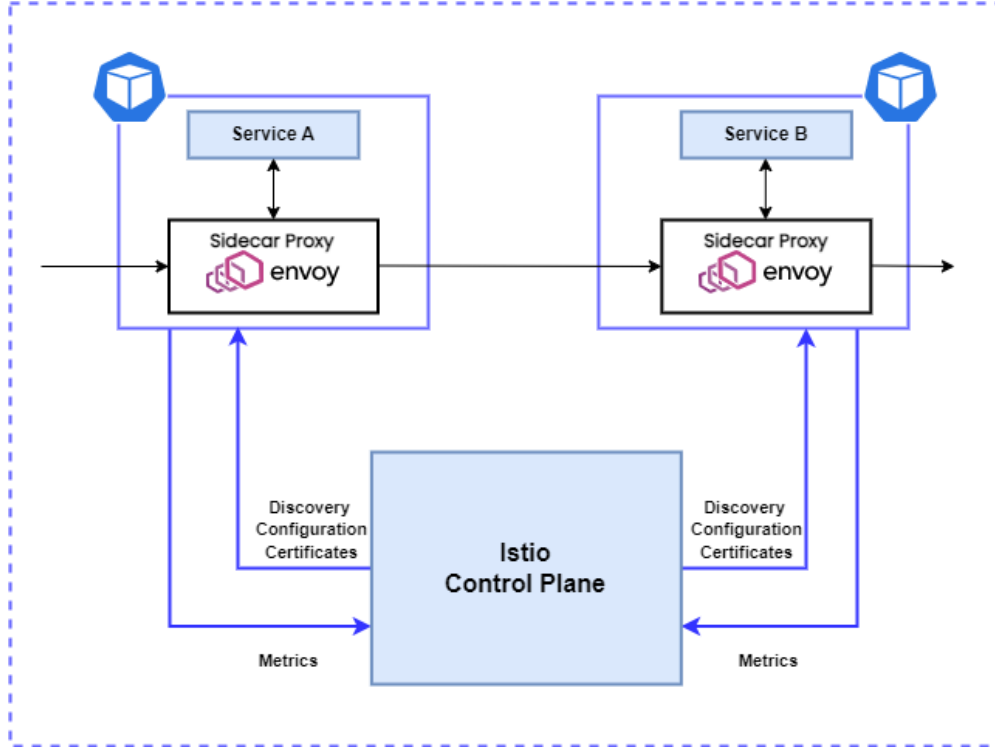


Figure 4: Istio Architecture

StornX uses Istio to route traffic among replica Pods with Istio’s locality-weighted distribution policies ³, ensuring that traffic is routed to co-located Pods with available resources, reducing cross-Node communication and preventing Pods from becoming overloaded. It also relies on requests-per-second (RPS) measurements from Istio telemetry to identify communication affinities, so that replicas are scheduled closer to the Pods they communicate with most. Finally, Istio is used to gather the communication graph between the Pods.

3.3 Prometheus

Prometheus is an open-source system monitoring and alerting toolkit that constantly monitors its targets, identifies problems before they even occur, and provides alerts when a crash does happen [22]. Prometheus collects and stores its metrics as time-series data, i.e., metrics information is stored with the timestamp at which it was recorded, alongside optional key-value pairs called labels. Prometheus targets can be an entire server, an application, a service, etc. For Prometheus to monitor a target, the target must be providing the correct format for its metrics and exposing them through the Prometheus endpoints. Many services do not have native Prometheus support, so an extra component, called Exporter, is needed to read the metrics from the service, transform them into a compatible format and expose the endpoint so Prometheus can pull these metrics.

³<https://istio.io/latest/docs/tasks/traffic-management/locality-load-balancing/distribute/>

A Kubernetes Node, which is a Linux Virtual Machine, does not provide native support for Prometheus, so in order to monitor the VM units, we need to add a Node Exporter. A Node Exporter is an official Prometheus exporter for capturing all the Linux system related metrics, which collects all the hardware and Operating System level metrics that are exposed by the kernel. Prometheus pulls metrics from its targets and stores them in a human-readable, textbased format. The metrics can then be requested by another tool or a user by sending a PromQL query to the Prometheus API Server. The metrics can be visualized either by the Prometheus server or by more advanced visualization tools like Grafana.

Istio exposes service-level telemetry, which Prometheus scrapes for monitoring and visualization. StornX uses the Prometheus API to retrieve key metrics such as CPU and memory utilization, requests per second, Pod communication links, network latency between Nodes, in order to make decisions for scaling, placement, and traffic distribution.

3.4 Regions and Availability Zones

Cloud infrastructure is organized into Regions and Availability Zones. A Region is a physical location around the world. An Availability Zone (AZ) is one or more physically separate data centers within a Region, each with independent power, networking, and connectivity. Figure 5 shows how cloud infrastructure is structured into Regions and Availability Zones. Each Region (orange), represents a geographic area such as Frankfurt, Stockholm, or Milan. Inside each Region there are multiple Availability Zones (blue), which are separate, independent data centers. For example, the Frankfurt Region contains three Availability Zones named eu-central-1a, eu-central-1b, and eu-central-1c. These Zones are connected by high speed, low latency links but remain independent in terms of power, networking, and connectivity.

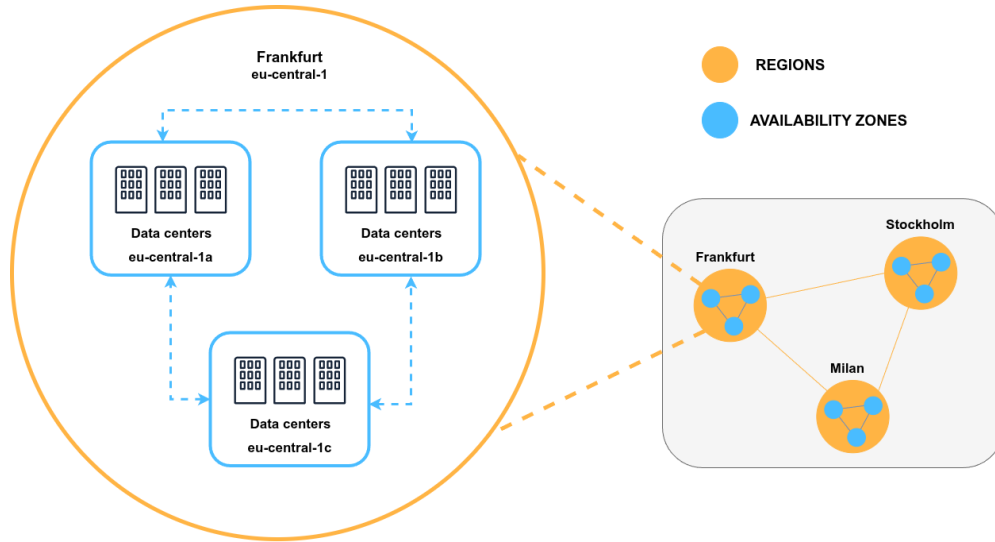


Figure 5: Regions & Availability Zones on AWS

Using multiple Availability Zones in cloud environments is the best practice for building resilient systems. By spreading workloads across AZs, applications can continue running even if one data center fails. This design improves availability, fault tolerance, and scalability compared to relying on a single zone. However, AZs have physical distance between them, which makes communication across them slower than intra-zone communication. Intra-zone communication generally has lower latency and in the case of AWS does not incur additional data transfer charges, although this may vary by provider. Cross-zone communication, on the other hand, introduces additional delay and often extra costs [23]. For workloads that exchange large volumes of data or communicate frequently, cross-zone communication reduces performance and lead to higher operational costs.

High availability and fault tolerance in Kubernetes clusters are achieved by distributing Nodes across multiple Availability Zones. If all Nodes and the control plane are located in

a single zone and that zone goes down due to a power outage or network issue, the entire cluster will become unavailable. By spreading Nodes and control plane components across zones, Kubernetes can reschedule workloads onto healthy Nodes in other zones, ensuring that the system continues to operate. Within a workload, replicas provide fault tolerance by allowing services to continue running even if individual Pods fail. When both the control plane and the replicas are distributed across multiple zones, the system achieves high availability and remains resilient to zone-level failures. As a result, scheduling and traffic management are very important factors. If all replica Pods are placed in the same zone, a failure in that zone can make the system unavailable, while spreading replicas across multiple zones improves resilience but also increases costs and response times because of cross-zone communication.

StornX uses AZs to improve the high availability and fault-tolerance of microservices. It schedules the first replica Pods on Nodes with sufficient resources across multiple AZs to achieve these goals. Once these goals are achieved, additional replicas are co-located with the Pods they communicate with most frequently, minimizing cross-zone and cross-Node communications and consequently response times and operational costs.

It is important to note that although StornX is designed to improve high-availability and fault-tolerance, this rule sometimes may be skipped. If a workload requires a new replica and its distribution across zones has not yet achieved, and the Nodes in the remaining AZs do not have sufficient resources, StornX will bypass the rule and proceed with scheduling. This is because StornX makes scheduling decisions immediately based on the current cluster state, while Node creation through the cluster autoscaler takes additional time. As a result, if no suitable Node is available, the rule is bypassed and the Pod is scheduled on an existing Node (if one exists). Otherwise, the Pod remains unscheduled until the cluster autoscaler provisions a new Node. By bypassing the rule, the new Pod is still created and does not remain unscheduled.

4 System Overview

In this section, we describe the overall system design of StornX and the design decisions that influenced its implementation. Our goal is to design a system with the following objectives:

- **Efficient placement across Nodes and Availability Zones:** One of the primary goals of our design is to achieve high availability and fault-tolerance for replica Pods. In many applications, the workload is unpredictable. Therefore, Kubernetes Deployments often start with a small number of replica Pods, sometimes even a single one and as the load increases, additional replicas are created to handle the traffic. To improve system resilience, these new replicas must be placed efficiently across the cluster. In cloud environments, placing all replicas on the same Node or within the same Availability Zone is risky. If either the Node or the entire AZ becomes unavailable, all replicas would fail simultaneously, causing significant downtime until Kubernetes reschedules them on healthy Nodes. Figures 6, 7, and 8 illustrate three scenarios for a Deployment with two replicas. In Figure 6 both replicas are placed on the same Node within a single AZ. In this case if the Node fails, the application becomes unavailable.

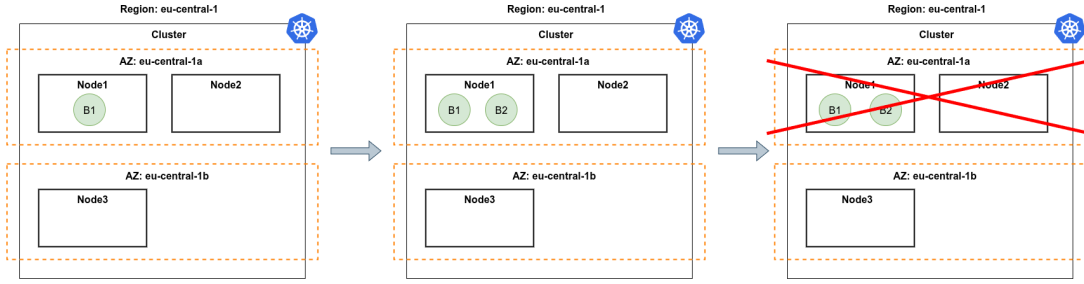


Figure 6: Single-zone & single-node placement

In Figure 7 replicas are placed on different Nodes within the same AZ. This placement provides resilience against Node failures, but the system remains at risk to an AZ outage.

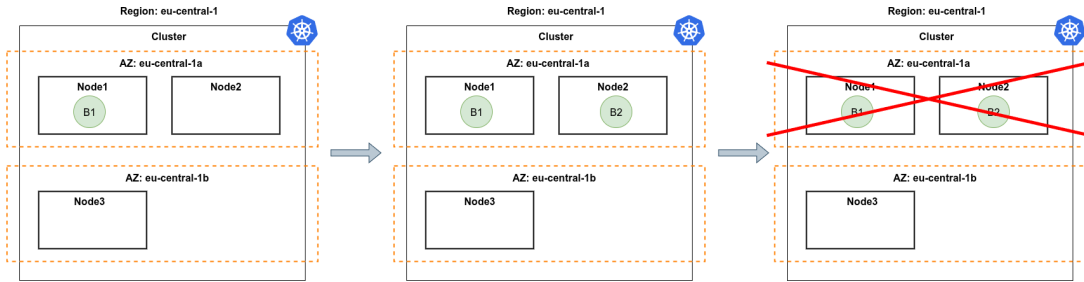


Figure 7: Single-zone & multi-node placement

In Figure 8 replicas are spreading across different Nodes in separate AZs. This configuration achieves the highest availability, as the system continues to operate even if an entire AZ fails.

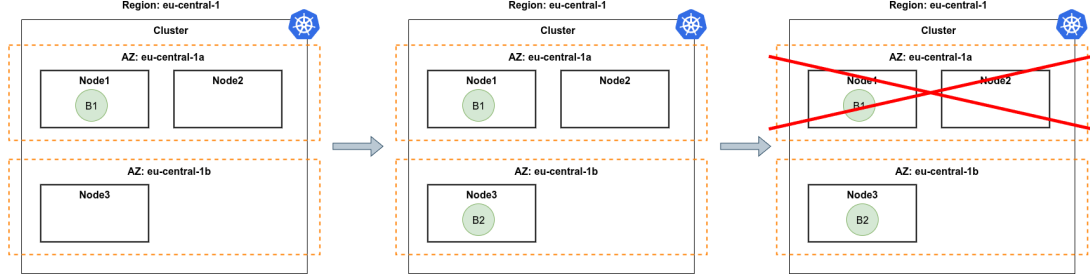


Figure 8: Multi-zone & multi-node placement

This comparison highlights the importance of distributing replicas not only across multiple Nodes but also across Availability Zones. For this reason, our job adopts the strategy shown in Figure 8 when scheduling the first replicas, ensuring resilience against both Node and AZ failures before optimizing placement for communication efficiency.

It is important to note that depending on the expected load some applications may initially deploy multiple replicas to handle early workload spikes. In such cases, the initial scheduling is performed by the default Kubernetes scheduler rather than StornX. Since these replicas typically do not receive significant traffic during startup, the StornX later reduces the number of replicas when low load is detected, often down to a single Pod. This behavior explains why our design assumes that Deployments usually begin with a single active replica. Additional replicas are created only when the load increases.

Enforcing a minimum replica count greater than one would affect all Deployments within the same namespace. This could lead to inefficient resource utilization, particularly when some stateless services are non-critical and can tolerate temporary downtime. Maintaining multiple replicas for every service would quickly consume cluster resources, especially in environments with limited capacity or strict cost constraints. Therefore, the current design avoids enforcing a higher minimum replica count until per-Deployment management is supported (read more in paragraph 3.1.5.1). Per-Deployment management means that StornX could manage each Deployment independently rather than treating all Deployments in a namespace the same. This would allow handling each microservice (where each microservice corresponds to a Deployment) individually instead of applying policies globally at the namespace level.

- **Efficient communication-aware placement:** In a microservice-based architecture the communication between the Pods can be represented by a directed acyclic graph (DAG). A Pod that sends requests is referred to as an *upstream* Pod, while those receiving requests are defined as *downstream* Pods. One of our key objectives is to place replicas closer to the Pods with which they communicate most frequently. In cloud environments with multiple Availability Zones, placing upstream and downstream Pods across different Zones or even across different Nodes, introduces additional network latency, which affects response times between services, and increase the operational costs. However, improving communication efficiency by co-locating Pods can reduce fault tolerance, since failures in a single Node or Zone may impact multiple services. Therefore, StornX balances both factors by first ensuring availability across Zones (previous objective) before optimizing placement for communication efficiency.

To evaluate the impact of cross-zone communication, we used kube-netlag (read more in paragraph 4.1) to measure latency between Availability Zones and among Nodes within the AWS Frankfurt region (*eu-central-1*). Tables 2 and 3 present the average and maximum network latencies. The tables are not perfectly symmetric because each direction was measured independently. Small differences between directions occur because data may travel through slightly different network paths. As expected, communication between the Nodes within the same AZ shows the lowest delays, while cross-AZ communication is slightly slower. The maximum latency values show occasional spikes up to 34 ms in our measurements. These spikes are much higher than the average values and can cause microservices to respond much more

slowly. In our design, the average latency is used as the primary metric for placement decisions, while maximum latency is considered only as an indication of potential worst-case behavior.

Source \ Destination (ms)	eu-central-1a	eu-central-1b	eu-central-1c
eu-central-1a	0.104	0.565	0.740
eu-central-1b	0.494	0.125	0.441
eu-central-1c	0.748	0.462	0.109

Table 2: Average one-way latency between Availability Zones

Source \ Destination (ms)	eu-central-1a	eu-central-1b	eu-central-1c
eu-central-1a	11	26	37
eu-central-1b	25	4	28
eu-central-1c	34	28	15

Table 3: Maximum one-way latency between Availability Zones

To address this, our design should identify the most suitable Node for each new replica based on upstream and downstream communication affinities, ensuring that replicas are co-located with the Pods that exchange the highest volume of requests.

- **Metric-aware traffic distribution:** The final goal of our design is to achieve dynamic traffic management on replica Pods. The default Kubernetes load-balancing mechanism (iptables) relies on randomized distribution and does not take into account real-time factors such as Pods’ resource utilization or latency between Nodes. As a result, traffic may be routed to Nodes with higher network latency or to overloaded Pods, while other Pods remain underutilized and capable of handling additional requests. To address this, we consider three parameters:

1. **Node-to-Node latency:** Node-to-Node latency defines the communication delay between the Nodes hosting upstream and downstream Pods, which may also reside in different Availability Zones. Lower latency corresponds to faster message delivery and shorter response times. By measuring and monitoring these latencies, the system prioritizes sending traffic to downstream Pods located on Nodes with minimal delay and ideally within the same Node or at least within the same Availability Zone. For reducing response times, this follows the opposite logic of fault tolerance, in which traffic prioritization favors network closeness of replicas over their deployment across different Availability Zones. This approach improves communication efficiency and enhances overall performance.
2. **Pod resources:** Each downstream Pod has limited computational capacity defined by its CPU and memory allocations. When a Pod operates close to its resource limits, additional incoming requests can cause queueing delays and performance degradation. To prevent overloading, the system continuously monitors the real-time resource utilization of downstream Pods and reduces the amount of traffic directed to heavily loaded ones, while redirecting more requests to less utilized Pods.
3. **Number of downstream replica Pods:** The number of available downstream replicas determines how much total traffic can be effectively handled by a service. A larger number of replicas indicates higher processing capacity and allows traffic to be distributed across multiple instances, reducing the load on individual Pods. Therefore, the system allocates a greater portion of upstream traffic to Nodes that have more downstream replicas available, since they can process more requests without performance loss.

By combining these three parameters, our system dynamically adjusts traffic allocation among downstream Pods. This ensures that requests are routed preferentially to replicas that are nearby, lightly loaded and have a sufficient number of downstream replicas, maintaining high performance and balanced resource utilization across the cluster.

To realize the above design objectives, we developed StornX, a Kubernetes framework that integrates autoscaling, replica placement and traffic distribution into a unified process. StornX remaining transparent to containerized microservices while providing optimized scheduling and traffic management. The system is composed of two core components: the **OptiScaler** (Node Scheduler), which performs horizontal scaling and intelligent placement of replica Pods by deciding when to create or remove replicas and scheduling them onto the most suitable Nodes to ensure both resiliency and communication efficiency, and the **OptiBalancer** (Traffic Scheduler), which manages dynamic traffic distribution across replicas by leveraging Istio locality weights and real-time metrics such as CPU, memory, and Node-to-Node latency to adaptively route traffic. Together, these two components allow StornX to prevent Pod overload, minimize cross-Node communication, reduce response times and ensure efficient resource utilization.

4.1 Kube-netlag

To support these objectives, StornX requires accurate and continuous visibility into the communication conditions of the cluster, particularly the network latency between Nodes and Availability Zones. Since latency directly affects placement and traffic routing decisions, obtaining reliable and real-time measurements is essential. For this purpose, we developed **Kube-NetLag**, a lightweight tool that monitors Node-to-Node latency and exports these measurements to StornX through Prometheus metrics. The following subsection describes Kube-NetLag in detail and explains how it supplies the latency information that enables StornX to make informed placement and traffic distribution decisions.

Kube-NetLag^{4 5} is a lightweight tool to measure network latency on a Kubernetes cluster. It runs as a DaemonSet, which ensures that a Pod is automatically deployed on every Kubernetes Node. Each Kube-NetLag Pod uses the Kubernetes API to discover all other Nodes in the cluster and creates a list of their IP addresses. Using this list, the tool performs active network tests with Netperf⁶ in order to measure the latency between Nodes. Since it is deployed as a DaemonSet, new Nodes are automatically included in the measurements as soon as they join the cluster. This makes the setup fully automated and requires no manual configuration.

The results of these measurements are exposed through a Prometheus metrics endpoint, making it easy to monitor Node-to-Node latency in real time. These metrics can be used to detect slow network links, compare communication performance between Nodes, and monitor changes in latency over time.

Kube-NetLag is an open-source project. It has been developed in parallel with StornX in order to provide the latency information that StornX needs to place replicas more efficiently and direct traffic toward Nodes with the lowest latency. Together, the two systems improve replica placement, optimize traffic distribution toward Nodes with the lowest latency, reduce application response times, and minimize cross-zone communication costs in Kubernetes clusters.

4.2 System Prerequisites

StornX depends on several external components (Figure 9). These external dependencies provide runtime metrics, traffic management, and latency measurements that enable StornX to make informed autoscaling, placement, and traffic distribution decisions.

Specifically, StornX depends on the Istio Service Mesh, which enriches Prometheus with metrics from running microservices and provides weighted traffic management capabilities that are essential for the OptiBalancer to distribute requests based on latency and resource usage. In addition, StornX relies on Prometheus to collect runtime metrics such as resource utilization (CPU and memory) for Pods and Nodes, and it collects fine-grained measurements including requests per second (RPS) and network latency, which are used for scaling and placement decisions. To capture network delays between nodes, the system also integrates with Kube-netlag (read more in paragraph 4.1), which measures Node-to-Node latency and exports these metrics to Prometheus.

⁴<https://github.com/AposLaz/kube-netlag>

⁵<https://artifacthub.io/packages/helm/kube-netlag/kube-netlag>

⁶<https://github.com/HewlettPackard/netperf>

Finally, StornX communicates with the Kubernetes API server to collect information about the cluster state, including Pods and Nodes. It also interacts directly with Prometheus to retrieve measurements such as requests-per-second, network latency, the communication links between Pods (upstream and downstream relationships) and resource utilization metrics.

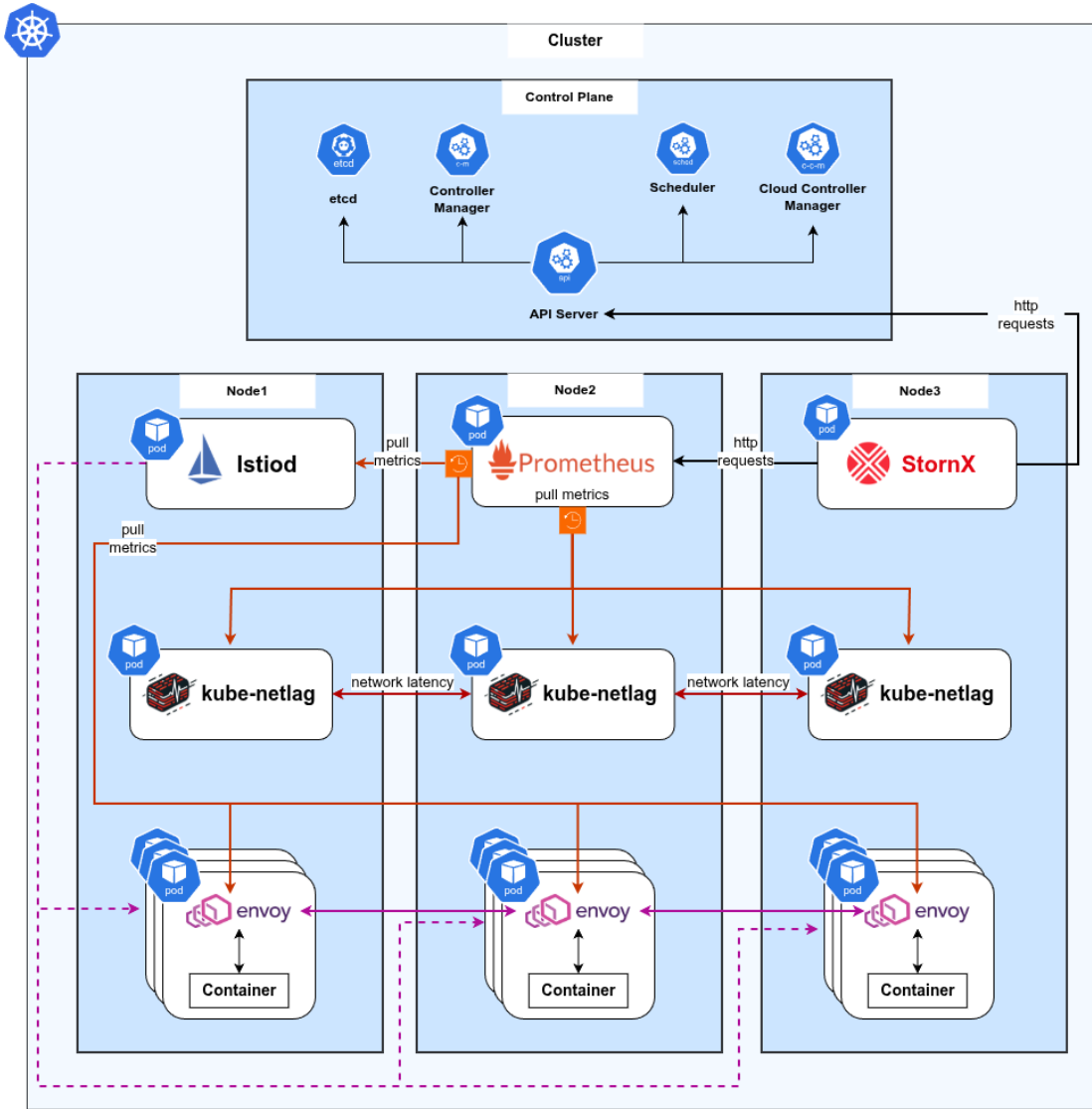


Figure 9: Architecture Overview

4.3 StornX Scope

StornX operates exclusively on Deployments in Kubernetes (read more in paragraph 3.1.5.2). Other resources like StatefulSets, Jobs, CronJobs or DaemonSets are not monitored by StornX. For reliable operation, StornX assumes that each Deployment has defined resource requests and limits. These values are essential for scaling and placement decisions by providing an accurate estimation of Pods' resource needs. If a Deployment does not specify them, StornX automatically estimates resource requests as the average resource usage of the Pod over the last 2 minutes, and sets the resource limits to either twice that request values or the Node's maximum available resources. This approach allows Pods without defined resources to run safely by assigning them realistic requests and limits based on recent usage and available Node capacity.

StornX operates at the namespace level rather than applying scaling and traffic management decisions separately to each Deployment (read more in paragraph 3.1.5.1). It monitors all Pods belonging to Deployments within user-defined namespaces and jointly optimizes autoscaling,

placement, and traffic distribution across them. This design allows to optimize scheduling and traffic distribution for entire applications grouped in a namespace, while other namespaces remain unaffected. In the current implementation, Kubernetes *ResourceQuotas* ⁷ are not used and StornX does not handle cases where such quotas are defined. The system relies on available cluster capacity when performing scaling operations. If a namespace with defined *ResourceQuotas* temporarily reaches the limits of available resources, StornX postpones the creation of new replicas and focuses on redistributing workloads among existing Pods until additional resources become available

In the following subsections, we present the two core components of StornX, OptiScaler and OptiBalancer and describe in detail how they implement the design objectives through autoscaling, intelligent replica placement, and dynamic traffic distribution.

4.4 System Architecture

In this subsection, we describe the algorithms of StornX and explain how it works. StornX is a configurable system, meaning that users can define its operational parameters such as monitored namespaces, resource metrics, scaling thresholds, and execution interval. This flexibility allows them to adapt the system to the specific needs of their workloads.

Before installing StornX on Kubernetes, users should define several parameters, otherwise the default values will be applied. The first parameter is the set of namespaces where StornX will monitor resources and perform autoscaling and traffic distribution actions. By default, monitoring is applied to the “default” namespace. Users can also specify which resources should be monitored. The currently supported options are CPU, memory, or a combination of both, meaning that StornX will track CPU and memory usage together. The default resource for monitoring is the CPU.

In addition to monitored resources, users can configure the upper and lower thresholds, which StornX uses for autoscaling decisions. The upper threshold indicates the point at which a Pod will scale up, while the lower threshold defines when a Pod will scale down. By default, the upper and lower thresholds are set to 80% and 20%, respectively. StornX runs in an infinite loop with a configurable interval. Users can define how often it executes and the default interval is 60 seconds. When all the above parameters are configured users can install StornX using the existing Helm Chart ⁸ or directly with the provided manifests ⁹. Once installed, StornX runs continuously at the configured interval. The main algorithm of StornX is described in Algorithm 1.

⁷<https://kubernetes.io/docs/concepts/policy/resource-quotas/>

⁸<https://artifacthub.io/packages/helm/stornx/stornx>

⁹<https://github.com/AposLaz/StornX>

Algorithm 1 StornX

```
1: procedure MAIN
2:   createFile(optFile)           ▷ This file stores the Deployments that scale up or down
3:   upperThreshold ← 80%
4:   lowerThreshold ← 20%
5:
6:   for each deployment in optFile do
7:     If all replica Pods of the Deployment are healthy apply OptiBalancer
8:     OptiBalancer(deployment)           ▷ Redistribute Traffic
9:     remove(optFile, deployment)   ▷ Remove Deployment from pending list after traffic
has been redistributed
10:  end for
11:
12:  for each namespace in NAMESPACES do
13:    deployments ← getDeploymentsByNamespace(namespace)
14:    replicaPods ← getReplicaPodsofDeployments(deployments)
15:
16:    highLoadPods ← getPodsAtThreshold(replicaPods, upperThreshold, "upper")
17:    lowLoadPods ← getPodsAtThreshold(replicaPods, lowerThreshold, "lower")
18:
19:    if lowLoadPods.length > 0 then           ▷ length = number of Pods in the list
20:      for each pod in lowLoadPods do
21:        clusterAvg ← getAvgMetricForDeployment(pod.deployment)
22:        if clusterAvg ≤ lowerThreshold and pod.deployment.rs.length > 1 then
23:          OptiScaler.scaleDown(pod, pod.deployment)           ▷ Remove replica Pod
24:          writeFile(optFile, pod.deployment)
25:        end if
26:      end for
27:    end if
28:
29:    if highLoadPods.length > 0 then
30:      for each pod in highLoadPods do
31:        clusterAvg ← getAvgMetricForDeployment(pod.deployment)
32:        nodeAvg ← getAvgMetricByNodeForDeployment(pod.deployment)
33:        if clusterAvg ≥ upperThreshold then
34:          OptiScaler.scaleUp(pod, pod.deployment)           ▷ Create a new replica
(OptiScaler decides the target Node)
35:          writeFile(optFile, pod.deployment)
36:          continue           ▷ Skip further checks for this Pod and move to the next one
37:        end if
38:        if nodeAvg ≥ upperThreshold then
39:          OptiBalancer(pod.deployment)           ▷ Redistribute Traffic
40:        end if
41:      end for
42:    end if
43:  end for
44:  sleep(60)           ▷ Pause the execution for a specified interval before restarting
45: end procedure
```

During each cycle, StornX monitors the Deployments within the specified namespaces and their replica Pods. For each namespace, it identifies Pods that exceed the upper threshold (*highLoadPods*) and Pods that fall below the lower threshold (*lowLoadPods*).

For *lowLoadPods*, StornX first checks whether **all replicas** of the Deployment are below the lower threshold. If so and the corresponding Deployment has more than one replica, the *OptiScaler* executes the **scaleDown** function to remove one underutilized replica at a time (lines 19-27). StornX never scales down a Deployment with only a single replica, as this would make the mi-

crosservice unavailable. Therefore, scale-down operations are performed only when the number of replicas is greater than one. Removing replicas gradually in this way avoids shrinking the Deployment too aggressively, leaving headroom so that the remaining replicas do not become suddenly over-utilized if the workload increases. In the current implementation, this minimum replica limit is fixed and cannot be configured by the user. However, exposing this threshold as a configurable parameter could further enhance high availability by allowing users to define the minimum number of replicas according to the criticality of their applications.

Since StornX currently operates at the namespace level (read more in paragraph 3.1.5.1), enforcing a minimum replica count greater than one would affect all Deployments within the same namespace. This could lead to inefficient resource utilization, particularly when some stateless services are non-critical and can tolerate temporary downtime. Maintaining multiple replicas for every service would quickly consume cluster resources, especially in environments with limited capacity or strict cost constraints. Therefore, the current design avoids enforcing a higher minimum replica count until per-Deployment management is supported.

Moreover, the reason StornX evaluates all replicas of a Deployment rather than a single one is that this could lead to a misleading assessment. For example, an underutilized replica may have been created recently and has not yet received enough traffic to increase its resource usage or it may receive less traffic during the current execution interval, both of which do not accurately represent the overall load of the Deployment. This approach ensures a reliable assessment of the Deployment’s workload prevents scaling down when it is not really needed.

For `highLoadPods` StornX handles scaling in a slightly different way (lines 29-42) compared to `lowLoadPods`. It first calculates the average resource usage across **all replicas** of the Deployment (`clusterAvg`). If `clusterAvg` exceeds the upper threshold, the *OptiScaler* executes the `scaleUp` function to create a new replica at a time. Unlike the HPA, StornX does not currently allow users to configure the number of replicas created during scale-up because scaling decisions are applied at the namespace level. Allowing users to specify how many replicas should be added during scaling at this stage would affect all Deployments uniformly, potentially causing unnecessary scaling. Once per-Deployment control is supported, the scale-up mechanism can be extended to optionally create multiple replicas at a time when appropriate.

If the `clusterAvg` remains below the threshold but the replicas on a specific Node exceed it, StornX does not scale up. Instead, it shifts some of the traffic from the overloaded Node to other Nodes hosting replicas that are not overloaded. To achieve this, StornX invokes the *OptiBalancer* to redistribute traffic among replicas and across Nodes, thereby reducing the resource usage of the overloaded Node. This two-step process ensures that scaling up occurs only when the average resource usage of the Deployment exceeds the threshold, while traffic redistribution addresses Node-level overloads without creating unnecessary replicas. This approach ensures that new replicas are created only when the entire Deployment is truly overloaded. When the problem is limited to a single Node, StornX avoids unnecessary scaling and instead redistributes traffic to underutilized replicas on other Nodes. In this way, the system reduces pressure on overloaded Nodes, keeps resource usage balanced across the cluster, and prevents the waste of resources that would result from creating extra replicas that are not actually needed.

Figure 10 illustrates this process. On the left, the cluster contains two Nodes with upstream replica Pods sending traffic to downstream replicas of the green Deployment. In this example the upper threshold is set to 80%. Although the downstream replicas on Node 2 exceed this threshold and are therefore considered overloaded, the average resource usage across all five replicas remains below the threshold. Because StornX evaluates the average load across the entire Deployment, this situation does not trigger a scale-up event. Instead, StornX detects that only the replicas on Node 2 are overloaded and invokes the *OptiBalancer*. As shown on the right, traffic is redistributed so that downstream replicas on Node 1 receive more traffic than downstreams on Node2. This redistribution reduces the load on the overloaded replicas, balances traffic across Nodes, and prevents unnecessary scale-up operations.

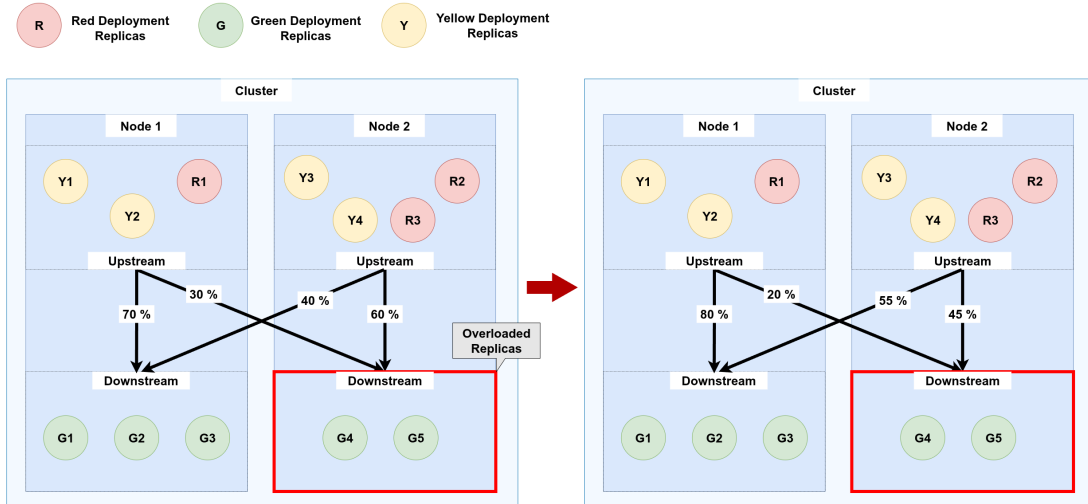


Figure 10: Node replica pods overloaded

Finally, after each scaling action StornX stores the affected Deployment in the file system (**optFile**). This file acts as a temporary record of Deployments that have recently scaled up or down. Its purpose is to delay traffic redistribution until the newly created or remaining replica Pods are fully running, since some Pods may take several seconds and in some cases up to one or more minutes to become ready. This ensures that redistribution occurs at an appropriate time rather than immediately after the scaling action (lines 6-10).

During each control cycle, StornX first checks the Deployments listed in **optFile**. If all replicas of a recorded Deployment are healthy and running, the system invokes the *OptiBalancer* to redistribute traffic among its replicas based on the new configuration. Once traffic redistribution has been completed, the corresponding Deployment entry is removed from the file, as it no longer required. In subsequent cycles, new scaling actions automatically add updated entries to **optFile**, replacing those that were already processed. In this way, **optFile** remains synchronized with the most recent scaling events and acts as a coordination mechanism between scaling and traffic management. Moreover, storing the file on disk allows StornX to preserve this information even in the event of a temporary system interruption, ensuring that Deployments pending traffic redistribution are handled reliably after recovery.

StornX itself is deployed as a Kubernetes Deployment with a single replica Pod. To persist the **optFile**, which stores the list of Deployments that have recently scaled, StornX relies on Kubernetes persistent storage. As shown in Figure 11, the StornX Pod mounts a Persistent Volume Claim (PVC) that is bound to a Persistent Volume (PV) providing the persistent storage. This design ensures that the contents of **optFile** are preserved independently of the Pod lifecycle. If the StornX Pod is restarted, rescheduled to another Node, or temporarily goes down, the information about scaled Deployments is not lost.

StornX is intentionally deployed with a single replica to maintain consistent control over scaling and traffic management decisions. Running multiple replicas would cause duplicate evaluations of the same Deployments and could trigger concurrent scale-up actions, leading to conflicting updates and resource instability. By keeping one active instance per namespace, StornX maintains stable control over scaling decisions and avoids redundant actions while relying on persistent storage to preserve state across Pod restarts.

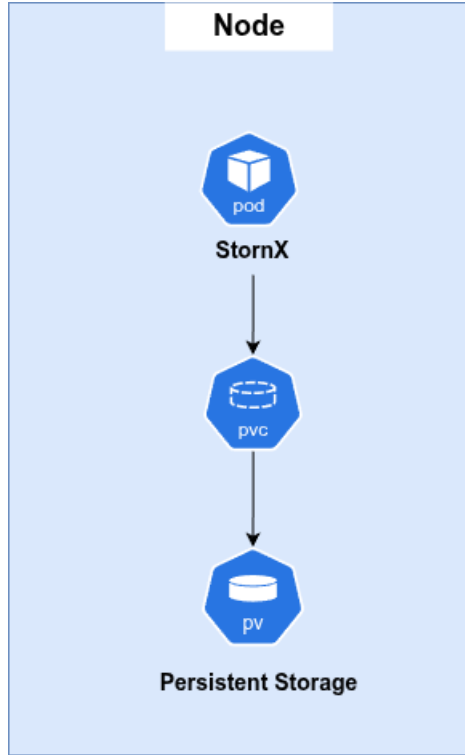


Figure 11: StornX Architecture

In summary, the main algorithm of StornX operates as a continuous control loop that periodically monitors resource utilization across Deployments and their replica Pods. During each iteration, StornX identifies replicas that are either underutilized or overloaded based on predefined thresholds and takes appropriate actions. Underutilized Deployments with more than one replica are scaled down, while overloaded Deployments trigger either scaling up or traffic redistribution actions, depending on whether all replicas or only a subset of replicas on certain Nodes are overloaded. This adaptive process ensures that resources are used efficiently while maintaining stable performance. To achieve this behavior, StornX relies on two core components: *OptiScaler* and *OptiBalancer*. *OptiScaler* is responsible for scaling and placement decisions, ensuring replicas are created or removed when necessary and scheduled on the most suitable Nodes, while *OptiBalancer* is responsible for managing traffic distribution across replicas.

4.4.1 OptiScaler

The *OptiScaler* component is designed to achieve the first two objectives outlined above: *efficient placement of replicas across Nodes and Availability Zones*, and *communication-aware placement*. It is responsible for horizontal scaling and placement decisions, ensuring that replicas are created or removed when necessary and scheduled on the most suitable Nodes. By doing so, *OptiScaler* improves fault-tolerance, balances workloads across the cluster, and minimizes communication latency between upstream and downstream replica Pods. In the following, we describe the core algorithms of *OptiScaler* and outline the reasoning behind its placement strategy.

Algorithm 2 outlines the operation of *OptiScaler* during the **scale up** process. Its goal is to select the most suitable Node for placing a new replica while satisfying resource, fault-tolerance, and communication requirements.

Algorithm 2 OptiScaler function

```
1: function OPTISCALER.SCALEUP(pod, deployment)
2:   nodes  $\leftarrow$  getNodesWithSufficientResources(pod)
3:
4:   if nodes == 0 then
5:     continue
6:   end if
7:
8:   Select fault-tolerant Nodes if available, otherwise use all candidate Nodes
9:   ftNodes  $\leftarrow$  getNodesGuaranteesFaultTolerance(nodes, pod)
10:
11:   upstreamPods  $\leftarrow$  getUpstreamDeployments(pod)
12:   downstreamPods  $\leftarrow$  getDownstreamDeployments(pod)
13:   candidateNode = " "
14:   if upstreamPods == 0 and downstreamPods == 0 then
15:     candidateNode  $\leftarrow$  getCandidateNodeLU(ftNodes)
16:   else if upstreamPods == 0 then ▷ This is a root pod
17:     candidateNode  $\leftarrow$  getCandidateNode(downstreamPods, ftNodes)
18:   else
19:     candidateNode  $\leftarrow$  getCandidateNode(upstreamPods, ftNodes)
20:   end if
21:   createReplicaPod(pod, candidateNode)
22: end function
```

In summary, the algorithm proceeds through the following steps:

1. In the first step, OptiScaler identifies the set of Nodes that have sufficient available resources to host the new replica (`getNodesWithSufficientResources`). If no suitable Node is found, the algorithm terminates. To avoid this situation, Cluster Autoscaler¹⁰ or Karpenter¹¹ can be used to provision additional Nodes (lines 2-6).
2. From the set of Nodes with sufficient resources, OptiScaler determines how replicas should be distributed across Nodes and Availability Zones (`getNodesGuaranteesFaultTolerance`) (lines 8-9). This step ensures that new replicas are placed on separate Nodes or in different Availability Zones, improving resilience against Node or Zone failures while maintaining a balanced distribution. This operation is described in detail in Section 4.4.1.1. However, in some cases, Nodes may not have available capacity in other Availability Zones or even within the same Zone to maintain resiliency against Node and Zone failures. To handle this situation, StornX proceeds by selecting the Nodes that have available resources, even if they are located in the same Zone or on Nodes that already host other replicas of the same Deployment. In such cases, the new replicas are more vulnerable to failures and there is a risk of them becoming unavailable in the event of an outage. Nevertheless, it is preferable to create these replicas to handle the incoming traffic rather than leaving them unscheduled. Leaving replicas unscheduled could lead to performance degradation, as the existing Pods may become overloaded and unable to handle the incoming traffic efficiently.
3. Once the best candidate Nodes are selected, the algorithm retrieves the upstream and downstream Deployments associated with the target replica Pods of the same Deployment (lines 11-12). These communication affinities (links) are later used (`getCandidateNode`) to reduce network latency by placing replicas closer to the Pods with which they exchange the most traffic.
4. Based on the Pod's communication links in the service graph, OptiScaler applies different strategies to select the most suitable Node (lines 13-20).

¹⁰<https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/>

¹¹<https://karpenter.sh/>

- (a) If the Pods do not have either upstream or downstream dependencies, this means that they function as monolithic services and do not communicate with other Pods. StornX is designed to handle this situation by selecting the placement of the new replica using a least-utilized (LU) strategy (lines 14-15). In this case, the new replica is placed on the Node that is the least utilized.
 - (b) For Pods with upstream or downstream connections, OptiScaler takes into account both network latency and traffic volume (RPS) between Nodes. This ensures that new replicas are placed close to the Pods they communicate with most frequently, while also minimizing latency and distributing traffic efficiently. This operation (lines 16-20) is described in detail in Section 4.4.1.2.
5. Finally, the new replica is scheduled on the selected candidate Node, completing the scaling and placement decision (line 21).

Algorithm 3 outlines the operation of OptiScaler during a **scale down** process. Its goal is to select an appropriate Node and replica Pod for removal, while preserving fault-tolerance, high availability and maintaining a balanced distribution of replicas across Nodes and Availability Zones.

Algorithm 3 OptiScaler function

```

1: function OPTISCALER.SCALEDOWN(pod, deployment)
2:   nodesWithReplicas  $\leftarrow$  uniqueNodes(deployment)
3:   ftNodes  $\leftarrow$  getNodesGuaranteesFaultToleranceForDown(nodesWithReplicas)
4:   cNode  $\leftarrow$  selectCandidateNodeForRemoval(ftNodes)
5:   deleteReplicaPod(deployment, podToDelete, cNode)            $\triangleright$  Remove replica pod
6: end function

```

The scale-down process in OptiScaler is more straightforward than scaling up. When a scale-down action is triggered, OptiScaler first determines the set of Nodes that currently host replicas of the target Deployment (line 2). From this set, it selects candidate Nodes following the same fault-tolerance principles used during scaling up (`getNodesGuaranteesFaultTolerance`) (line 3). Once a Node is chosen, one of the replica Pods running on that Node is selected for removal using a simple random selection strategy (line 4). Finally, the selected Pod is deleted (line 5). This approach helps maintain a balanced distribution of replicas across Nodes and Availability Zones while ensuring that at least one replica of the Deployment always remains active to avoid service disruption.

4.4.1.1 Efficient placement of replicas across Nodes and Availability Zones

The *efficient placement of replicas across Nodes and Availability Zones (AZs)* is the first objective of StornX. The goal is to maximize fault-tolerance and high-availability by distributing replicas across the cluster. In cloud environments, placing all replicas on a single Node or within a single AZ introduces significant risks. A Node failure or an AZ outage could bring down all replicas simultaneously, causing downtime. Conversely, spreading replicas with an inefficient manner may lead to increase latency and operational costs. Therefore, an effective placement strategy should balance fault-tolerance with performance, placing replicas in a way that protects against failures while keeping communication fast and cost-effective.

StornX achieves this through a two-phase placement algorithm that considers both the distribution of replicas across Availability Zones and the replicas on the same Node. Phase A focuses on spreading replicas across different Availability Zones to maximize fault-tolerance and high-availability, while Phase B optimizes placement within Zones to improve communication efficiency. Once Phase A has achieved its goal of spreading replicas across different Availability Zones, Phase B is executed to optimize placement within Zones. In each scheduling cycle, only one of these phases is executed at a time. They do not run in parallel and once Phase A completes, Phase B takes place. It is important to note that this strategy does not directly determine the final Node where the replica will be scheduled. Instead, its role is to identify

the set of candidate Nodes that best satisfy the system’s fault-tolerance and high-availability requirements. The final placement decision is then made in a subsequent step (Section 4.4.1.2), which also considers additional factors such as communication affinities and network latency.

In Phase A, the algorithm focuses on spreading the first replicas across different Availability Zones to ensure high-availability and fault-tolerance (Figure 12). This spreading is performed across a predefined maximum value of three Zones, or across all AZs if fewer than three exist, since most major cloud providers typically offer three Zones per region. To illustrate this strategy, consider a cluster deployed across three Availability Zones, each containing two Nodes. Initially, the Deployment has a single replica. When scaling up, the first two additional replicas should be placed in different Availability Zones whenever possible. As shown in Figure 12, the first replica is initially placed on Node1 in eu-central-1a. When the system scales up and needs to create a second replica, the algorithm selects candidate Nodes located in a different Availability Zone to maximize distribution. In this example, the candidate Nodes are Node3 and Node4 in eu-central-1b, and Node5 and Node6 in eu-central-1c. The algorithm described in Section 4.4.1.2 then selects the most suitable Node from this candidate set (Node3 in this case). When a third replica is required, the candidate Nodes are Node5 and Node6 in eu-central-1c, ensuring that the Deployment has replicas placed in all three Availability Zones. This distribution strategy ensures that the system can tolerate both Node and Zone-level failures from the very early stages of scaling. Once this initial distribution goal is achieved, the algorithm transitions to Phase B.

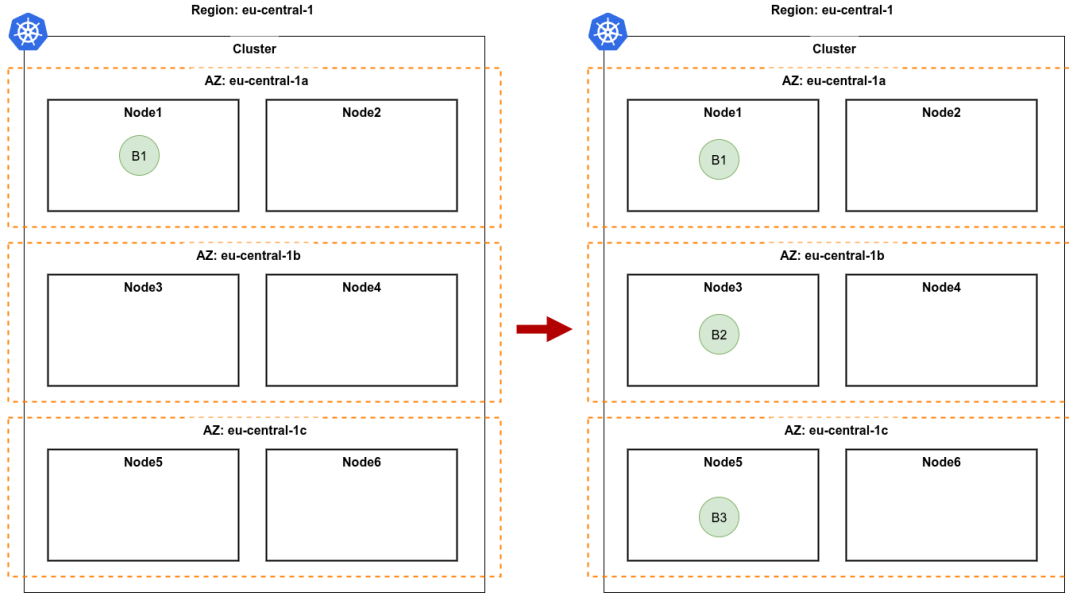


Figure 12: Phase A: spreading the first replicas across different Availability Zones for fault-tolerance and high-availability

In Phase B, the algorithm focuses on improving communication efficiency. Once replicas have been distributed across all Availability Zones, StornX no longer spreads replicas evenly but instead begins to place new replicas on Nodes that already host replicas of the same Deployment. At this point, Phase B does not decide the final placement location. Instead, its purpose is to limit the eligible Nodes based on fault-tolerance rules and the skew constraint (the skew is defined below), forming a candidate set of Nodes from which the final scheduling decision will later be made by a subsequent step (Section 4.4.1.2), which also considers additional factors such as Node-to-Node latency and traffic affinity (RPS) from upstream and downstream Pods.

By placing multiple replicas on the same Node within the same Zone, StornX ensures that Pods that frequently communicate with each other are kept close, improving overall performance. However, placing too many replicas within the same Availability Zone or on a single Node introduces the risk of reduced system availability. If that Node or Zone becomes unavailable, all replicas located there will go down at the same time, leaving the system with fewer active

replicas to handle incoming requests and potentially causing interruptions. To prevent this, StornX uses a maximum skew of two (2). This means that no Node is allowed to have more than two replicas more than any other Node. If a Node reaches this skew limit or runs out of resources, the algorithm selects other candidate Nodes within the same Availability Zone before considering Nodes in other Zones.

Figure 13 illustrates this behavior. After the first three replicas have been placed across different Availability Zones during Phase A, the new candidate Nodes are Node1, Node3, and Node5. In this example, Node3 is shown as the selected Node purely for illustration. The actual decision is not random and will be made later by the communication-aware placement step in Section 4.4.1.2, which evaluates the candidate Nodes using latency measurements and traffic affinity. When additional replicas are scheduled, and once three replicas are placed on Node3, the maximum skew is reached. If more replicas are required, Node3 is no longer eligible and the candidate set is reduced from three Nodes to two (Node1 and Node5). This ensures balanced replica distribution and maintains fault-tolerance.

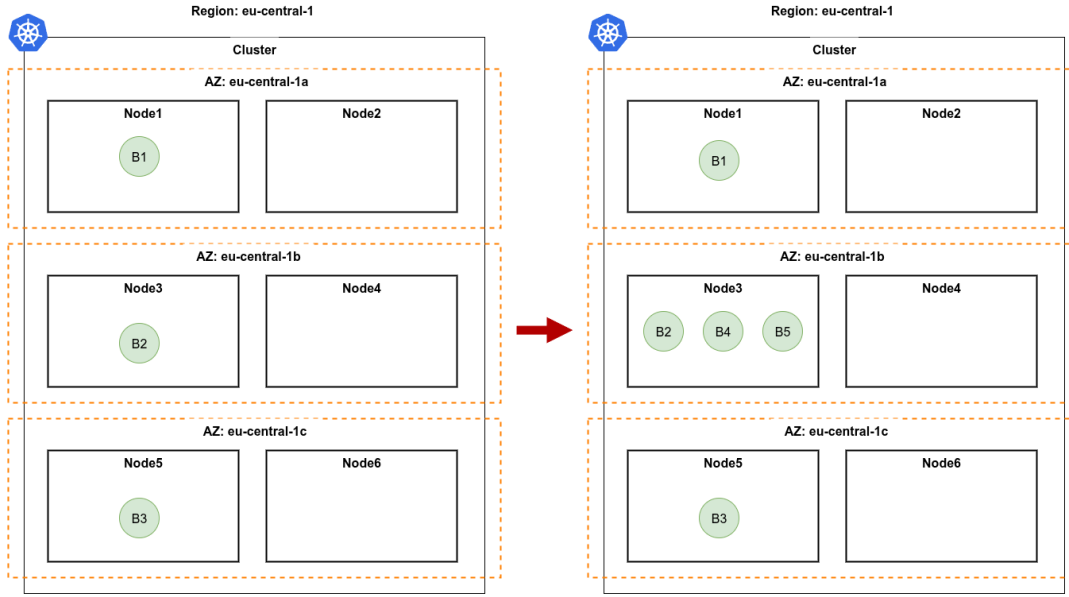


Figure 13: Phase B: co-locating replicas on existing Nodes to reduce communication overhead and costs

The two-phase logic described above applies only during scale-up. When the Deployment scales down, the objective becomes the opposite: remove replicas while still preserving fault-tolerance and a balanced distribution of Pods across Nodes and Availability Zones. To achieve this, OptiScaler follows a simpler but structured process. First, OptiScaler considers only Nodes that host more than one replica of the same Deployment, since removing the only replica on a Node would reduce fault-tolerance and make the service unavailable. If multiple such Nodes exist, the algorithm selects the Node with the highest latency from the upstream Pods. Once a Node is selected, one of its replicas is removed randomly. This approach ensures that scale-down is performed in a controlled way, preserving availability and fault-tolerance, maintaining balanced distribution, and keeping the remaining replicas on Nodes with lower latency to upstream Pods.

The function that is responsible for finding the fault-tolerance candidate Nodes is `getNodesGuaranteesFaultTolerance` and its process is described in Algorithm 4. The fault-tolerance algorithm follows a two-level decision process. It first determines the most suitable Availability Zones to preserve high availability and resilience to failures, and then selects candidate Nodes within those Zones. The output is not a single placement decision but a constrained set of Nodes that best satisfy high-availability and fault-tolerance objectives under the current system constraints.

Algorithm 4 `getNodeGuaranteesFaultTolerance`

```
1: function GETNODESGUARANTEESFAULTTOLERANCE(deployment, zonesNodes, nodesWithResources)
2:   replicasByNode  $\leftarrow$  countReplicasPerNode(deployment)
3:   replicasByZone  $\leftarrow$  mapNodesToZones(zonesNodes, replicasByNode)
4:   availableZones  $\leftarrow$  keys(zonesNodes)
5:   maxZones  $\leftarrow$  min(3, availableZones.length)
6:   zonesWithResources  $\leftarrow$  zonesHaving(nodesWithResources)
7:   skew  $\leftarrow$  3
8:    $\triangleright$  Phase A (Zones): spread across AZs up to maxZones
9:   if deployment.replicas < maxZones then
10:    candidateZones  $\leftarrow$  zonesWithNoReplicas(zonesWithResources, replicasByZone)
11:    if candidateZones.length = 0 then
12:      candidateZones  $\leftarrow$  AZsWithFewestReplicas(zonesWithResources, replicasByZone)
13:    end if
14:  else  $\triangleright$  Phase B (Zones): prefer AZs that already host replicas, apply zone
    skew check
15:    candidateZones  $\leftarrow$  AZsWithMostReplicas(zonesWithResources, replicasByZone)
16:    candidateZones  $\leftarrow$  filterZonesWithinSkew(candidateZones, replicasByZone, skew)
17:  end if
18:   $\triangleright$  Node selection within chosen zones
19:  candidateNodes  $\leftarrow$  []
20:  for each zone in candidateZones do
21:    nodesInZone  $\leftarrow$  getNodeByZone(zone)
22:    nodesInZone  $\leftarrow$  filterNodesWithResources(nodesInZone, nodesWithResources)
23:    nodesWithCounts  $\leftarrow$  attachReplicaCounts(nodesInZone, replicasByNode)
24:    hostingNodes  $\leftarrow$  filterNodesWithReplicas(nodesWithCounts)
25:    if hostingNodes.length > 0 then  $\triangleright$  Prefer nodes that already host replicas
26:      ordered  $\leftarrow$  sortDesc(hostingNodes, by = replicas)
27:    else  $\triangleright$  No replicas in this zone. Pick least loaded nodes
28:      ordered  $\leftarrow$  sortAsc(nodesWithCounts, by = replicas)
29:    end if
30:    for each node in ordered do
31:      if hasResources(node) then
32:        candidateNodes  $\leftarrow$  append(candidateNodes, node)
33:      end if
34:    end for
35:  end for
36:  return unique(candidateNodes)
37: end function
```

In more detail, the `getNodeGuaranteesFaultTolerance` function works as follows:

1. First, the algorithm counts how many replicas of the Deployment run on each Node and in each Availability Zone. The mapping `replicasByNode` stores the number of replicas per Node, while `replicasByZone` aggregates these counts per Zone (lines 2-3). It also computes the set of available Zones, and determines the maximum number of Zones to use, which is set to three or to the total number of available Zones if fewer exist. The limit of three is chosen because most major cloud providers typically offer three Availability Zones per region. The step also identifies which Zones currently have Nodes with free resources and define the skew parameter (lines 4-6). Together, these values provide an overview of how replicas are currently distributed across the cluster.
2. Next, the algorithm decides whether it should operate in Phase A or Phase B. If the total number of replicas of the Deployment is smaller than `maxZones`, Phase A is applied (line 9). In Phase A, the goal is to spread replicas across as many Zones as possible. The algorithm therefore first selects Zones with no replicas of the current Deployment (`zonesWithNoReplicas`). If none exist, it selects the Zones with the fewest replicas

- (`AZsWithFewestReplicas`) in order to keep the distribution balanced (lines 10-13).
3. If the Deployment already has at least one replica in each of the desired Zones (that is, `deployment.replicas` is greater than or equal to `maxZones`), the algorithm switches to Phase B (line 14). In Phase B, the focus is no longer on filling new Zones but on refining placement within the existing Zones. The algorithm selects the Zones with the highest number of replicas (`AZsWithMostReplicas`) and then checks that the difference in replica counts between Zones does not exceed the allowed skew (lines 15-16). This prevents a single Zone from hosting too many replicas compared to the others.
 4. After selecting the candidate Zones, the algorithm identifies the candidate Nodes within these Zones (lines 19-30). For each Zone, it first retrieves all Nodes that belong to this Zone and filters them to keep only the Nodes that have enough capacity to host an additional replica (`filterNodesWithResources`). It then splits the Nodes into two groups: Nodes that already host replicas of the Deployment (`hostingNodes`) and Nodes that do not.
 5. If a Zone already contains Nodes that host replicas of the Deployment, those Nodes are preferred. In this case, hosting Nodes are sorted in descending order by replica count (lines 23-24). If no Node in the Zone currently hosts replicas, the algorithm instead sorts all Nodes in the Zone in ascending order by their replica count (`sortAsc`), so that the least loaded Nodes are considered first (lines 25-27). In both cases, only the Nodes with sufficient available resources are added to the candidate Nodes list (lines 28-30).
 6. Finally, the algorithm returns the set of unique candidate Nodes (line 32). This set represents the Nodes that satisfy the fault-tolerance, AZ distribution, and resource availability constraints. The next step of the placement process, described in Section 4.4.1.2, will evaluate these candidate Nodes using communication metrics such as latency and traffic affinity in order to choose the final Node for the new replica.

4.4.1.2 Efficient communication-aware placement

The second objective of StornX is to place new replicas in a way that reduces cross-Node and cross-Zone communication. Once the system has identified candidate Nodes (Algorithm 4), OptiScaler focuses on selecting the most suitable Node from this set to optimize communication efficiency. Once the Node is chosen the new replica is placed on it. The main idea behind this approach is to keep Pods that exchange large amounts of traffic as close to each other as possible. Placing Pods nearby makes communication faster leading to lower response times and operational costs. To achieve this, OptiScaler collects three key metrics:

1. **Network latency:** It monitors network latency to prioritize Nodes with low latency between each other. Lower latency ensures improved response times.
2. **Communication links:** It collects the communication links (graph) of the target Deployment. This graph captures the upstream and downstream relationships between services. Upstream Deployments are those that send traffic to the target Deployment, while downstream Deployments are those that receive traffic from it. Analyzing these relationships helps identify which Pods communicate with the replica Pods of target Deployment.
3. **Requests-per-second (RPS):** The RPS measures the amount of traffic exchanged between the replica Pods of the target Deployment and its upstream and downstream Pods. Higher RPS indicates stronger communication affinities.

Using these metrics, OptiScaler evaluates each candidate Node to find the best placement for the new replica. For every candidate Node, OptiScaler considers both the network latency between Nodes and the RPS values to make the placement decision. Nodes with lower latency and higher RPS are preferred, as they indicate faster communication and stronger traffic affinity between services. The chosen Node should have high RPS values and low latency to the target Node of the Deployment. Ideally, the selected Node should also host one or more upstream

or downstream replica Pods that communicate with the replicas of target Deployment, since co-locating them reduces response times and operational costs.

OptiScaler considers both upstream and downstream Pods when deciding where to place the new replica, but only one of them is used in the score calculation. It first checks which group of Pods is responsible for the increased workload. In most cases, a Deployment scales up because it receives increased traffic from upstream services, so upstream Pods are used for scoring. Focusing on upstream Pods helps identify where most of the requests originate, allowing OptiScaler to place new replicas closer to these sources and thereby reducing response times and improving communication efficiency. If a Deployment has no upstream service then downstream Pods are used instead for placement decisions. In this case, OptiScaler prefers the Node where its replicas receives most of the outgoing requests from the target Deployment. This practically means that candidate Nodes that hosts either upstream or downstream replica Pods are preferred over Nodes that do not host any.

To determine the most suitable Node, OptiScaler uses a weighted scoring function that combines both communication affinity (measured by RPS from upstream or downstream Pods) and network latency. The score is computed between the set of candidate Nodes C , which is obtained from Algorithm 4, and the set of Nodes L which may or may not currently host upstream or downstream replica Pods. Each candidate Node $i \in C$ is a possible placement location for the new replica, while each Node $j \in L$ corresponds to a Node whose Pods may communicate with the target Deployment. By comparing every candidate i with each Node j , we can determine how suitable i is based on network latency and communication affinity. The scoring formula is defined as:

$$\text{score}_i = \text{normRPS}_j + \text{normLat}_{j \rightarrow i} \quad (1)$$

where

$$\text{normRPS}_j = \frac{\text{totalRPS} - \text{RPS}_j}{\max(1, \text{totalRPS})}, \quad (2)$$

$$\text{normLat}_{j \rightarrow i} = \frac{\text{Latency}_{j \rightarrow i}}{\max(1, \text{totalLatency})}. \quad (3)$$

- **normRPS_j** represents the normalized RPS between Node j and the Node of the replica pods of the target Deployment. In practice, this measures the amount of traffic exchanged between the Node hosting the upstream or downstream Pods and the Node hosting the replica Pods of the target Deployment. Higher values indicate stronger communication affinities between these Nodes and are preferred.
- **normLat_{j→i}** represents the normalized latency between Node j and candidate Node i . Lower values are preferred since they correspond to faster communication.

RPS_j tells us which Node_j is currently sending the most traffic to the Deployment. The StornX uses this information to understand where the load is coming from, because the new replica should be placed close to the Node that generates the most traffic. Once the system identify the Node j that sends the highest amount of traffic to the Deployment (highest affinity), it compares the latency from Node j to each candidate Node i . The Node i with the best overall score, reflecting low latency and high communication relevance, is selected as the placement location.

The candidate Node with the lowest score is selected as the optimal placement for the new replica.

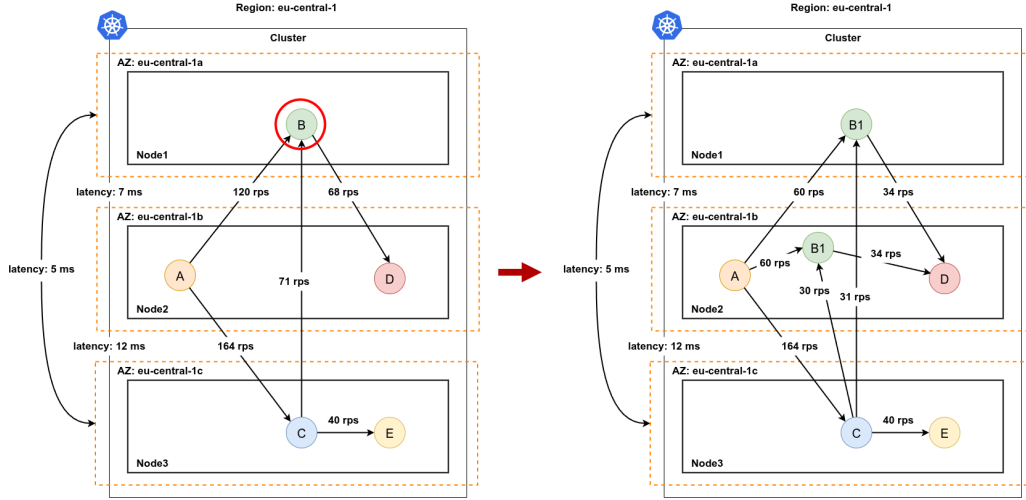


Figure 14: Replica Pod placement

Figure 14 illustrates the process of finding the most suitable with a practical example. In the initial state (left), Deployment B is overloaded, receiving traffic from the replica Pods of Deployments A and C. After applying the fault-tolerance filtering step, the candidate Nodes for placing the new replica of B are Node2 and Node3. Both of these Nodes already host one upstream Pod each. The replica Pod of the Deployment A located on Node2 and sends a higher volume of traffic to B (120 RPS) compared to replica Pod of the Deployment C on Node3 (71 RPS). The latency within the same Node is 0 ms, while the latency between Node2 and Node3 is 12 ms in both directions. Placing the new replica (B1) on Node2 therefore minimizes latency and keeps it close to the source with the strongest communication affinity, leading to faster communication and lower operational costs. As a result, Node2 is chosen for the new replica.

4.4.2 OptiBalancer

OptiBalancer is the second core component of StornX, and it addresses the third main objective of the system, which is *metric-aware traffic distribution*. While OptiScaler is responsible for scaling and placing replica Pods, OptiBalancer manages how traffic is distributed among these replicas after they have been deployed. Its goal is to route traffic toward replicas that are closer in terms of network distance and have enough available resources to handle additional load. By doing this, OptiBalancer reduces unnecessary cross-Node and cross-Zone communication, response times and prevents Pods from becoming overloaded.

When OptiBalancer is triggered, it collects three key metrics that shape its traffic distribution decisions:

1. **Resource utilization:** It monitors resource utilization, such as CPU and memory, for each replica Pod to assess the remaining capacity relative to the Pod's resource limits. This metric is used to prevent sending additional traffic to overloaded replica Pods and even to shift traffic from overloaded replicas to those that are not.
2. **Network latency:** It evaluates the latency between Nodes and prioritizes replicas that are co-located or connected by low-latency links. This ensures that traffic is sent through faster network paths, resulting in better performance and improved efficiency.
3. **Number of replicas per Node:** Nodes that host more replicas can handle a larger amount of traffic because the workload can be shared across multiple Pods. This allows them to process more requests without overloading individual Pods. OptiBalancer takes this into account and routes a larger portion of traffic to such Nodes.

OptiBalancer uses these metrics to sending larger amount of traffic to downstream Pods running on the same Node, since local routing reduces latency and avoids unnecessary cross-Node and cross-Zone communication. If local replicas become overloaded or unavailable, traffic is

gradually shifted to other replicas within the same or different Availability Zones. This dynamic adjustment keeps response times low while avoiding overload by directing traffic to replicas that still have capacity. To implement this adaptive distribution, OptiBalancer employs an algorithm that calculates traffic weights between upstream and downstream Pods. The algorithm combines latency, resource usage, and replica count to decide how traffic should be split. Once the weights are computed, they are applied to update the routing rules in real time.

The OptiBalancer operates at the Node level. All upstream Pods located on the same Node send the same amount of traffic and all downstream Pods on a Node receive the same amount of traffic. First, the algorithm determines the set of upstream Nodes u that generate traffic for a given Deployment and the set of downstream Nodes j that host the target replicas. All upstream Pods residing on the same Node are grouped together, as they send an equal amount of traffic. Similarly, all downstream Pods on the same Node are grouped together and the total incoming traffic to each Node j is calculated based on the proportion of requests it receives from all upstream Nodes. The following steps describe the complete operation of OptiBalancer:

As the first step of its operation, OptiBalancer collects runtime metrics for traffic distribution. For every downstream Node j that hosts replicas of the target Deployment, relative to an upstream Node u , OptiBalancer collects three key metrics. These metrics include the network latency between Nodes, the resource utilization of the downstream replicas and the number of downstream replica Pods belonging to the current target Deployment. The *target Deployment* refers to the Deployment whose traffic is subject to redistribution by OptiBalancer. These metrics are defined as:

$$L_j = \text{latency}(u \rightarrow j), \quad R_j = \text{load}(j), \quad n_j = \text{replicas}(j)$$

where

$$u : \text{Upstream Node that sends traffic to the Nodes hosting replicas of the target Deployment} \quad (4)$$

$$j : \text{Downstream Node that hosts replicas of the target Deployment and receives traffic from } u \quad (5)$$

$$L_j : \text{Latency from upstream Node } u \text{ to downstream Node } j \quad (6)$$

$$R_j : \text{Average resource utilization (e.g., CPU or memory) of the target Deployment replicas running on Node } j \quad (7)$$

$$n_j : \text{Number of target Deployment replicas hosted on Node } j \quad (8)$$

$$(9)$$

After collecting the necessary metrics, OptiBalancer normalizes them to ensure comparability. For every downstream Node j relative to an upstream Node u , the normalized metrics are computed as follows:

$$\hat{L}_j = \frac{L_j}{\sum_{k \in D} L_k}, \quad \hat{R}_j = \frac{R_j}{\sum_{k \in D} R_k}, \quad \hat{n}_j = \frac{n_j}{\sum_{k \in D} n_k} \quad (10)$$

where

$$D : \text{Set of all downstream Nodes that receive traffic from upstream Node } u \quad (11)$$

The traffic weight assigned to each downstream Node j is then calculated using a composite score that integrates latency, load, and replica count:

$$w_j = (1 - \min(0.9, \hat{L}_j)) \times (1 - \min(0.9, \hat{R}_j)) \times \hat{n}_j \quad (12)$$

where

$$\hat{L}_j : \text{Normalized latency between upstream and downstream Nodes,} \quad (13)$$

$$\hat{R}_j : \text{Normalized average resource load of Node } j, \quad (14)$$

$$\hat{n}_j : \text{Normalized number of replicas hosted on Node } j. \quad (15)$$

This expression increases the weight of Nodes with low latency, low resource load, and a higher number of replicas, making them more likely to receive additional traffic. To avoid extreme scoring outcomes, an upper bound of 0.9 is applied to the normalized latency and load values. Without this limit, a downstream Node with temporarily high latency or resource usage could receive a weight of zero. In that situation, OptiBalancer would stop sending traffic to the replicas of that Node. This is undesirable because its replicas would remain idle, and when conditions improve they would have no recent traffic history, making the system slow to recover and re-balance. By applying the 0.9 cap, every Node that has replicas continues to receive a small amount of traffic, even if it is temporarily slower or heavily utilized. This ensures that replicas remain active, responsive, and ready to handle more traffic once their latency or load returns to normal levels.

Once the weights are computed, they are normalized to obtain the proportion of traffic assigned to each downstream Node:

$$T_j = \frac{w_j}{\sum_{k \in D} w_k}. \quad (16)$$

To preserve locality and reduce cross-Node communication, OptiBalancer enforces a minimum local traffic ratio of $m = 0.5$. This constraint ensures that at least half of the traffic originating from an upstream Node is routed to replicas co-located on the same Node. When the number of upstream Pods on Node u is less than or equal to the number of downstream replicas on Node j ($u \leq j$), a minimum of 50% of the traffic is kept local. When this local traffic portion falls below the threshold, OptiBalancer directs more requests to local downstream replicas and proportionally reduces traffic sent to remote Nodes, then normalizes the distribution to maintain consistency. This adaptive balancing strategy ensures stable routing, minimizes unnecessary inter-Node transfers, and still allows a small portion of traffic to be distributed to other Nodes to support load balancing and fault tolerance.

OptiBalancer is executed when a new replica is created or removed to re-evaluate traffic distribution, since the number of downstream replicas has changed, or when all replicas from a Deployment on a Node become overloaded. In this case, OptiBalancer recalculates traffic weights and if needed shifts part of the traffic away from overloaded Pods to those that can still handle additional load, ensuring balanced and efficient traffic distribution.

To maintain routing stability, OptiBalancer introduces a change threshold parameter (Δ). Before applying new routing rules, it computes the total deviation between the current and target traffic distributions. If the deviation is below the threshold ($\Delta < 5\%$), the update is skipped to prevent frequent and unnecessary reconfigurations caused by minor metric fluctuations. Only when the deviation exceeds the threshold does OptiBalancer apply the new distribution, ensuring stable routing decisions and preventing traffic from constantly switching between Nodes.

4.4.2.1 OptiBalancer Operational Cases

The OptiBalancer algorithm handles different traffic redistribution scenarios depending on the role and network position of the *replicated Pod* or the Pod running on a highly loaded Node (i.e., the newly created or overloaded instance) within the application graph. A Pod may act as a *Front-end service* (receiving external requests), a *Middle-tier service* (receiving and forwarding traffic), or a *Back-end service* (receiving traffic only). The following cases illustrate how OptiBalancer adjusts traffic distribution based on these roles. In all examples, the Pods may reside on different Nodes or Zones within the cluster.

1. The replicated Pod is the Front-end Service (UI) (Figure 15)

In this case, the replicated Pod (e.g. B) represents the *Front-end* of the application, the entry point for user requests. Initially, a single Pod (B1) receives all incoming requests and forwards them to downstream components (C1 to Cn). After replication (e.g. creation of B2), OptiBalancer automatically redistributes incoming requests between B1 and B2 according to the calculated traffic weights. Both replicas then jointly forward requests to the downstream components (C1...Cn).

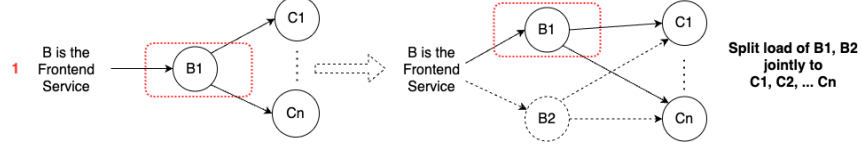


Figure 15: Case 1 - The replicated Pod acts as the Front-end service (UI).

2. The replicated Pod has one or more upstream components (Figure 16)

In this scenario, the replicated Pod (e.g., B) is not at the front of the service graph but receives requests from one or more upstream components (e.g., A1).

- (a) When B1 is replicated into B2, OptiBalancer redistributes the traffic coming from A1 between B1 and B2. This adjustment prevents B1 from becoming overloaded and improves responsiveness for the upstream service.
- (b) When B also has downstream components (C1...Cn), OptiBalancer performs a two-level balancing. It first splits the incoming load from A1 between B1 and B2, and then distributes the combined outgoing load from both replicas toward C1...Cn. This ensures coordinated scaling across multiple tiers of the application.

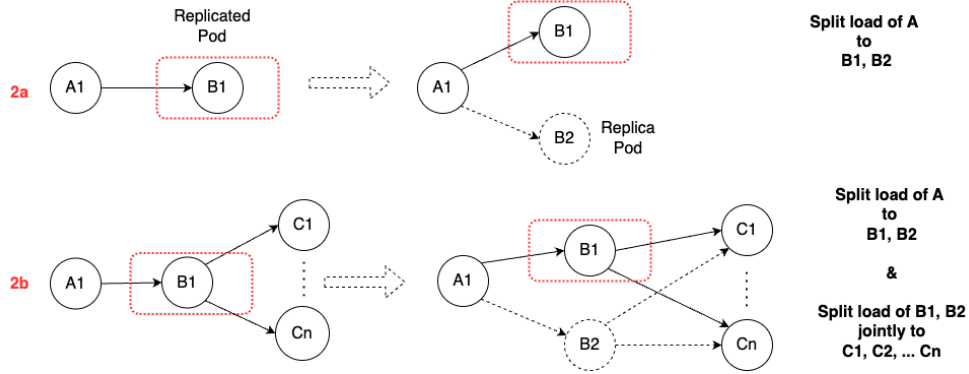


Figure 16: Case 2 - The replicated Pod has one or more upstream components.

3. The replicated Pod has multiple upstream components (Figure 17)

In this case, multiple upstream components (e.g., A1...An) send requests to the same service B.

- (a) When B1 is replicated into B2, OptiBalancer redistributes the aggregate traffic from all upstream components (A1...An) across the available replicas (B1 and B2). This balances incoming load and reduces contention on a single Pod.
- (b) If the replicated service also connects to downstream components (C1...Cn), OptiBalancer extends the adjustment in both directions, splitting load from multiple upstreams across replicas and from replicas to downstream targets. This provides full-path load balancing across the dependency chain.

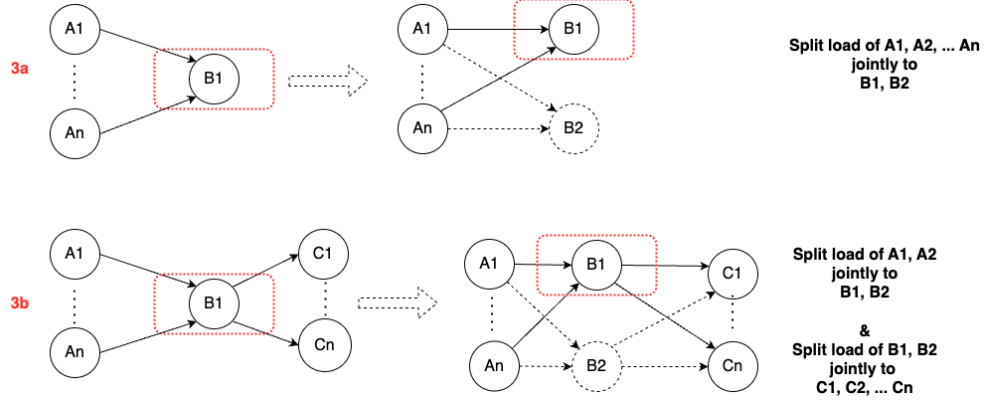


Figure 17: Case 3 - The replicated Pod has multiple upstream components.

Across all cases, OptiBalancer maintains consistency and stability by dynamically recalculating routing weights whenever a new replica is added, removed, or becomes overloaded. The system ensures that both upstream and downstream communication flows are proportionally redistributed, minimizing latency, avoiding overload, and maintaining balanced utilization throughout the service graph.

OptiBalancer redistributes traffic among replicas at the Node level using runtime metrics. It assigns higher traffic to downstream Nodes that are closer (low latency), less loaded, and host more replicas, while enforcing a minimum locality ratio and a deviation threshold to keep routing stable. Algorithm 5 describes the whole process.

Algorithm 5 OptiBalancer

```
1: function OPTIBALANCER(deployment)
2:   upstream  $\leftarrow$  getUpstreamDeployments(deployment)
3:   downstream  $\leftarrow$  getDownstreamDeployments(deployment)
4:   if upstream > 0 then
5:     TrafficSplit(upstream, deployment)  $\triangleright$  Redistribute traffic for upstream replicas
6:   end if
7:   if downstream > 0 then
8:     TrafficSplit(deployment, downstream)  $\triangleright$  Redistribute traffic for downstream replicas
9:   end if
10: end function
11:
12: function TRAFFICSPLIT(upstream, downstream)
13:   for each um in upstream do
14:     totalLatency  $\leftarrow$  0
15:     totalLoad  $\leftarrow$  0
16:     totalReplicas  $\leftarrow$  0
17:     for each dm in downstream do
18:       totalLatency  $\leftarrow$  totalLatency + dm.node.latencyum.node $\rightarrow$ dm.node
19:       totalLoad  $\leftarrow$  totalLoad + dm.node.load
20:       totalReplicas  $\leftarrow$  totalReplicas + dm.replicas
21:     end for
22:     weights  $\leftarrow$  []
23:     for each dm in downstream do
24:       normalizedLatency  $\leftarrow$   $\frac{\text{Latency}_{um.node \rightarrow dm.node}}{\text{totalLatency}}$ 
25:       normalizedLoad  $\leftarrow$   $\frac{dm.load}{\text{totalLoad}}$ 
26:       normalizedReplicas  $\leftarrow$   $\frac{dm.replicas}{\text{totalReplicas}}$ 
27:       weightdm  $\leftarrow$   $(1 - \min(0.9, \text{normalizedLatency})) \times (1 - \min(0.9, \text{normalizedLoad})) \times \text{normalizedReplicas}$ 
28:       append(weights, weightdm)
29:     end for
30:     totalWeights  $\leftarrow$   $\sum \text{weights}$ 
31:     for each dm in downstream do
32:       Tdm  $\leftarrow$   $\frac{\text{weight}_{dm}}{\text{totalWeights}}$ 
33:     end for
34:      $\triangleright$  Apply minimum locality ratio and routing stability rules
35:     Tdm  $\leftarrow$  enforceLocalRatio(um, downstream, Tdm, m = 0.5)
36:     apply  $\leftarrow$  checkDeviationThreshold(um, downstream, Tdm,  $\Delta$  = 0.05)
37:     if apply = false then
38:       continue  $\triangleright$  Skip update if the change is below threshold
39:     end if
40:     for each dm in downstream do
41:       updateTrafficSplit(um, dm, Tdm)
42:     end for
43: end function
```

The OptiBalancer Algorithm 5 operates at the Node level. All upstream Pods located on the same Node send the same amount of traffic and all downstream Pods on a Node receive the same amount of traffic.

In summary, the algorithm proceeds through the following steps:

1. First, OptiBalancer identifies the upstream Deployments and their replica Pods that send traffic to the target Deployment, as well as the downstream Deployments and their replicas that receive traffic from it (lines 2-3). These relationships determine the direction in which traffic redistribution must occur (read more in Section 4.4.2.1).

2. Next, if upstream Deployments exist, OptiBalancer redistributes traffic received by the target Deployment (`TrafficSplit(upstream, deployment)`) (lines 5-7). If downstream Deployments exist, it redistributes traffic sent from the target Deployment to other services (`TrafficSplit(deployment, downstream)`) (lines 9-11). These two redistributions are handled independently because traffic may need adjustment in one direction but not the other.
3. The `TrafficSplit` function performs the full traffic redistribution logic. It takes one upstream Node at a time and evaluates all downstream Nodes that host replicas. Based on runtime metrics such as latency, resource usage, and replica count, it computes traffic weights for each downstream Node. After calculating these weights, the function applies locality rules to keep a portion of the traffic on replicas running on the same Node. It then checks whether the new distribution differs enough from the current one to justify an update. If the change is significant, the routing rules are updated accordingly (lines 12-43).
4. Inside the `TrafficSplit` function, OptiBalancer processes one upstream Node at a time instead of individual Pods, treating all Pods on that Node as a single traffic source. This is because Pods on the same Node send the same amount of traffic and downstream Pods on that Node receive the same amount of traffic, so handling them separately would not change the result and would only add extra complexity. For each upstream Node, OptiBalancer then goes through all downstream Nodes that host replicas and sums up three quantities: the total latency, the total resource load and the total number of replicas (lines 13-21).
5. After the totals have been computed, OptiBalancer evaluates each downstream Node using the same three aspects: latency, load and number of replicas (lines 23-29). For every downstream Node it calculates normalized latency, normalized load and normalized replica count and combines them into a single weight. Nodes with low latency, higher available capacity, and more replicas receive larger weights. This ensures that replicas that are close, lightly loaded, and available in higher numbers receive most of the incoming traffic.
6. Once all weights have been computed, OptiBalancer converts them into traffic ratios for each downstream Node (lines 30-33). It does this by dividing each weight by the sum of all weights, so that all ratios T_{dm} add up to 1. These ratios express how the traffic from the current upstream Node should ideally be split among the downstream Nodes.
7. Before applying these new ratios, OptiBalancer enforces two additional rules to keep routing stable and efficient (lines 34-38). First, the minimum locality constraint is applied (`enforceLocalRatio`) so that when possible, at least half of the traffic remains on replicas running on the same Node as the upstream Pods, helping avoid unnecessary cross-Node communication. Next, the routing stability check (`checkDeviationThreshold`) compares the new ratios with the currently applied ones and determines whether the change is large enough to apply. If the difference is very small (below 5%), the update is skipped to avoid frequent minor adjustments.
8. Finally, if the deviation threshold is met, OptiBalancer updates the routing rules using the computed traffic ratios (lines 39-41).

In summary, OptiBalancer completes the overall StornX design by ensuring that once replicas are created and placed, traffic is directed to them intelligently based on resource usage, network latency, and communication patterns. This improves overall system efficiency, reduces operational costs, and ensures stable application performance under varying workloads.

5 Experimental Results

This chapter presents the experimental evaluation of the proposed StornX system and compares its performance against the Kubernetes Horizontal Pod Autoscaler (HPA) for autoscaling and the default Kubernetes scheduler for Pod placement. For load balancing, the evaluation considers both Istio’s Random algorithm, which represents the default Kubernetes load-balancing mechanism and OptTraffic. The following subsections describe the experimental infrastructure, the application benchmarks and finally present the experimental results.

5.1 Cloud Infrastructure

All experiments were conducted on the Amazon Web Services (AWS) cloud platform using the Elastic Kubernetes Service (EKS). A production-like cloud environment was created to provide realistic conditions for evaluating StornX. The cluster was deployed in the Frankfurt region (eu-central-1) and spanned three Availability Zones (eu-central-1a, eu-central-1b, and eu-central-1c) to assess the impact of replica placement and cross-zone network traffic. This multi-zone setup also enables controlled failure scenarios, where a complete Availability Zone outage is emulated by removing all worker Nodes belonging to that zone. Kubernetes was running version v1.33. Table 4 summarizes the main configuration of the experimental cluster.

Attribute	Configuration
Cloud Provider	Amazon Web Services (AWS)
Region	eu-central-1 (Frankfurt)
Availability Zones	eu-central-1a, eu-central-1b, eu-central-1c
Kubernetes Version	v1.33

Table 4: EKS Cluster Configuration

The AWS infrastructure comprised two separate node groups with different resource capacities. The first node group consisted of five **m5.xlarge** instances, each with 4 vCPUs and 16 GB of memory, representing high-capacity worker nodes. The second node group contained three **m5.large** instances, each with 2 vCPUs and 8 GB of memory, representing lower-capacity worker nodes. This heterogeneous configuration allowed the evaluation of how StornX adapts its placement strategy when nodes differ in computational capacity. Table 5 outlines the specifications of the two node groups used in the experiments.

Node Group	Instance Type	vCPUs	Memory (GB)	Node Count
Node Group 1	m5.xlarge	4	16	5
Node Group 2	m5.large	2	8	3

Table 5: Node Group Specifications

Incoming traffic was distributed through an internet-facing **AWS Application Load Balancer (ALB)**, which forwarded HTTPS requests to the EKS worker nodes across all Availability Zones. Within the cluster, a **Kubernetes Ingress Controller**¹²¹³ managed these requests, providing centralized routing and control over how traffic was directed to Kubernetes Services. The overall AWS infrastructure used for the experiments is illustrated in Figure 18.

¹²<https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>

¹³<https://kubernetes.io/docs/concepts/services-networking/ingress/>

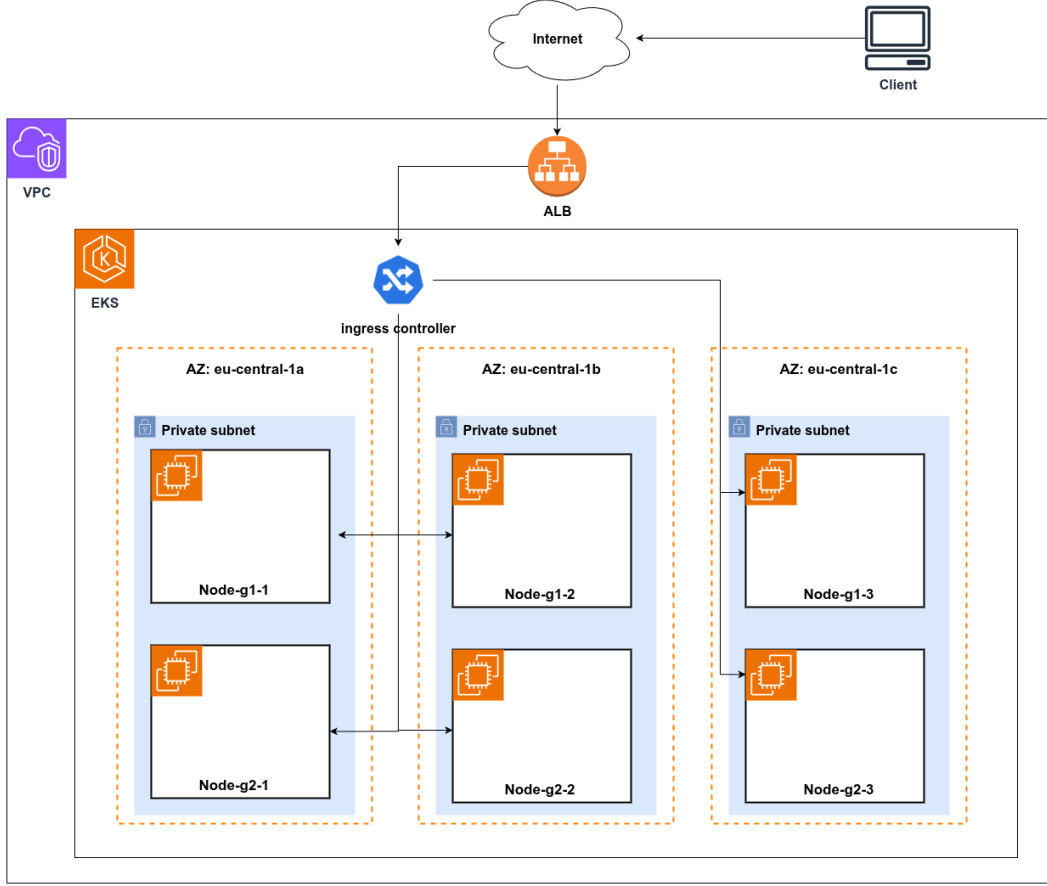


Figure 18: Overview of the AWS infrastructure used for the experiments.

All the dependency tools described in Section 4.2 were deployed inside the experimental Kubernetes cluster to support the operation of StornX. The Istio Service Mesh (version v1.24.3) provided observability, telemetry, and fine-grained traffic management between services. Prometheus was used to collect all runtime and performance metrics, including CPU and memory usage for Pods and Nodes, resource consumption, request rates, and service-to-service latency, enabling StornX to make informed autoscaling and placement decisions. Finally, Kube-NetLag measured the latency between nodes both within and across Availability Zones, providing precise insight into how network latency affects service placement.

StornX itself was deployed as a standalone Deployment within the cluster. Together, these tools formed a reliable monitoring and observability layer that enabled accurate evaluation of StornX’s performance under realistic conditions.

5.2 Benchmark Applications, Workloads, and Evaluation Tools

The present study employs two distinct applications to evaluate and validate the proposed service placement strategy, as well as to analyze system scalability and load balancing. These applications are *Online Boutique*¹⁴ and the *OpenTelemetry Demo*¹⁵. All applications were deployed in the same heterogeneous EKS cluster described in Section 5.1.

¹⁴<https://github.com/GoogleCloudPlatform/microservices-demo>

¹⁵<https://opentelemetry.io/docs/demo/>

5.2.1 Benchmark Applications

Online Boutique

The Online Boutique¹⁶ is an open-source cloud-native application developed by Google to demonstrate modern microservice architectures and deployment practices. The application simulates an e-commerce platform composed of multiple microservices that together deliver a complete online shopping experience. Its functionality includes catalog browsing, shopping cart management, order checkout, payment processing, currency conversion, product recommendations, and user notifications. Each microservice is implemented independently and communicates with others using either HTTP or gRPC protocols.

As described on the official GitHub page of the application, the Online Boutique is composed of 12 microservices written in different languages. Figure 19 provides a detailed overview of Online Boutique’s architecture. More specifically:

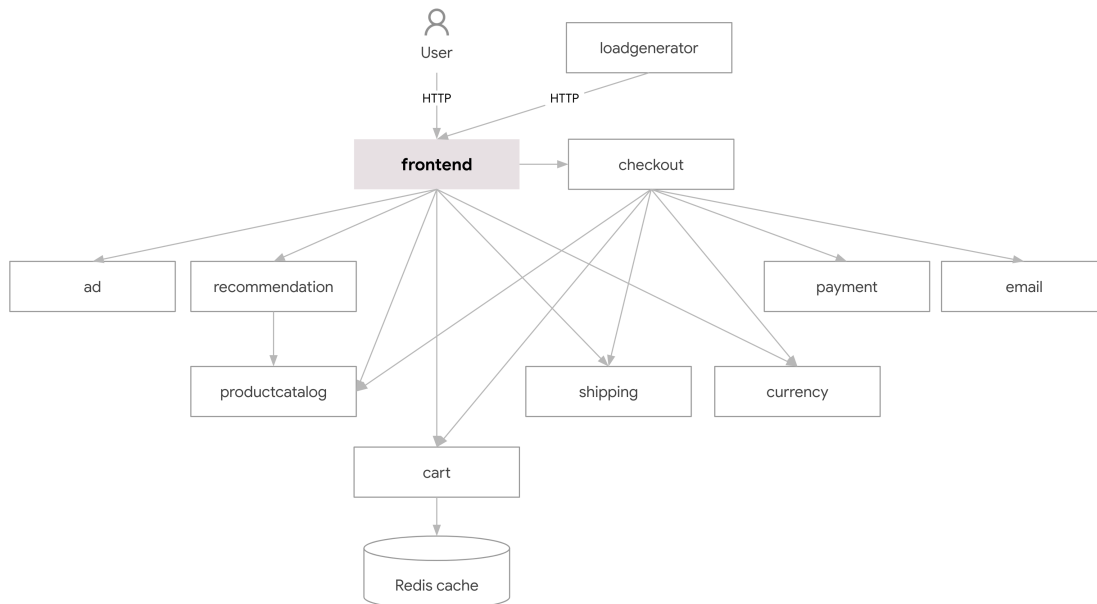


Figure 19: Online-boutique architecture

- **Frontend (Go):** Exposes an HTTP server to serve the website. Does not require signup/login and generates session IDs for all users automatically.
- **Cart Service (C#):** Stores the items in the user’s shopping cart in Redis and retrieves it.
- **Product Catalog Service (Go):** Provides the list of products from a JSON file and ability to search products and get individual products.
- **Currency Service (Node.js):** Converts one money amount to another currency. Uses real values fetched from European Central Bank. It’s the highest QPS service.
- **Payment Service (Node.js):** Charges the given credit card info (mock) with the given amount and returns a transaction ID.
- **Shipping Service (Go):** Gives shipping cost estimates based on the shopping cart. Ships items to the given address (mock).
- **Email Service (Python):** Sends users an order confirmation email (mock).
- **Checkout Service (Go):** Retrieves user cart, prepares order and orchestrates the payment, shipping and the email notification.

¹⁶<https://github.com/GoogleCloudPlatform/microservices-demo>

- **Recommendation Service (Python):** Recommends other products based on what's given in the cart.
- **Ad Service (Java):** Provides text ads based on given context words.
- **Load Generator (Python/Locust):** Continuously sends requests imitating realistic user shopping flows to the frontend.

The Online Boutique uses a Redis instance to store and retrieve the contents of user shopping carts. In the original implementation, Redis is deployed as a *Deployment* in Kubernetes rather than as a clustered setup. Since the application is not configured to support Redis clustering, it was deployed as a *StatefulSet* in our experimental configuration to prevent it from being scaled. As already described in Section 3.1.5.2, enabling scaling would require additional synchronization and data consistency mechanisms between replicas. Extending StornX to support scaling of StatefulSets is considered future work.

OpenTelemetry Demo

The OpenTelemetry Demo is designed to showcase end-to-end observability across a polyglot microservices system, and it simulates an online shopping platform where users browse products, add items to a cart, and complete purchases. It consists of multiple services built in different programming languages that communicate via HTTP and gRPC, each instrumented with OpenTelemetry to generate traces, metrics, and logs. The demo includes user-facing components such as the frontend and checkout services, as well as backend services like cart, payment, and recommendation. It also integrates observability tools including the OpenTelemetry Collector, Jaeger, Prometheus, and Grafana. This setup provides a practical, multi-language testbed for evaluating observability, performance, and system behavior under real-world conditions.

OpenTelemetry is composed of 24 microservices written in different languages. Figure 20 provides an overview of the architecture of the OpenTelemetry Demo.

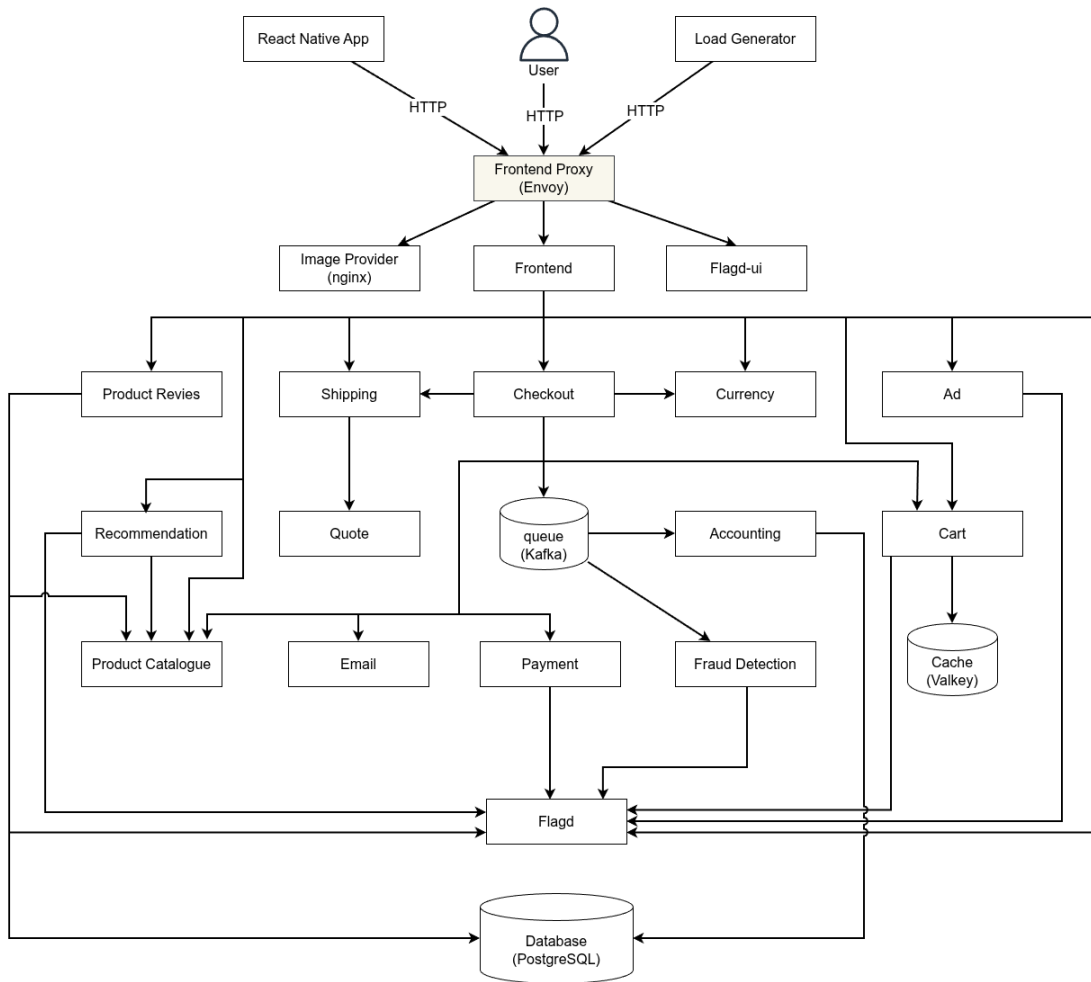


Figure 20: OpenTelemetry Demo architecture

- **Frontend (Next.js):** Web interface of the e-commerce application that allows users to browse products and place orders.
- **Frontend Proxy (Envoy):** Routes traffic to the appropriate services and handles some instrumentation.
- **Ad Service (Java):** Provides promotional product advertisements.
- **Cart Service (.NET):** Manages the user's shopping cart and stores data in a Redis-compatible Valkey instance.
- **Checkout Service (Go):** Orchestrates the order process by interacting with the cart, payment, shipping, and email services.
- **Currency Service (C++):** Converts between different currencies.
- **Email Service (Ruby):** Sends order confirmation messages to customers.
- **Payment Service (Node.js):** Processes payments.
- **Product Catalog Service (Go):** Manages product information and supports search and retrieval functions.
- **Recommendation Service (Python):** Suggests related or complementary products to users based on their shopping cart.
- **Shipping Service (Rust):** Calculates and returns shipping costs for the items in the order.

- **Quote Service (PHP)**: Provides shipping quotes to support cost estimation.
- **Accounting Service (.NET)**: Handles accounting processes related to completed orders.
- **Fraud Detection Service (Kotlin)**: Detects potential fraudulent transactions.
- **Image Provider (Nginx)**: Serves product images for display in the frontend.
- **Load Generator (Locust/Python)**: Simulates user traffic and generates realistic workloads for testing system performance.
- **Flagd (Feature Flag Service)**: Allows dynamic configuration and simulation of feature changes or failure scenarios.
- **OpenTelemetry Collector**: Central component that receives, processes, and exports telemetry data from all services.
- **Jaeger, Prometheus, Grafana, and OpenSearch**: Provide visualization, monitoring, and analysis for distributed traces, metrics, and logs.
- **Valkey**: In-memory data store used by the Cart service.
- **Kafka**: Message broker for asynchronous communication between services.

Among the components listed above Valkey and Kafka were configured as *StatefulSets* and therefore excluded from the scaling and placement analysis to maintain result consistency.

Considered Services

These benchmarks exhibit diverse communication patterns and scaling behaviors, enabling us to evaluate StornX under various workload conditions. Table 6 summarizes the total number of services and the subset of stateless services considered for autoscaling and placement decisions in each application.

Application	# Services (Total)	# Stateless Services
OpenTelemetry Demo ¹⁷	24	20
Online Boutique ¹⁸	12	10

Table 6: Benchmark applications and their stateless services.

Only the stateless microservices were included in the experimental evaluation, as they can be safely replicated and rescheduled without affecting data consistency. Stateful components such as *Valkey*, *Kafka*, and database backends were excluded from autoscaling and placement analysis.

5.2.2 Workload Generation

Traffic generation and load testing were performed using **Locust**¹⁹, a scalable open-source tool designed for simulating realistic user behavior in distributed systems. Locust was used to produce controlled and reproducible workloads with configurable request rates and traffic patterns. This allowed us to emulate different user interaction scenarios. Tables 7 and 8 present the request distributions used in Locust for the two application benchmarks.

¹⁹<https://locust.io>

Request	HTTP Method	Request Distribution (%)
View Homepage	GET	4
Browse Product	GET	53
View Cart	GET	16
Add to Cart	POST	11
Checkout	POST	5
Set Currency	POST	11

Table 7: Request distribution for Google’s Online Boutique using Locust.

Request	HTTP Method	Request Distribution (%)
View Homepage	GET	4
Flood Homepage	GET	5
Browse Product	GET	35
Get Recommendations	GET	10
Get Ads	GET	15
View Cart	GET	15
Add to Cart	POST	15
Checkout (Single item)	POST	3
Checkout (Multi-item)	POST	3

Table 8: Request distribution for OpenTelemetry Demo using Locust.

5.2.3 Infrastructure Cost Analysis Tool

To estimate the operational costs associated with inter-zone communication, we employed **Kubecost**²⁰. Kubecost provides real-time cost monitoring and allocation for Kubernetes clusters by analyzing resource usage and network traffic metrics. In our experiments, it was used to quantify the egress traffic and data transfer costs across zones. The cost measurements were integrated into the experimental analysis to assess how the placement and load-balancing strategies of the compared frameworks affect overall performance and cost.

5.3 Results

StornX is a unified framework that integrates autoscaling, scheduling, and load-balancing functionalities. To evaluate its performance, we compared it against two baseline configurations. The first baseline, called *Istio-Random* or HPA Istio-Random [25], combines the Horizontal Pod Autoscaler (HPA) for autoscaling, the default Kubernetes scheduler for pod placement, and the Istio Random load-balancing strategy, which represents Kubernetes’ default random traffic distribution mechanism.

The second baseline is OptTraffic [9], a load-balancing approach that optimizes traffic distribution across nodes by attempting to localize traffic within the same node based on the number of upstream and downstream pods, thereby reducing cross-node communication. Unlike StornX, OptTraffic does not include autoscaling or initial scheduling capabilities. Instead, it uses an exchange policy, meaning that rather than creating and placing them onto specific Nodes or removing replicas, it relocates existing Pods between Nodes to improve locality and balance the workload across the cluster. As mentioned earlier, this exchange process can cause new pods to remain temporarily unscheduled until the migration is complete. This behavior can lead to performance degradation during periods of high traffic if the cluster does not have enough resources to schedule new replicas or pods.

To ensure a fair comparison, the OptTraffic load-balancing strategy was combined with OptScaler 4.4.1, the component of StornX responsible for autoscaling and scheduling decisions. This setup made it possible to isolate the impact of the load-balancing mechanism from the other two functionalities. The combined configuration is referred to as *OptTraffic* or *OptScaler-OptTraffic* throughout the evaluation.

²⁰<https://github.com/kubecost>

The metric used for autoscaling is CPU utilization, as it reflects the computational demand handled by each service. As user requests increase, more processing power is required to handle them, leading to higher CPU usage. When this usage exceeds a defined threshold, set to 70% in our experiments, both the StornX OptiScaler and the Horizontal Pod Autoscaler (HPA) interpret it as a sign of overload and create new replicas to distribute the workload more evenly. The predefined default threshold is 80%, however this value is fully configurable. Users may adjust it according to the characteristics of their application and the desired scaling behavior, without requiring any modification to the system code. In our experimental evaluation, the threshold was intentionally set to 70% in order to trigger scaling actions earlier and more frequently. This choice increases the number of observable scaling events and allows a clearer comparison of how each approach reacts to load increases and behaves under stress conditions.

The experimental results are divided into three parts: (i) *Stress Test*, (ii) *Load Test*, and (iii) a *High Availability* evaluation under a controlled Availability Zone failure, where all Nodes of an entire AZ **are removed** during steady load. The Stress Test represents a worst case scenario in which the workload increases suddenly and reaches very high levels. This test shows how quickly each system reacts to sharp increases in traffic and how well it maintains performance when resources are under strong pressure. The Load Test represents a realistic operating scenario in which traffic increases gradually over time. This test allows us to evaluate long term stability, resource efficiency and communication overhead during normal operation. Together, these tests help distinguish how each system behaves during sudden demand spikes and how it performs during everyday workload variations.

5.3.1 Mapping concurrent users to request rate

In Locust, the workload is defined in terms of concurrent users repeatedly executing tasks that represent user activity. A task represents a single user action (e.g., viewing a page, browsing a product, or performing a checkout). Each user executes a task and then waits for a randomized period before executing the next task.

According to the Locust execution model [26], the workload generation process is driven by concurrent users executing tasks. For a simplified representation of the relationship between users and generated requests, the request rate (RPS) can be expressed as:

$$RPS = U \times \frac{R_{task}}{T_{wait}}$$

where U denotes the number of concurrent users, R_{task} the average number of HTTP requests generated per task execution, and T_{wait} the configured waiting time between task executions (between 0.5 and 2 seconds in our experiments).

This formulation highlights that the request rate increases proportionally with the number of concurrent users.

5.3.2 Stress Test Comparison

The stress test evaluation focuses on five key performance criteria, including the average 99th percentile response time (P99 latency), CPU utilization, the total number of replicas created during the test, the estimated cross-zone communication cost, and the egress traffic between zones. Egress traffic refers to the actual amount of data transferred between zones (measured in gigabytes), while the cross-zone communication cost is the operational cost charged by the cloud provider based on those transferred data. Together, these metrics provide a complete view of system behavior under heavy workloads and show how efficiently each framework uses available resources. For this test, 20 frontend replicas were initially deployed for the *Online Boutique* application and 15 for the *OpenTelemetry Demo* to handle the starting traffic and prevent early delays in service responses. All other services started with one running replica. After the test began, the autoscalers automatically created additional replicas as the system load increased. The experiments were conducted using predefined numbers of concurrent users to evaluate how each system scales and adapts under high-load conditions. In the *Online Boutique* application, the test included 800, 1000 and 1200 concurrent users and approximately 780, 970, and 1160

RPS, respectively, while in the *OpenTelemetry Demo* it involved 600, 1000, and 1400 users and approximately 560, 930, and 1300 RPS, respectively.

5.3.2.1 Response time

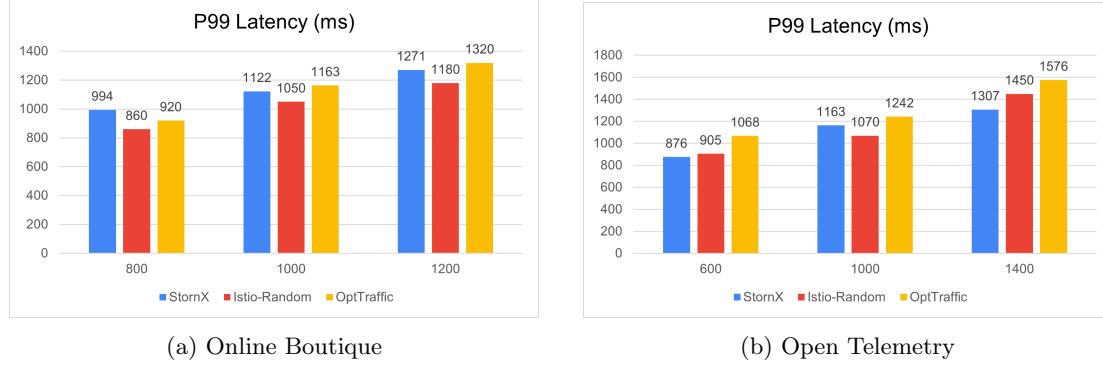


Figure 21: Stress test P99 latency results comparing StornX, Istio Random and OptTraffic

For the *Online Boutique*, HPA-Istio Random achieved the lowest P99 latency in all test scenarios. StornX followed with higher values and OptScaler-OptTraffic consistently showed the slowest responses (except for 800 users). Compared to Istio Random, StornX produced higher latency across all load levels, with differences that ranged from around 7% to 13%. When compared to OptTraffic, StornX performed better and reduced latency by up to 4%. These results indicate that the Online Boutique workload, which has a simple communication paths, benefits more from the lightweight request distribution of Istio Random than from communication aware placement.

In the *OpenTelemetry Demo*, the behavior changed significantly. StornX achieved the lowest latency at most load levels and delivered clear improvements over the other approaches. Compared to HPA-Istio Random, StornX reduced latency by around 13% at the first load level and by around 10% under the highest load, while performing slightly worse at the intermediate level. When compared to OptScaler-OptTraffic, StornX achieved consistent improvements and reduced latency from 6% up to 17%. These gains result from the communication aware placement and metric based routing decisions of StornX, which keep Pods with strong communication links close to each other and avoid overloading specific Nodes.

5.3.2.2 CPU utilization

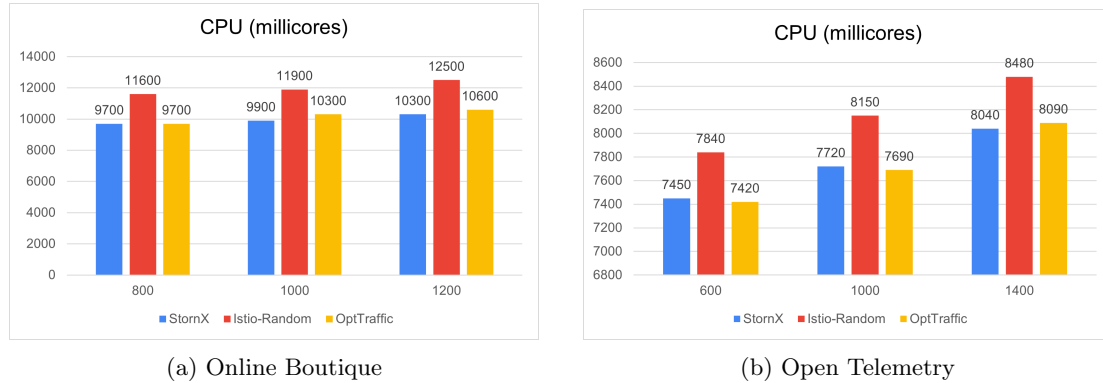


Figure 22: Stress test Replicas count & CPU utilization results comparing StornX, Istio Random and OptTraffic

The CPU results in Figures 22a and 22b show that StornX and OptiScaler-OptTraffic used almost the same amount of CPU in both applications. HPA-Istio Random consistently used more CPU, with overheads between 9% percent and 14% percent in the Online Boutique and between 5% percent and 6% in the OpenTelemetry Demo. These results indicate that StornX and OptiScaler-OptTraffic distribute workload more efficiently, while HPA-Istio Random places additional load on the cluster during high traffic.

5.3.2.3 Cost & Egress Traffic

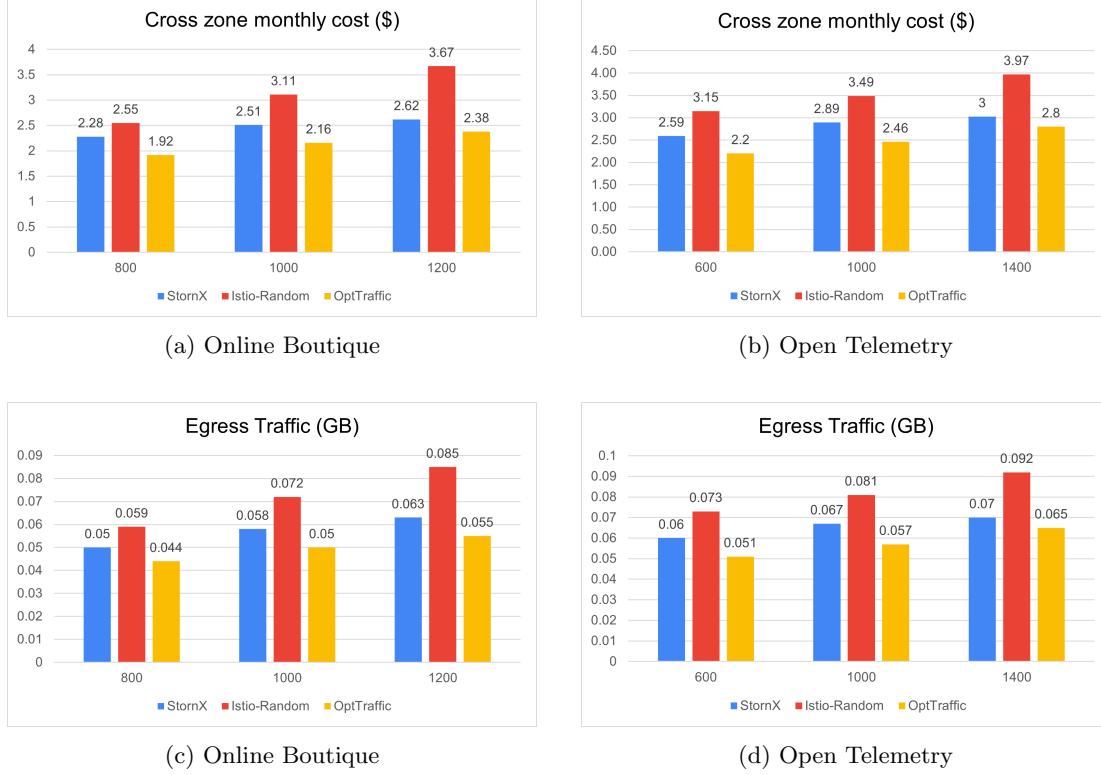


Figure 23: Stress test Cost & Egress Traffic results comparing StornX, Istio Random and OptTraffic.

Figures 23a and 23b show the estimated cross-zone communication cost, while Figures 23c and 23d illustrate the corresponding egress traffic for both benchmark applications. In all cases, the HPA-Istio Random configuration produced the highest costs and network traffic. This behavior occurs because its random load-balancing mechanism distributes requests without considering service locality, causing frequent cross-zone communication and higher data transfer expenses. In addition, the default Kubernetes scheduler, unlike StornX's OptiScaler, does not take pod affinity into account when placing pods. As a result, services that frequently communicate with each other are often deployed on different nodes or zones, which increases the distance between them and leads to higher communication overhead, egress traffic and operational costs.

The OptTraffic setup showed the lowest cost and egress traffic. This result confirms that OptTraffic is very effective at keeping most communication inside the same node, which helps to reduce cross-zone traffic and network overhead. However, this locality behavior also has drawbacks. Because traffic is concentrated on local replicas, Nodes can become overloaded if the local replicas are not sufficient to handle the incoming load. Unlike StornX, OptTraffic does not consider CPU usage or latency when deciding whether to redirect traffic to other Nodes. As a result, while it keeps costs low, it may suffer from increased response time under high load conditions.

StornX had slightly higher cost and egress traffic than OptTraffic. This is because StornX

also considers CPU usage when distributing traffic. When the pods within the same Node becomes overloaded, it may redirect some traffic to other Nodes to avoid delays and keep the system stable. Even though this causes a small increase in communication cost, it helps maintain smoother performance during high load.

5.3.2.4 Replicas

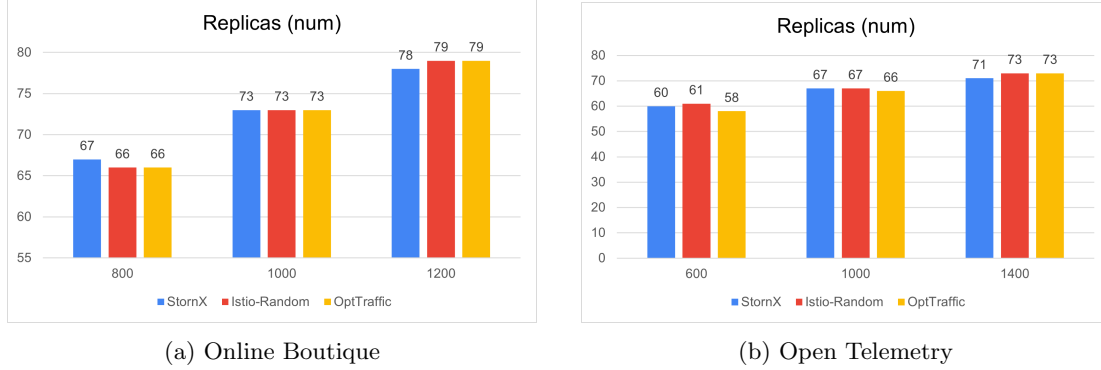


Figure 24: Stress test Replicas count results comparing StornX, Istio Random and OptTraffic

Figures 24a and 24b show that the number of replicas was almost the same for all three systems during the stress test. In both the *Online Boutique* and the *OpenTelemetry Demo*, StornX HPA Istio-Random, and OptScaler-OptTraffic scaled in a very similar way as the workload increased. The small differences in replica counts show that all autoscalers reacted similarly to CPU pressure and created new replicas at nearly the same rate.

5.3.2.5 Summary of Stress Test Comparison

The stress test results show clear differences in how the three systems behave under heavy workload. StornX provided the most balanced performance across both benchmark applications. It achieved low and stable latency in the OpenTelemetry Demo and maintained efficient CPU usage in both applications. These results arise from its communication-aware placement and metric-based routing, which direct traffic toward Pods that are close to upstream services and away from overloaded Nodes. This approach reduces queuing delays and distributes work more evenly across the cluster.

HPA-Istio Random reacted quickly to CPU spikes and created replicas at almost the same rate as the other systems. Its simple random distribution strategy produced good latency for the Online Boutique, where the communication graph is small and most services follow short request paths. However, the same strategy caused high egress traffic and communication cost in both applications, because it does not consider locality or the communication structure of the system. As a result, it frequently routed requests across Zones and created unnecessary network overhead.

OptScaler-OptTraffic showed the lowest communication cost and the lowest egress traffic in both applications. This happens because OptTraffic strongly favors intra Node communication. It routes most requests to replicas that run on the same Node as the caller and avoids sending traffic to other Nodes or Zones. This behavior reduces cross Zone transfers and lowers operational cost. At the same time, it concentrates a large amount of traffic on a single Node and may overload Pods more quickly when the initial placement is not ideal.

Overall, StornX demonstrated the most effective balance between performance, stability, and cost efficiency. In addition to its latency and resource benefits, StornX also showed improved resilience to failures. Because it distributes replicas and traffic more evenly across Nodes and Availability Zones, the system avoids creating single points of failure and is less affected when a Node becomes overloaded or temporarily unavailable. This behavior contrasts with OptTraffic, where traffic concentration on a single Node increases the risk that a failure or slowdown

will immediately affect performance. HPA-Istio Random distributes load broadly but does so without considering service relationships or locality, which improves fault tolerance but introduces unnecessary communication overhead. By combining resource utilization, latency metrics, and communication-aware routing, StornX maintains stable performance, adapts smoothly to changing workload conditions, and continues operating reliably even under high load or partial failures.

5.3.3 Load Test Comparison

The load test evaluation aims to assess the performance and efficiency of each framework under realistic and gradually changing workloads. Unlike the stress test, which pushes the system beyond its capacity limits, the load test checks how well each framework handles normal changes in user traffic while keeping performance stable. Five main metrics were analyzed: the 99th percentile response time (P99 latency), CPU utilization, total infrastructure cost, egress traffic across zones and the number of created replicas. Together, these indicators provide a comprehensive view of how efficiently each system balances responsiveness, scalability, and communication overhead during typical operation.

The tests were performed using a Gaussian-shaped workload pattern, where the traffic gradually increases until it reaches a stable high load. For the *Online Boutique* application, the load gradually increased to a maximum of 1600 concurrent users. In the case of the *OpenTelemetry Demo*, the test involved up to 1400 users and ran continuously for approximately one hour and thirty minutes, allowing sufficient time to observe system stability and autoscaling behavior. For this test, all services started with a single replica Pod. This experimental setup enables a fair comparison between StornX, HPA Istio-Random, and OptiScaler-OptTraffic in terms of scalability, cost efficiency, and overall performance under normal operating conditions.

5.3.3.1 Gaussian workload profile

The load tests follow a Gaussian-shaped workload pattern, where the number of concurrent users increases gradually over time until reaching a peak value and then remains stable for a sustained period. This workload pattern is designed to emulate realistic traffic growth and allow the autoscaling mechanisms to react under steady and increasing load conditions.

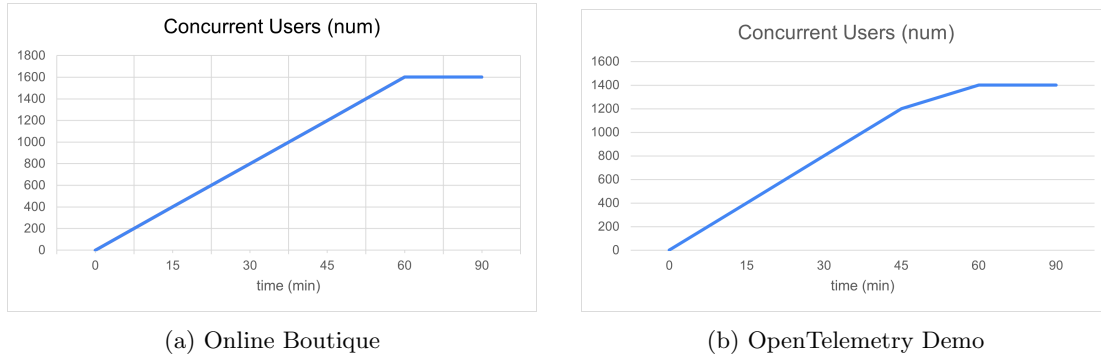


Figure 25: Gaussian-shaped workload used in the load tests, showing the evolution of the number of concurrent users over time for both benchmark applications.

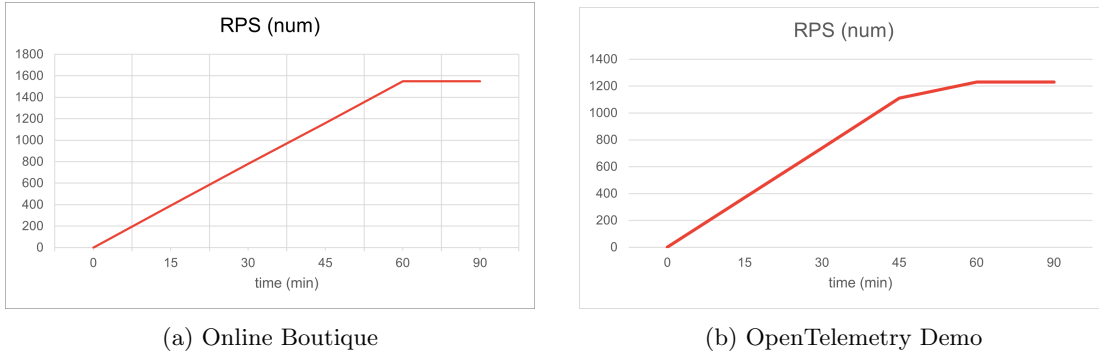


Figure 26: Estimated RPS over time during the Gaussian load test, derived from concurrent users using the Locust workload model.

Both load tests ran for approximately 1 hour and 30 minutes, allowing sufficient time for the systems to reach a steady operating state and for autoscaling decisions to take effect. During the first 60 minutes, the number of concurrent users increases gradually, leading to a proportional increase in the generated RPS. After reaching the peak workload, the user load remains stable, allowing the systems to operate under sustained high load and enabling the evaluation of steady-state performance and autoscaling behavior.

5.3.3.2 Response time

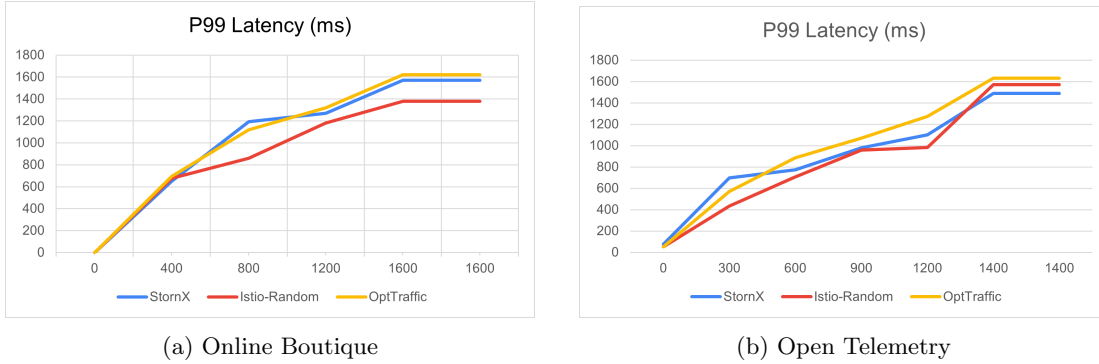


Figure 27: Load test P99 latency results comparing StornX, Istio Random and OptTraffic

In the *Online Boutique* load test, HPA-Istio Random delivered the lowest latency at almost every load level. At 400 users its values were only slightly below those of StornX, but as the load increased the gap grew, with HPA-Istio Random maintaining a clear advantage at 800, 1200 and 1600 users. OptiScaler-OptTraffic recorded the highest latency across the test. These results show that, under smooth and gradually increasing load, the simple communication structure of the application favors Istio's lightweight request distribution, while StornX remained stable without sudden spikes and stayed consistently better than OptTraffic.

The *OpenTelemetry Demo* showed a different pattern. At low and medium load, HPA-Istio Random kept a small advantage over StornX, but as the system approached higher load levels StornX became the best performer. At 1200 and 1400 users, it achieved lower latency than both HPA-Istio-Random and OptiScaler-OptTraffic, reducing delays by about 5% compared to Istio and by around 10% compared to OptTraffic. These results indicate that, under moderate and gradually changing load, random traffic distribution can achieve slightly lower latency when sufficient resources are already available. At higher load levels, the placement and routing decisions made by StornX prevented certain Nodes from becoming overloaded, which helped keep latency lower during the final stages of the test.

5.3.3.3 CPU

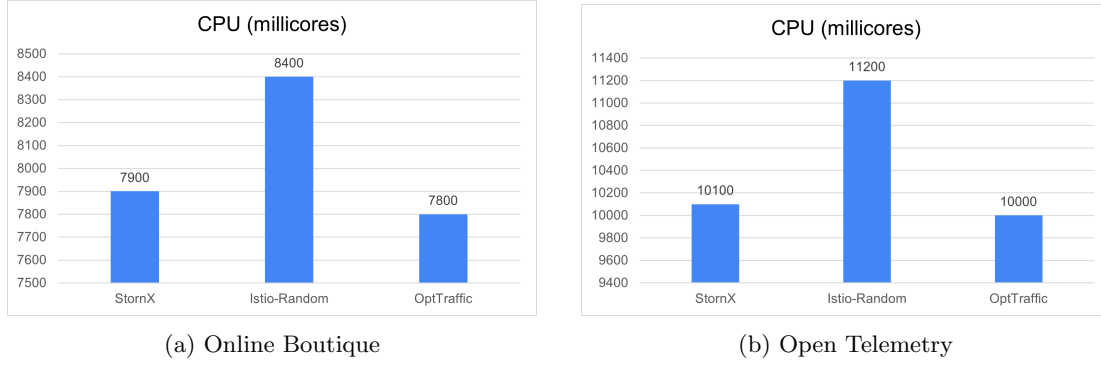


Figure 28: Load test CPU results comparing StornX, Istio Random and OptTraffic

The CPU utilization results in Figures 28a and 28b show how each system managed the growing workload during the load test. In the *Online Boutique* application, StornX recorded an average CPU usage of about 10100 millicores, which is slightly lower than the 10800 millicores observed for HPA-Istio Random. The OptiScaler-OptTraffic setup maintained the lowest CPU usage at around 10000 millicores.

The *OpenTelemetry Demo* followed a similar pattern. Here, StornX used about 10.100 millicores, compared to 11.200 millicores for HPA-Istio Random and 10.000 millicores for OptiScaler-OptTraffic. Although OptTraffic consumed slightly less CPU, this came with the trade-off of higher latency, suggesting that it reacted more slowly to performance degradation and network congestion. StornX, on the other hand maintained a balanced CPU utilization pattern, keeping resource usage near optimal levels while preventing large latency increases.

5.3.3.4 Cost & Egress Traffic

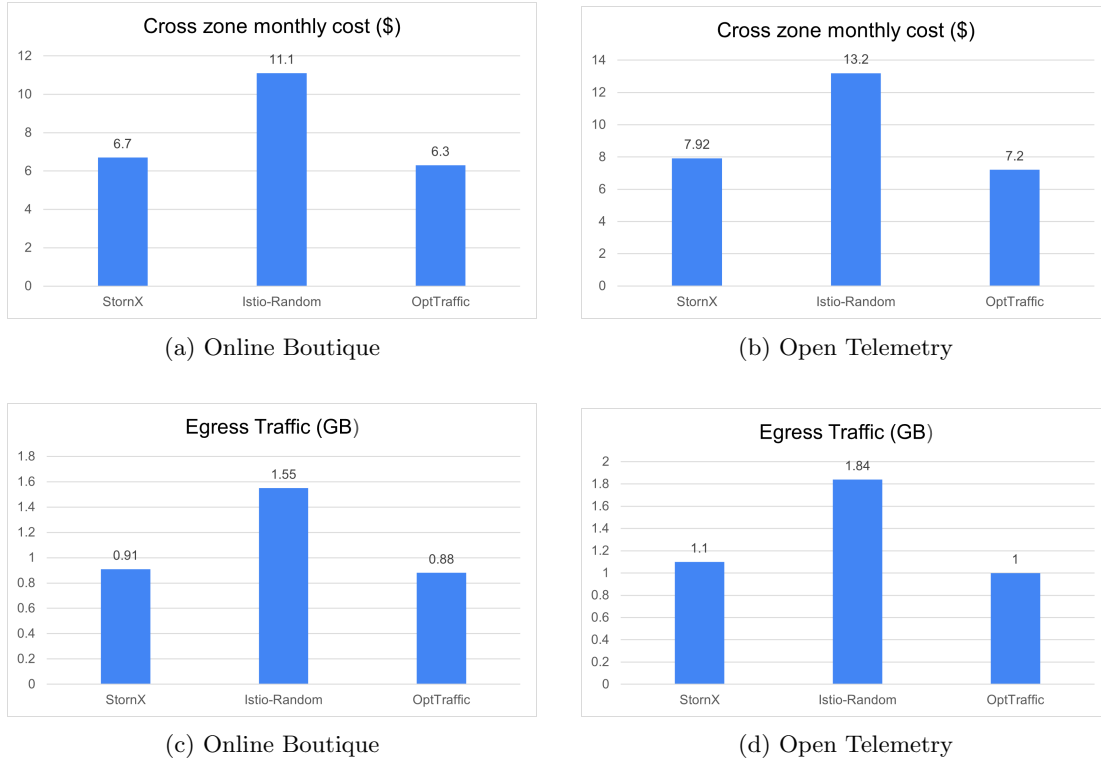


Figure 29: Load test Cost & Egress Traffic results comparing StornX, Istio Random and OptTraffic.

Figures 29a and 29b present the estimated montly cross-zone communication cost, while Figures 29c and 29d show the corresponding egress traffic. In the *Online Boutique* application, StornX reduced the total communication cost by about 40% compared to HPA-Istio Random (6.7 \$ vs. 11.1 \$), mainly due to its communication-aware scheduling and its ability to distribute traffic based on network latency. OptiScaler-OptTraffic achieved a slightly lower cost (6.3 \$) by maintaining most communication within the same node. Egress traffic followed a similar trend, with StornX recording 0.91 GB compared to 1.55 GB for HPA-Istio Random and 0.88 GB for OptiScaler-OptTraffic.

A comparable pattern was observed in the *OpenTelemetry Demo*, where StornX achieved a cost of 7.92 \$ versus 13.2 \$ for HPA Istio-Random and 7.2 \$ for OptiScaler-OptTraffic. Egress traffic was also lower, with values of 1.1 GB, 1.84 GB, and 1 GB, respectively. Although OptiScaler-OptTraffic slightly reduced cost by focusing on local traffic, StornX offered a more balanced outcome, combining near-optimized cost and egress levels with lower latency and smoother scaling. Overall, StornX proved effective at maintaining cost-efficient communication patterns without sacrificing performance.

5.3.3.5 Replicas

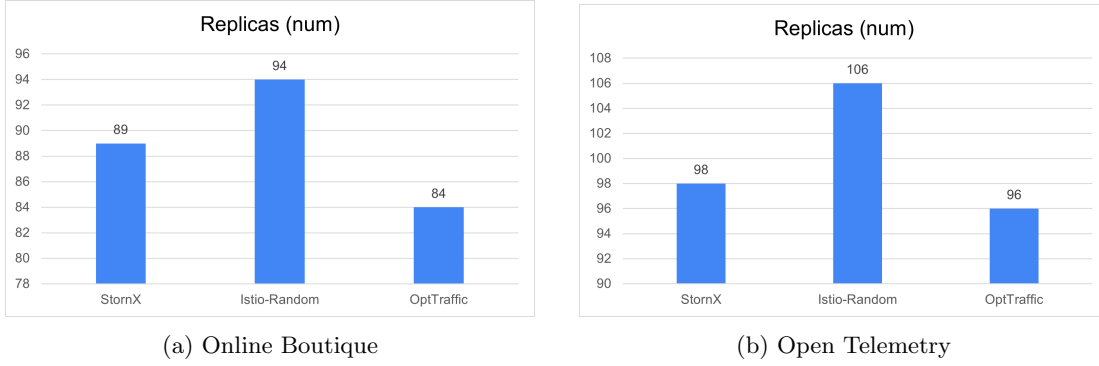


Figure 30: Load test Replicas count results comparing StornX, Istio Random and OptTraffic

Figures 30a and 30b illustrate the number of replicas created by each system during the load test. In the *Online Boutique* application, StornX deployed 89 replicas, compared to 94 with HPA-Istio Random and 84 with OptiScaler-OptTraffic. In the *OpenTelemetry Demo*, StornX scaled to 98 replicas, while HPA-Istio Random reached 106 and OptiScaler-OptTraffic 96. These results show that StornX reduced the total number of replicas by approximately 6% compared to HPA-Istio Random and maintained a similar replica count to OptiScaler-OptTraffic, differing by less than 3%.

5.3.3.6 Summary of Load Test Comparison

The load test results show that all three systems, StornX, HPA-Istio Random, and OptiScaler-OptTraffic, handled changing workloads effectively, were able to handle gradually increasing workloads, but each exhibited different strengths and limitations.

StornX maintained stable and consistent performance across all tests, showing a good balance between responsiveness and efficiency. Although it did not always reach the lowest latency, it consistently achieved lower cost and egress traffic compared to HPA-Istio Random and delivered smoother performance as system load increased. It also deployed fewer replicas than HPA-Istio Random, demonstrating that it scaled only when required and avoided unnecessary resource allocation. Its metric-aware design, which considers both resource utilization and latency, allowed it to adapt intelligently to changing workload and network conditions, keeping performance steady and resource use efficient.

The HPA-Istio Random configuration achieved the lowest latency in most experiments. Its random traffic distribution worked efficiently under moderate and predictable load patterns. However, this configuration also deployed more replicas than StornX, leading to higher CPU consumption and significantly greater cross-zone communication costs. This behavior indicates less efficient resource utilization, as the system tended to overprovision resources in response to fluctuating workloads rather than adapting smoothly to changing demand. This shows that the system added extra replicas to maintain performance rather than optimizing existing placement or traffic behavior.

The OptiScaler-OptTraffic setup was the most cost-efficient overall, as its load-balancing policy minimized cross-zone communication. It also maintained slightly fewer replicas than StornX which contributed to its lower total cost. However, this strict locality also created performance trade-offs. Under high load, traffic routed on local replicas that eventually became overloaded, resulting in increased latency. Since OptTraffic updates its routing distribution only after replicas are created or removed, it reacts more slowly to overload conditions. As a result, the system may operate efficiently but with reduced responsiveness under sustained or rapidly changing workloads.

Overall, the load test demonstrates that the systems optimize different objectives: HPA-Istio Random prioritizes responsiveness, OptTraffic prioritizes cost and locality, and StornX provides a balanced compromise between performance, efficiency, and stability under realistic and varying traffic patterns. Beyond performance, StornX's balanced traffic distribution across multiple Nodes and Zones provides an additional advantage in terms of fault tolerance and high availabil-

ity. Unlike OptTraffic, which routes most traffic to replicas running on the same Node, StornX maintains multiple active communication paths across Nodes and Zones. This allows the system to continue operating with less disruption if a Node or Zone fails, whereas locality-dependent approaches may experience noticeable delays before recovering or redistributing traffic.

5.3.4 High Availability Under Availability Zone Failure

This section evaluates the high availability characteristics of the examined systems under a controlled Availability Zone (AZ) failure. The goal of this experiment is to assess how each framework reacts to a sudden loss of infrastructure capacity while the system is operating under steady load, and how quickly it recovers to stable performance.

5.3.4.1 Experimental Setup

To evaluate high availability, we perform controlled failure scenarios in which a complete Availability Zone outage is emulated **by removing all worker Nodes belonging to a single AZ**, since inducing an actual zone outage is not feasible. The experiments are conducted under steady workload conditions to ensure that any observed performance degradation is caused by the failure rather than changes in the applied workload.

For the Online Boutique application, a constant workload of 300 concurrent users is applied, while for the OpenTelemetry Demo the workload is fixed at 2000 concurrent users. In both cases, traffic is generated using Locust and maintained until the system reaches a stable operating state and the P99 latency converges to its minimum achievable value for the given workload. Before introducing the failure, the system is allowed to reach a stable state, where the number of replicas no longer changes and response times have stabilized.

Once the system reaches a steady operating state, all Nodes of one Availability Zone are removed, resulting in the loss of approximately 30% of the active replicas. The workload remains unchanged throughout the experiment. System behavior is observed over a ten-minute window to capture both the immediate impact of the failure and the subsequent recovery process.

System behavior is evaluated using two end-to-end metrics: the 99th percentile response time (P99 latency), which captures user-perceived performance degradation, and the error rate, which reflects the fraction of failed requests during and after the failure. The error rate is defined as the percentage of client requests that result in failed responses, including HTTP error status codes and requests that time out due to unavailable or overloaded service endpoints.

In the following figures, the x axis represents the time elapsed in minutes since the Availability Zone failure, allowing us to observe both the immediate impact of the failure and the recovery behavior over time.

Each high availability experiment runs for approximately one and a half hours. During the initial phase, a steady workload is applied and maintained until the system reaches a stable operating state, with a fixed number of replicas and stable response times. Once steady-state operation is achieved, an Availability Zone failure is injected by removing all worker Nodes belonging to a single zone. The system behavior is then analyzed over a ten-minute observation window following the failure, which captures both the immediate impact and the recovery process. This ten-minute window is used solely for analysis and does not represent the total duration of the experiment.

5.3.4.2 Online Boutique Results

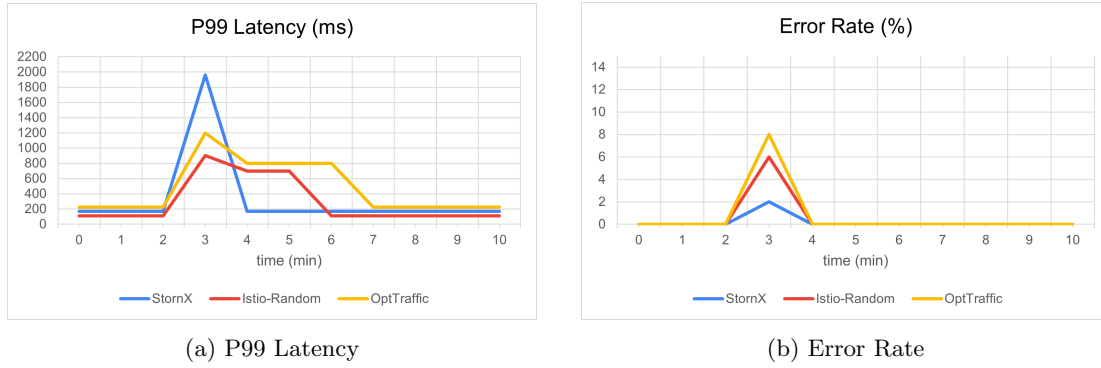


Figure 31: P99 Latency and Error Rate results comparing StornX, Istio Random and OptTraffic on an AZ failure for Online Boutique

Figure 31 presents the P99 latency and error rate observed over time during the Availability Zone failure for the Online Boutique application. Before the failure, all three systems exhibit stable latency and zero error rate. When the AZ failure occurs, all systems experience a latency spike and a short burst of errors due to connection losses and the sudden removal of service endpoints.

StornX shows a sharp but short-lived latency spike, reaching its peak immediately after the failure. However, latency rapidly returns to baseline levels within a short time window. The error rate also increases briefly but remains lower than the other systems and quickly drops back to zero. This behavior indicates that StornX reacts aggressively to the failure by rapidly redistributing traffic among the remaining replicas, minimizing the duration of degraded performance.

HPA-Istio Random exhibits a smaller initial latency spike compared to StornX, but latency remains elevated for a longer period before stabilizing. After the Availability Zone failure, traffic is redistributed randomly across the remaining replicas without accounting for their current load or performance. As a result, replicas that are already operating close to their capacity limits receive the same amount of traffic as lightly loaded ones, leading to sustained delays. The error rate spike is higher than that of StornX, as traffic continues to be routed to replicas that become overloaded immediately after the failure. Because Istio Random does not monitor runtime performance metrics such as latency or CPU utilization, it cannot prioritize healthy replicas, resulting in slower recovery.

OptiScaler-OptTraffic experiences both a high latency spike and a long recovery period. After the failure, latency remains high for several minutes before returning to baseline. The error rate spike is the highest among the three systems. This behavior is caused by OptTraffic's strong preference for locality, which concentrates traffic on a limited subset of replicas. When a large fraction of these replicas disappears due to the Availability Zone failure, the remaining replicas become overloaded. Traffic is then redistributed slowly, since routing decisions are updated only when scaling events occur, rather than in response to immediate performance changes.

5.3.4.3 OpenTelemetry Demo Results

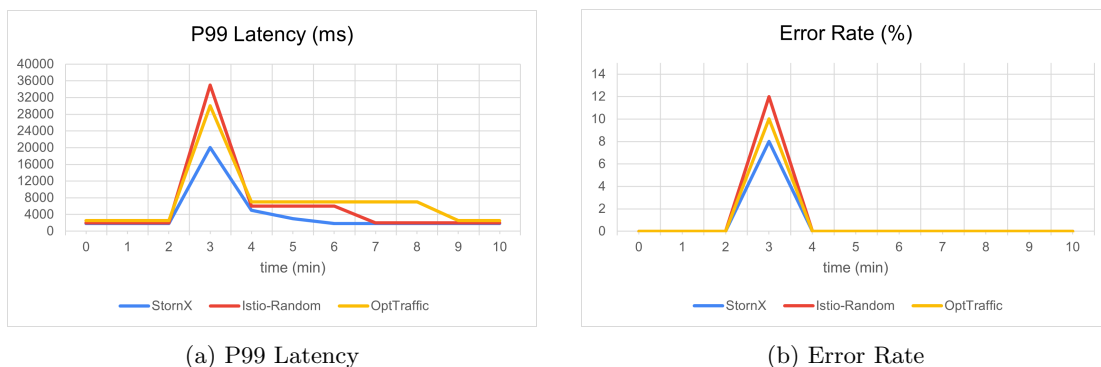


Figure 32: P99 Latency and Error Rate results comparing StornX, Istio Random and OptTraffic on an AZ failure, for OpenTelemetry

Figure 32 shows the corresponding results for the OpenTelemetry Demo, which features a larger number of services and deeper inter-service communication chains.

As expected, the failure has a stronger impact across all systems compared to the Online Boutique, resulting in significantly higher latency spikes and error rates.

StornX again demonstrates fast recovery. When the zone fails, some replicas are lost, and the remaining replicas suddenly receive more traffic, causing a short increase in latency and errors. StornX quickly detects which of the remaining replicas become overloaded and reduces the traffic sent to them, redirecting requests to other healthy replicas in the cluster. As replicas in the failed zone are rescheduled and become available again, StornX gradually increases the traffic sent to them based on their observed performance. As a result, latency and error rates return to normal levels within a short time.

HPA-Istio Random exhibits the highest peak latency among the three systems and the highest error rate during the failure. While the system eventually stabilizes, both latency and errors persist longer than in StornX. Random traffic distribution causes requests to be forwarded to replicas that are already under stress, leading to an increasing impact of failure between multiple services.

OptiScaler-OptTraffic experiences both a high latency spike and a long recovery period. The latency remains high for several minutes before returning to baseline. The error rate spike is the highest among the three systems. This happens because OptTraffic strongly favors locality, causing a large part of the redirected traffic to be handled by replicas that are already under heavy load. When many replicas are lost due to the zone failure, the remaining replicas must process significantly more traffic, increasing response time and error rates. Performance improves only after additional replicas are rescheduled or created by autoscaling.

The failed requests observed in the high availability experiments are primarily caused by the injected Availability Zone failure rather than by sustained system overload. When an entire Availability Zone is removed, a significant fraction of the service replicas become unavailable instantaneously, leading to dropped connections, request timeouts, and temporary overload of the remaining replicas. As traffic redistribution and replica rescheduling take place, a short burst of failed requests is expected.

In contrast, the load and Gaussian workload experiments presented earlier do not involve infrastructure failures and apply gradual workload variations. As a result, failed requests are negligible in those experiments and were not a primary metric of interest.

5.3.4.4 Summary of High Availability Evaluation

The high availability experiments evaluate how StornX, HPA-Istio Random, and OptiScaler-OptTraffic respond to a complete Availability Zone failure under steady workload conditions. In all experiments, the removal of an entire zone leads to an immediate loss of approximately 30% of active replicas, causing a short-term increase in latency and error rates across all systems. However, the duration and severity of this degradation differ significantly depending on the

traffic management strategy.

StornX consistently demonstrates the fastest recovery in both benchmark applications. Although it experiences an initial latency and error spike due to the sudden loss of replicas, these effects are short-lived. By dynamically adjusting traffic distribution among the remaining replicas based on observed latency and CPU utilization, StornX avoids sustained overload on individual services. As replicas in the failed zone are rescheduled and become available again, traffic is gradually rebalanced, allowing the system to return to stable performance within a short time window.

HPA-Istio Random redistributes traffic randomly across the remaining replicas after the failure. While this approach avoids extreme load concentration, it does not account for differences in replica load or performance. As a result, replicas that are already under stress receive the same amount of traffic as lightly loaded ones, leading to longer-lasting latency and error spikes compared to StornX. Recovery is therefore slower, particularly in applications with deeper service dependency chains.

OptTraffic shows the slowest recovery and the most severe degradation during Availability Zone failures. Its strong preference for locality concentrates traffic on a limited set of replicas. When a large fraction of these replicas is lost, the remaining replicas become heavily overloaded. Since routing adjustments in OptTraffic are primarily tied to scaling events rather than immediate performance changes, traffic redistribution occurs slowly, resulting in high latency and increased error rates until additional replicas are rescheduled or created.

Overall, the high availability evaluation highlights that adaptive, performance-aware traffic management is critical for fast recovery from large-scale failures. While HPA-Istio Random and OptTraffic offer baseline resilience, StornX achieves better high availability by rapidly redistributing load, limiting performance degradation, and enabling faster stabilization after Availability Zone failures.

5.3.5 Overall Summary of Experimental Results

The evaluation of StornX, HPA-Istio Random, and OptScaler-OptTraffic across both the stress and load tests shows that each system offers different strengths depending on the application characteristics and the type of workload. StornX consistently provided a balanced combination of latency, resource usage, and communication cost. Although it did not always achieve the lowest latency under simpler workloads, it maintained stable performance without sudden spikes, avoided unnecessary Node-level congestion, and kept CPU usage close to optimized levels in both applications.

HPA-Istio Random performed well in scenarios where the communication paths were short and the workload increased smoothly. In the Online Boutique application, it achieved the lowest latency in most cases due to its lightweight traffic distribution approach. However, this came at the cost of significantly higher cross-zone communication and egress traffic, which resulted in increased operational overhead during both the stress and load tests. Its CPU usage was also consistently higher than that of the other systems.

OptScaler-OptTraffic demonstrated the lowest communication cost and egress traffic across all experiments, since its routing strategy favors Node-local communication. This behavior keeps data transfer overhead low but can also concentrate traffic on specific Nodes, which led to higher latency when the load increased. Despite using slightly less CPU than StornX in some cases, it responded more slowly to rising performance pressure and showed the highest latency in most stress and load test scenarios.

Overall, StornX delivered the most balanced behavior, combining efficient resource usage with low communication cost and stable latency. Beyond performance under normal and high load, the high availability evaluation showed that StornX also recovers faster from large-scale failures, such as the loss of an entire Availability Zone. By dynamically redistributing traffic among remaining replicas based on runtime performance metrics, StornX limits the duration of service degradation and restores stable operation more quickly than the baseline approaches. HPA-Istio Random reacted quickly to load changes but introduced substantial communication overhead, while OptTraffic minimized cost at the expense of responsiveness and load distribution. In addition to performance, StornX demonstrated stronger high-availability characteristics compared to OptTraffic. Because it actively balances communication across replicas, Nodes, and

Zones rather than relying on strict locality, it avoids single-Node dependency and can continue operating smoothly when individual replicas or Nodes degrade or fail. This makes StornX not only efficient under varying workload conditions but also more resilient under real operational disruptions.

5.4 StornX Performance

StornX’s internal operation cycle involves collecting metrics, making autoscaling and scheduling decisions, and updating traffic rules. In our experiments, the complete decision loop for the *Online Boutique* application took approximately 30-40 seconds when five (5) concurrent services required autoscaling actions, while the *OpenTelemetry Demo* required around one minute to complete when more than eight (8) concurrent services needed autoscaling actions. This longer processing time is due to the larger number of services and the higher level of inter-service communication in OpenTelemetry, which increase the complexity of scheduling and traffic updates. Although these values are acceptable for a prototype, they highlight a scalability limitation that should be improved in future versions of StornX. If this delay increases further in larger applications, it could lead to system degradation and reduced performance, as scaling and scheduling actions may occur too slowly to respond to fast workload changes. In contrast, the HPA operates on a single-service basis, allowing it to respond faster to load variations, which makes it more efficient for isolated scaling decisions.

Currently, StornX operates at the namespace level, managing all Deployments within the same namespace as a single group. This means that when a decision cycle runs, it collects data from every Deployment in the namespace and applies scaling, scheduling, and traffic rules globally. While this design simplifies management, it increases processing time as the number of services grows. Handling each Deployment individually instead of processing the entire namespace at once would significantly reduce the execution time of each cycle and improve responsiveness, especially for applications with many microservices.

For this reason, it is recommended to run a separate instance of StornX for each namespace rather than managing multiple namespaces within the same deployment. This design helps shorten the decision cycle by isolating monitoring and decision-making processes, which reduces processing overhead and improves scalability. In future versions, we plan to extend StornX to support per-service control, allowing users to selectively enable or disable autoscaling, scheduling, and traffic management for individual Deployments within a namespace. This enhancement will make the system more flexible, decrease unnecessary computation, and further enhance performance in larger and more complex environments.

Finally, we evaluated the resource consumption of StornX. Although it performs coordinated autoscaling, scheduling, and traffic management across multiple services, it requires approximately 150 millicore CPU and 300 MiB of memory during the experiments.

6 Conclusion

This thesis presented StornX, a unified framework that integrates autoscaling, scheduling, and load balancing to improve the performance, scalability and cost efficiency of microservice-based applications in Kubernetes environments. The system extends traditional autoscaling and scheduling mechanisms by incorporating network latency and service runtime and communication metrics into its decision process. In addition to scaling and placement, StornX introduces an adaptive traffic management layer that dynamically adjusts load balancing rules based on real-time performance data. By directing requests toward less loaded and topologically closer service replicas, it minimizes cross-zone communication and reduces network delays. Through this combination of autoscaling, metrics-aware scheduling and load balancing, StornX provides a coordinated orchestration model that achieves a good balance between responsiveness, stability, and infrastructure cost.

The evaluation used two widely adopted microservice benchmarks, Online Boutique and the OpenTelemetry Demo, deployed on a heterogeneous Amazon EKS cluster across three Availability Zones. Both stress and load tests were conducted to assess system performance. StornX was compared against two baseline configurations. The first baseline, HPA with the default Kubernetes scheduler and Istio Random load balancing, represents the standard configuration where HPA performs autoscaling. The second baseline, OptiScaler-OptTraffic, combines the StornX autoscaling and placement logic with the OptTraffic load balancer, which prioritizes local traffic distribution to reduce communication between zones and infrastructure cost.

The experimental results demonstrated that StornX achieved the most balanced overall performance across all scenarios. Under heavy workloads, it maintained low and stable latency, efficient CPU utilization, and significantly reduced cross-zone communication cost compared to HPA Istio-Random, while also deploying fewer replicas. This indicates that StornX scales only when necessary, preventing redundant resource usage and maintaining efficient operation, even as demand increased. Although OptiScaler-OptTraffic minimized cost more aggressively, it did so at the expense of higher latency and slower response to changing load. Based on the observed routing patterns, StornX distributes requests across multiple Nodes, while OptTraffic keeps most traffic local to a single Node. This design difference suggests that StornX may handle traffic redistribution more gracefully, if a Node becomes overloaded or unavailable, since multiple replicas are already active and receiving traffic. In addition, the high availability evaluation under Availability Zone failures showed that StornX recovers faster and with lower performance degradation, compared to the baseline approaches. By dynamically redistributing traffic based on runtime performance metrics, StornX avoids sustained overload on the remaining replicas and restores stable operation within a shorter time window. This behavior highlights that the coordinated integration of autoscaling, placement, and adaptive traffic management not only improves performance and cost efficiency, but also enhances system robustness under large-scale infrastructure failures. Overall, the integration of CPU utilization, latency-aware, and communication-aware routing enables StornX to achieve an effective balance between performance, scalability, and cost efficiency, while potentially supporting more stable behavior under changing runtime conditions.

The experiments also revealed a scalability limitation in the current prototype. The complete decision cycle took about 30-40 seconds for smaller applications and nearly one minute for larger ones, because the system monitors and processes all Deployments within a namespace at once. This overhead may delay scaling decisions and temporarily affect performance in large environments. Future versions of StornX should include per-service control, so that users can apply autoscaling, scheduling, and load-balancing rules to specific Deployments. This enhancement would reduce processing time, improve responsiveness, and support more efficient operation in complex applications.

Although this thesis has provided valuable insights and optimization strategies for autoscaling, service placement and traffic distribution, there are several areas that can be explored in future research.

One important improvement would be to introduce fine-grained control mechanisms that allow users to enable or disable StornX functionalities for each microservice individually. This would minimize unnecessary computations and improve responsiveness in applications with many services.

Another important step would be to extend StornX to support StatefulSets and other workloads that require persistent storage, enabling coordinated scaling and scheduling for databases, message brokers and similar components. A particular challenge arises, because StatefulSets use persistent volumes that are not shared between Pods. Future work should explore how traffic splitting can be applied, when requests depend on specific data stored in a Pod's volume.

StornX could also be extended to work across multiple clusters located in different regions. This would make it possible to move or recreate Pods between clusters in order to reduce latency and improve performance. In this case, the traffic split mechanism would also need to distribute traffic across clusters, not only within a single one. Using real-time network latency metrics between Nodes or provider-supplied latency data could make scheduling and autoscaling decisions more accurate. In multi-cluster environments, communication delays are usually higher and less stable than within a single cluster. This makes such environments a good test for StornX, as its ability to adapt to latency changes would become even more important.

Another direction for future work is to evaluate StornX in edge and fog computing environments. Running Nodes at these layers and placing user-facing services closer to end users can further reduce response time. Since edge and fog systems often experience higher and more variable latency, they would also demonstrate how effectively StornX can adjust to changing network conditions.

In addition, future work could focus on implementing recovery-aware placement strategies. When a Node fails and later recovers, the system could redistribute the replicas so that the deployment graph remains as close as possible to its previous structure. This could involve exchanging Pods between Nodes to maintain optimized communication patterns.

Finally, future work could also involve the integration of predictive models that learn workload and latency patterns over time. By anticipating future demand, StornX could proactively adjust scaling and placement decisions, further reducing variations in response time and improving cost optimization under changing conditions.

In conclusion, StornX introduces a unified, metric-aware approach to managing Kubernetes microservices that bridges the gap between performance optimization and cost efficiency. The results show that it can dynamically adapt to different workload and network conditions, while maintaining stable performance and efficient resource utilization.

References

- [1] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Springer, Cham, 2017. doi: 10.1007/978-3-319-67425-4_12.
- [2] L. Wojciechowski *et al.*, “NetMARKS: Network Metrics-AwaRe Kubernetes Scheduler Powered by Service Mesh,” in *Proc. IEEE INFOCOM – IEEE Conference on Computer Communications*, Vancouver, BC, Canada, 2021, pp. 1–9. doi: 10.1109/INFOCOM42981.2021.9488670.
- [3] Z. Wu, Y. Deng, H. Feng, Y. Zhou, and G. Min, “Blender: A Traffic-Aware Container Placement for Containerized Data Centers,” in *Proc. Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Grenoble, France, 2021, pp. 986–989. doi: 10.23919/DATE51398.2021.9474067.
- [4] J. Zhou, X. Li, Q. Wang, X. Qin, W. Miao, and J. Tian, “Balancing Load: An Adaptive Traffic Management Scheme for Microservices,” in *Proc. IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, Nanjing, China, 2023, pp. 641–648. doi: 10.1109/ICPADS56603.2022.00089.
- [5] X. Li, J. Zhou, X. Wei, D. Li, Z. Qian, J. Wu, X. Qin, and S. Lu, “Topology-Aware Scheduling Framework for Microservice Applications in Cloud,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1635–1649, 2023. doi: 10.1109/TPDS.2023.3238751.
- [6] E. G. M. Petrakis, V. Skevakis, P. Eliades, A. Aznavouridis, and K. Tsakos, “ModSoft-HP: Fuzzy Microservices Placement in Kubernetes,” *Electronics*, Special Issue: Parallel and Distributed Cloud, Edge and Fog Computing: Latest Advances and Prospects, vol. 13, no. 1, p. 65, 2024. doi: 10.3390/electronics13010065.
- [7] A. Prountzos and E. G. M. Petrakis, “DeFog: Dynamic Micro-Service Placement in Hybrid Cloud-Fog-Edge Infrastructures,” *International Journal of Web and Grid Services*, vol. 20, no. 3, pp. 266–291, July 2024.
- [8] A. Datta, G. M. Hassan, and A. Alelyani, “Optimizing Cloud Performance: A Microservice Scheduling Strategy for Enhanced Fault-Tolerance, Reduced Network Traffic, and Lower Latency,” *IEEE Access*, 2024. doi: 10.1109/ACCESS.2024.3373316.
- [9] X. Zhu, Y. Li, L. Yao, Z. Qi, Y. Xu, P. Wang, W. Wang, and X. Zhu, “On Optimizing Traffic Scheduling for Multi-replica Containerized Microservices,” in *Proc. 52nd International Conference on Parallel Processing (ICPP ’23)*, Association for Computing Machinery, New York, NY, USA, 2023, pp. 358–368. doi: 10.1145/3605573.3605646.
- [10] H. Park, H. Kim, D. Kim, G. Lim, and H. Yu, “Optimizing Traffic Allocation for Multi-replica Microservice Deployments in Edge Cloud,” in *Service-Oriented Computing. ICSOC 2024*, W. Gaaloul, M. Sheng, Q. Yu, and S. Yangui, Eds., Lecture Notes in Computer Science, vol. 15404. Springer, Singapore, 2025. doi: 10.1007/978-981-96-0805-8_7.
- [11] Q. M. Nguyen, L. A. Phan, and T. Kim, “Load-Balancing of Kubernetes-Based Edge Computing Infrastructure Using Resource Adaptive Proxy,” *Sensors*, vol. 22, no. 8, p. 2869, Apr. 2022. doi: 10.3390/s22082869.
- [12] F. Fayaza, “Service Oriented Architecture in Enterprise Application,” arXiv preprint arXiv:2112.08012, 2021. doi: 10.48550/arXiv.2112.08012.
- [13] Kubernetes, “Kubernetes Documentation: Concepts Overview,” Available: <https://kubernetes.io/docs/concepts/overview/>. [Accessed: Oct. 2025].
- [14] G. Yu, P. Chen, and Z. Zheng, “Microscaler: Cost-Effective Scaling for Microservice Applications in the Cloud With an Online Learning Approach,” *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 1100–1116, Apr.–Jun. 2022. doi: 10.1109/TCC.2020.2985352.

- [15] A. Kwan, J. Wong, H.-A. Jacobsen, and V. Muthusamy, “HyScale: Hybrid and Network Scaling of Dockerized Microservices in Cloud Data Centres,” in *Proc. IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, Dallas, TX, USA, 2019, pp. 80–90. doi: 10.1109/ICDCS.2019.00017.
- [16] M. Xu, L. Yang, Y. Wang, C. Gao, L. Wen, G. Xu, L. Zhang, K. Ye, and C. Xu, “Practice of Alibaba cloud on elastic resource provisioning for large-scale microservices cluster,” *Software: Practice and Experience*, vol. 54, no. 1, pp. 39–57, 2024. doi: 10.1002/spe.3271.
- [17] S. Luo, H. Xu, K. Ye, G. Xu, L. Zhang, J. He, G. Yang, and C. Xu, “Erms: Efficient Resource Management for Shared Microservices with SLA Guarantees,” in *Proc. 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’23)*, Association for Computing Machinery, New York, NY, USA, 2023, pp. 62–77. doi: 10.1145/3567955.3567964.
- [18] B. Cai, B. Wang, M. Yang, and Q. Guo, “AutoMan: Resource-efficient provisioning with tail latency guarantees for microservices,” *Future Generation Computer Systems*, vol. 143, pp. 61–75, 2023. doi: 10.1016/j.future.2023.01.014.
- [19] Horizontal Pod Autoscaler, “Horizontal Pod Autoscaling,” Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. [Accessed: Oct. 2025].
- [20] Amazon Web Services, “What is a Service Mesh?,” Available: <https://aws.amazon.com/what-is/service-mesh/>. [Accessed: Oct. 2025].
- [21] Istio, “What is Istio?,” Available: <https://istio.io/latest/docs/overview/what-is-istio/>. [Accessed: Oct. 2025].
- [22] Prometheus, “Overview,” Available: <https://prometheus.io/docs/introduction/overview/>. [Accessed: Oct. 2025].
- [23] Amazon Web Services, “Amazon EC2 On-Demand Pricing,” Available: <https://aws.amazon.com/ec2/pricing/on-demand/>. [Accessed: Oct. 2025].
- [24] Wikipedia, “Netperf,” Available: <https://en.wikipedia.org/wiki/Netperf>. [Accessed: Oct. 2025].
- [25] Istio, “Load Balancing Options,” Available: <https://istio.io/latest/docs/reference/config/networking/destination-rule/>. [Accessed: Oct. 2025].
- [26] Locust, “Writing a Locustfile,” Available: <https://docs.locust.io/en/stable/writing-a-locustfile.html>. [Accessed: Oct. 2025].